Apprendre Python en autonomie

Cours, illustrations et exercices corrigés pour apprendre seul·e

Romain Tavenard

octobre 2026

CONTENTS

1 Variables, types et opérations **9**
- 1.1 Un tout premier programme . . . 9
- 1.2 Les variables . . . 10
- 1.3 L'affectation en détail . . . 13
- 1.4 Les opérations sur les nombres . . . 15
- 1.5 Opérations sur les chaînes et conversions . . . 17
- 1.6 Lire un message d'erreur . . . 18
- 1.7 Exercices . . . 21
- 1.8 Bilan du chapitre . . . 22

2 Les conditions **23**
- 2.1 Les booléens et les comparaisons . . . 23
- 2.2 L'instruction `if` . . . 25
- 2.3 `else` : sinon... . . . 26
- 2.4 `elif` : sinon si... . . . 26
- 2.5 Conditions imbriquées . . . 28
- 2.6 Exemple guidé : le prix d'un billet de train . . . 30
- 2.7 Exercices . . . 31
- 2.8 Bilan du chapitre . . . 31

3 Les boucles **33**
- 3.1 La boucle `while` . . . 34
- 3.2 La boucle `for` . . . 36
- 3.3 Les accumulateurs . . . 38
- 3.4 Quelle boucle choisir ? . . . 39
- 3.5 Boucles imbriquées . . . 39
- 3.6 Exercices . . . 40
- 3.7 Bilan du chapitre . . . 42

4 Les fonctions et les modules **43**
- 4.1 Des fonctions, vous en utilisez déjà . . . 43
- 4.2 Définir ses propres fonctions . . . 44
- 4.3 Arguments nommés et valeurs par défaut . . . 48
- 4.4 Variables locales . . . 49
- 4.5 Les modules . . . 50
- 4.6 Exercices . . . 52

4.7 Bilan du chapitre . . . 54

5 Les listes **55**
5.1 Créer une liste . . . 56
5.2 Accéder aux éléments . . . 56
5.3 Parcourir une liste . . . 59
5.4 Manipuler une liste . . . 61
5.5 Copier une liste . . . 64
5.6 Bonus : les listes en compréhension . . . 66
5.7 Super bonus : parcourir plusieurs listes en même temps . . . 66
5.8 Exercices de synthèse . . . 67
5.9 Bilan du chapitre . . . 68

6 Les chaînes de caractères **71**
6.1 Conversion d'une chaîne en nombre . . . 72
6.2 Analogie avec les listes . . . 72
6.3 Principales méthodes des chaînes . . . 74
6.4 Mettre en forme des chaînes de caractères . . . 76
6.5 Exercices . . . 78
6.6 Bilan du chapitre . . . 78

7 Dictionnaires, *tuples* et *sets* **81**
7.1 Créer un dictionnaire . . . 82
7.2 Lire le contenu d'un dictionnaire . . . 82
7.3 Modifier le contenu d'un dictionnaire . . . 83
7.4 Parcourir un dictionnaire . . . 84
7.5 Exemple guidé : compter des occurrences . . . 85
7.6 Des valeurs plus riches . . . 85
7.7 Deux autres collections : les *tuples* et les *sets* . . . 87
7.8 Exercices . . . 89
7.9 Bilan du chapitre . . . 90

8 Déboguer et tester son code **91**
8.1 Les erreurs en Python . . . 92
8.2 Trouver un *bug* : une démarche méthodique . . . 94
8.3 Les tests unitaires . . . 95
8.4 Exercices . . . 97
8.5 Bilan du chapitre . . . 98

9 Lire et écrire des fichiers **99**
9.1 Lister les fichiers d'un dossier avec le module `os` . . . 100
9.2 Fichiers textuels et encodage . . . 101
9.3 Lire des fichiers textuels . . . 102
9.4 Écrire des fichiers textuels . . . 108
9.5 Exercices . . . 111
9.6 Bilan du chapitre . . . 111

10 `numpy` et le calcul matriciel **113**
10.1 Les tableaux `numpy` . . . 114
10.2 Les opérations élément par élément . . . 115
10.3 Produit matriciel . . . 116
10.4 Créer des tableaux usuels . . . 117

10.5 Accéder à des sous-parties d'un tableau . . . 117
10.6 Statistiques sur les tableaux . . . 118
10.7 Bonnes pratiques : éviter les boucles . . . 120
10.8 Exercices . . . 121
10.9 Bilan du chapitre . . . 122

11 Découvrir ses données avec `pandas` 123
11.1 Le jeu de données de ce chapitre . . . 125
11.2 `DataFrame` et `Series` . . . 125
11.3 Charger un fichier CSV . . . 127
11.4 Explorer un jeu de données . . . 128
11.5 Sélectionner des colonnes . . . 130
11.6 Filtrer des lignes . . . 131
11.7 `loc` et `iloc` . . . 133
11.8 Trier . . . 135
11.9 Exercices . . . 136
11.10 Bilan du chapitre . . . 137

12 Préparer et résumer ses données avec `pandas` 139
12.1 Créer et modifier des colonnes . . . 140
12.2 Les valeurs manquantes . . . 142
12.3 Texte et dates . . . 143
12.4 Résumer par groupes : `groupby` . . . 145
12.5 Combiner des tableaux . . . 147
12.6 Enregistrer le résultat . . . 148
12.7 Exercices . . . 149
12.8 Bilan du chapitre . . . 150

13 Visualiser ses données avec `matplotlib` et `seaborn` 151
13.1 Quel graphique pour quelle question ? . . . 152
13.2 Les bases de `matplotlib` . . . 152
13.3 Des graphiques statistiques avec `seaborn` . . . 159
13.4 Exercices . . . 165
13.5 Bilan du chapitre . . . 166

14 Modélisation statistique avec `statsmodels` 169
14.1 Correspondances avec R et SAS . . . 170
14.2 La régression linéaire . . . 171
14.3 La régression logistique . . . 176
14.4 Les tests statistiques classiques . . . 177
14.5 Exercices . . . 178
14.6 Bilan du chapitre . . . 179

15 Mini-projets 181
15.1 Projet 1 : Bulletins de notes en Python « pur » ☆ ☆ . . . 182
15.2 Projet 2 : Les manchots de l'archipel Palmer ☆ ☆ . . . 183
15.3 Projet 3 : Qui a survécu au naufrage du Titanic ? ☆ ☆ ☆ . . . 184

16 Fiche mémo 185
16.1 Bases (chapitres 1 à 4) . . . 185
16.2 Listes (chapitre 5) . . . 186
16.3 Chaînes de caractères (chapitre 6) . . . 187

16.4 Dictionnaires, *tuples*, *sets* (chapitre 7) . . . 187
16.5 Tests (chapitre 8) . . . 187
16.6 Fichiers (chapitre 9) . . . 188
16.7 `numpy` (chapitre 10) . . . 188
16.8 `pandas` (chapitres 11 et 12) . . . 189
16.9 Visualisation (chapitre 13) . . . 190
16.10 Modélisation (chapitre 14) . . . 190
16.11 Correspondances R / SAS / Python . . . 191
16.12 Les erreurs les plus courantes . . . 192

17 Et après ? **193**

Ce document est une version du polycopié *Introduction à Python* de la filière MIASHS de l'Université de Rennes 2, **réécrite pour pouvoir apprendre seul·e, sans cours en présentiel**, avec un objectif précis : **acquérir les bases nécessaires pour analyser des données** (*data science*) avec Python.

Il vous mène des toutes premières lignes de code jusqu'à l'analyse d'un vrai jeu de données avec les bibliothèques `pandas`, `matplotlib`, `seaborn` et `statsmodels`. Vous y trouverez :

- des **schémas** pour visualiser ce qui se passe dans la machine ;
- des questions « **Testez-vous** » au fil de la lecture, pour vérifier immédiatement que vous avez compris ;
- des **exercices progressifs** (de ☆ à ☆ ☆ ☆) avec des **indices** et des **corrections détaillées** ;
- des encadrés « **Erreurs fréquentes** » qui montrent les vrais messages d'erreur de Python et comment les corriger ;
- un **bilan** à la fin de chaque chapitre pour faire le point sur ce que vous savez faire ;
- des **mini-projets** pour mettre tout cela en pratique et une **fiche mémo** pour réviser.

Ce document est distribué librement sous licence CC BY-NC-SA. Le polycopié d'origine a notamment bénéficié des apports d'Aurélie Lemaitre, d'Agnès Maunoury et de Jean-Christophe Burnel. N'hésitez pas à faire vos remarques à son auteur dont vous trouverez le contact sur sa page web.

À qui s'adresse ce document ?

À toute personne qui souhaite apprendre à programmer en Python pour analyser des données, **sans aucun prérequis** en programmation. Quelques notions de statistique (moyenne, médiane, quartiles, corrélation, régression) seront utiles dans la dernière partie.

Vous connaissez déjà R ou SAS ?

Ce document a été pensé pour vous aussi ! Vous connaissez déjà la logique de l'analyse de données ; il vous reste à apprendre la **syntaxe** et les **habitudes** de Python, qui diffèrent sur des points importants.

- Des encadrés « **Si vous venez de R ou de SAS** » signalent, dans chaque chapitre, les pièges à éviter (indices qui commencent à 0, indentation, `return` obligatoire, copie des listes…) et donnent des **tables de correspondance** entre R, SAS et Python.
- Vous pouvez parcourir plus vite les chapitres 1 à 8 : lisez les encadrés « À retenir » et « Si vous venez de R ou de SAS », répondez aux questions « Testez-vous », et faites au moins les exercices ☆ ☆ . Ne les sautez pas complètement : `pandas` repose entièrement sur ces bases, et la plupart des erreurs viennent de réflexes R ou SAS appliqués à Python.
- Le chapitre 14 vous montre comment retrouver vos modèles habituels (`lm`, `glm`, `PROC REG`, `PROC LOGISTIC`, tests…) avec `statsmodels` et `scipy`.

Il suffit de savoir utiliser un ordinateur et d'avoir un peu de patience : apprendre à programmer, c'est un peu comme apprendre une langue étrangère, on progresse surtout en pratiquant régulièrement.

Le parcours proposé

Les chapitres sont conçus pour être lus **dans l'ordre**, chacun s'appuyant sur les précédents. Les durées indiquées sont des estimations (lecture + exercices) : prenez le temps qu'il vous faut !

Partie	Chapitre	Durée indicative
Les bases de Python	1. Variables, types et opérations	1h30
	2. Les conditions	1h30
	3. Les boucles	2h
	4. Les fonctions et les modules	2h
Les collections de données	5. Les listes	2h30
	6. Les chaînes de caractères	1h30
	7. Dictionnaires, *tuples* et *sets*	2h
	8. Déboguer et tester son code	1h
Analyser des données	9. Lire et écrire des fichiers	1h30
	10. `numpy` et le calcul matriciel	1h30
	11. Découvrir ses données avec `pandas`	2h30
	12. Préparer et résumer ses données avec `pandas`	2h30
	13. Visualiser ses données avec `matplotlib` et `seaborn`	2h
	14. Modélisation statistique avec `statsmodels`	2h
S'entraîner	Mini-projets	2h à 4h chacun

Les chapitres 1 à 8 forment le **socle** de programmation : ne les négligez pas, même si vous avez hâte d'analyser des données ! `pandas` repose entièrement sur ces notions (variables, fonctions, listes, dictionnaires…), et la plupart des difficultés rencontrées plus tard viennent de bases fragiles. Les chapitres 9 à 14 vous donnent ensuite les outils de l'analyse de données, que vous mettrez en œuvre dans les mini-projets.

Conseil

Plutôt que de faire 10 heures d'un coup, préférez des séances régulières de 1h à 2h (par exemple 3 fois par semaine). Au début de chaque séance, relisez rapidement le **bilan** du chapitre précédent.

Comment lire les encadrés

Tout au long de ce document, vous rencontrerez différents types d'encadrés. Voici à quoi ils ressemblent et ce qu'ils signifient.

Objectifs du chapitre

Au début de chaque chapitre : ce que vous saurez faire à la fin.

À retenir

Les points essentiels, à connaître par cœur.

Erreur fréquente

Les pièges dans lesquels (presque) tout le monde tombe au début, et comment s'en sortir.

Testez-vous

Une petite question pour vérifier votre compréhension. Essayez **vraiment** d'y répondre (dans votre tête ou sur papier) avant de déplier la réponse.

Réponse

La réponse apparaît quand on clique sur le titre de l'encadré.

Exercice 0.1 ☆

Un exercice à faire sur ordinateur. Le nombre d'étoiles indique la difficulté :

- ☆ : application directe du cours ;
- ☆ ☆ : demande de combiner plusieurs notions ou de réfléchir un peu ;
- ☆ ☆ ☆ : plus difficile, pour aller plus loin (ne vous découragez pas si vous bloquez !).

Indice

Un coup de pouce si vous êtes bloqué·e. N'ouvrez l'indice qu'après avoir cherché un moment.

Solution

Une correction commentée. Il y a souvent plusieurs bonnes solutions : si la vôtre est différente mais donne le bon résultat, elle est sans doute correcte aussi !

Pour aller plus loin

Des compléments facultatifs, que vous pouvez sauter lors d'une première lecture.

Dans cette version PDF, les réponses, indices et solutions sont masqués : vous les trouverez (repliés) dans la version web de ce document.

Où écrire et exécuter du code Python ?

Les blocs de code de ce document

Durant la lecture, vous trouverez des blocs de code tels que celui-ci :

```
def f(v):
    return v ** 2

x = 5
y = f(3 * x + 2)
print(y)
```

```
289
```

Le bloc situé sous le code correspond à ce qu'affiche ce code lorsqu'on l'exécute.

À retenir

Ne vous contentez pas de lire les exemples : retapez-les vous-même, exécutez-les, puis modifiez-les pour voir ce qui change. C'est de loin la meilleure façon d'apprendre. Évitez le copier-coller, qui donne l'illusion d'avoir compris : taper le code vous oblige à faire attention à chaque détail (parenthèses, deux-points, décalages…).

Option 1 : les fenêtres de code intégrées (version web)

Dans la version web de ce document, vous trouverez des fenêtres d'éditeur de code vous permettant de tester vos solutions aux exercices directement dans votre navigateur, sans rien installer. En voici une : appuyez sur « Run » pour exécuter le code, puis modifiez le code et observez le résultat en le ré-exécutant.

Option 2 : sans installation, dans le navigateur

Si vous ne pouvez pas installer de logiciel, le site Basthon propose une console et des carnets (*notebooks*) Python qui fonctionnent directement dans le navigateur.

Option 3 (recommandée) : installer Python sur votre ordinateur

Pour travailler confortablement, il est préférable d'installer Python sur votre ordinateur. Deux outils sont utiles, selon la partie du document :

- **Pour les chapitres 1 à 9**, le logiciel Thonny est idéal pour débuter : il est gratuit, disponible sous Windows, macOS et Linux, et il installe Python en même temps que lui. Son interface est très simple : une zone **éditeur** (en haut) pour écrire un programme et l'enregistrer dans un fichier `.py`, un bouton **Run** (triangle vert) pour l'exécuter, et une **console** (en bas) où s'affichent les résultats. Thonny peut aussi afficher les valeurs de vos variables (menu *View* > *Variables*), ce qui est très pratique pour comprendre ce que fait un programme.
- **Pour l'analyse de données (chapitres 10 à 14 et projets)**, on travaille dans des **notebooks Jupyter** : des documents qui mêlent cellules de code, résultats (tableaux, graphiques) et texte, ce qui est idéal pour explorer des données et garder une trace de son analyse. Le plus simple est d'installer la distribution Anaconda, qui contient Python, Jupyter, `numpy`, `pandas`, `matplotlib`, `seaborn`, `statsmodels` et `scipy`. Vous pouvez aussi utiliser Google Colab, qui fait fonctionner des notebooks dans le navigateur, sans installation, avec toutes ces bibliothèques (un compte Google est nécessaire).

Rien ne vous empêche d'utiliser Jupyter dès le début, si vous préférez un seul outil.

Les fichiers de données

Plusieurs chapitres utilisent des fichiers de données (dans le dossier `donnees` qui accompagne ce document) : `notes.csv`, `fable.txt`, et trois jeux de données réels très utilisés pour apprendre l'analyse de données : `penguins.csv` (des manchots de l'Antarctique), `titanic.csv` (des passagers du Titanic) et `flights.csv` (le trafic d'une compagnie aérienne de 1949 à 1960). Placez votre notebook ou votre script dans le même dossier que le dossier `donnees`.

Console ou fichier ?

Il y a deux façons de faire exécuter du code Python :

1. dans une **console** (le prompt `>>>`) : chaque instruction est exécutée dès que vous appuyez sur Entrée. C'est pratique pour faire de petits essais ;
2. dans un **fichier** `.py` (un *script*) que l'on fait exécuter en entier. C'est ce que l'on fait dès que le programme fait plus de quelques lignes, car on garde une trace de ce que l'on a écrit.

En ligne de commande (dans un terminal), on lance la console avec `python` (ou `python3`) et on exécute un script avec `python nom_de_mon_fichier.py`. Assurez-vous d'utiliser Python 3 (la console affiche sa version au démarrage) : ce document ne s'applique pas à Python 2, qui n'est plus utilisé.

Un outil précieux : Python Tutor

Certains exemples de ce document sont accompagnés d'un lien « *Voir ce code s'exécuter pas à pas dans Python Tutor* ». Le site Python Tutor permet d'exécuter un programme **instruction par instruction** (bouton *Next*) en dessinant à chaque étape les variables et leurs valeurs. C'est l'équivalent d'un enseignant qui expliquerait au tableau ce qui se passe dans la machine : n'hésitez pas à l'utiliser dès qu'un code vous semble mystérieux, y compris le vôtre !

Voici un premier exemple à essayer :

```
prenom = "Aude"
age = 20
age = age + 1
print(prenom, "a", age, "ans")
```

Voir ce code s'exécuter pas à pas dans Python Tutor

Méthode de travail conseillée

Pour chaque chapitre :

1. **Lisez les objectifs** pour savoir où vous allez.
2. **Lisez une section** en retapant et en exécutant les exemples. Modifiez-les : que se passe-t-il si… ?
3. **Répondez aux questions « Testez-vous »** avant de regarder la réponse. Si vous vous êtes trompé·e, relisez la section : c'est normal et c'est comme ça qu'on apprend.
4. **Faites les exercices** dans l'ordre. Cherchez au moins 10 à 15 minutes avant d'ouvrir l'indice, puis encore un peu avant d'ouvrir la solution. Une fois la solution lue, **refaites l'exercice sans la regarder**.
5. **Faites le bilan** en fin de chapitre : cochez ce que vous savez faire et revenez sur ce qui reste flou.

Que faire quand on est bloqué·e ?

1. **Lisez le message d'erreur** en entier, et en particulier sa dernière ligne (nous apprendrons à les lire dès le chapitre 1).
2. **Affichez vos variables** avec `print` à différents endroits du programme pour vérifier qu'elles contiennent ce que vous croyez.
3. **Exécutez votre code pas à pas** dans Python Tutor ou dans Thonny.
4. **Expliquez votre code à voix haute**, ligne par ligne, comme si vous l'expliquiez à quelqu'un d'autre (même à un canard en plastique : c'est une technique reconnue chez les programmeurs !). On trouve souvent l'erreur en cours de route.
5. **Faites une pause.** Sérieusement : la solution vient souvent après.

C'est parti !

CHAPTER 1

VARIABLES, TYPES ET OPÉRATIONS

Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- afficher du texte et des résultats de calcul avec `print` ;
- créer des variables, leur donner des valeurs et connaître leur type ;
- utiliser les opérateurs arithmétiques (`+`, `-`, `*`, `/`, `//`, `%`, `**`) ;
- convertir une valeur d'un type à un autre (`int`, `float`, `str`) ;
- lire un message d'erreur Python.

Durée indicative : 1h30.

1.1 Un tout premier programme

Commençons par le plus simple des programmes : afficher un message à l'écran. On utilise pour cela la **fonction** `print` :

```
print("Bonjour !")
```

```
Bonjour !
```

Le texte à afficher est placé **entre guillemets** (on parle de *chaîne de caractères*) et **entre parenthèses** après le mot `print`. On peut aussi demander à Python d'afficher le résultat d'un calcul :

```
print(12 * 5)
```

```
60
```

ou encore afficher plusieurs choses à la suite, séparées par des virgules (Python insère alors un espace entre elles) :

```
print("12 fois 5 font", 12 * 5)
```

```
12 fois 5 font 60
```

Un programme est une **suite d'instructions** exécutées les unes après les autres, de haut en bas :

```
print("Première ligne")
print("Deuxième ligne")
print("Troisième ligne")
```

```
Première ligne
Deuxième ligne
Troisième ligne
```

Enfin, tout ce qui suit un symbole # sur une ligne est un **commentaire** : Python l'ignore complètement. Les commentaires servent à expliquer le code aux humains qui le lisent (y compris vous-même dans trois semaines !).

```
# Ceci est un commentaire : il n'est pas exécuté
print("Le commentaire n'a rien affiché")  # on peut aussi␣
↪commenter en fin de ligne
```

```
Le commentaire n'a rien affiché
```

Testez-vous

Qu'affiche le programme suivant ?

```
print("3 + 4")
print(3 + 4)
```

1.2 Les variables

1.2.1 Qu'est-ce qu'une variable ?

Dès qu'un programme fait plus que quelques calculs, on a besoin de **mémoriser des valeurs** pour les réutiliser plus tard. En Python, les données sont stockées dans des **variables**. Une variable est une association entre un **nom** (choisi par vous) et une **valeur**, qui peut changer au cours de l'exécution du programme.

On peut se représenter une variable comme une **étiquette** portant un nom, accrochée à une boîte qui contient une valeur :

Pour créer une variable, on utilise le signe =, appelé **opérateur d'affectation** :

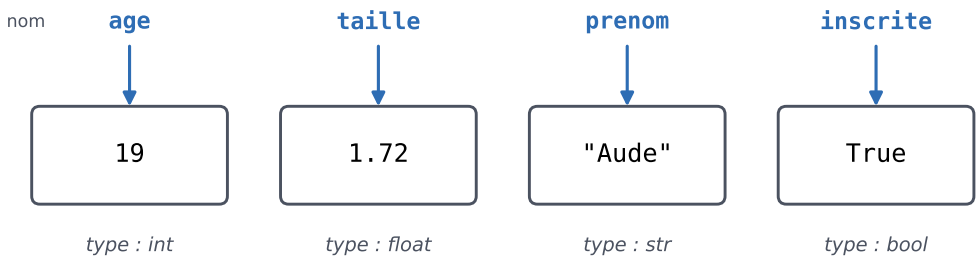


Figure1.1: Quatre variables et leurs valeurs. Chaque valeur a un **type**.

```
age = 19
taille = 1.72
prenom = "Aude"
inscrite = True
```

Ces instructions n'affichent rien : elles se contentent de ranger des valeurs en mémoire. On peut ensuite utiliser les variables en écrivant simplement leur nom :

```
print(prenom)
print(age + 1)
print(prenom, "mesure", taille, "m")
```

```
Aude
20
Aude mesure 1.72 m
```

À retenir

Le signe = en Python ne signifie **pas** « est égal à » comme en mathématiques, mais « **prend la valeur** ». L'instruction `age = 19` se lit : « la variable `age` prend la valeur 19 ». On met toujours **le nom de la variable à gauche** et **la valeur à droite**.

1.2.2 Les types de base

Chaque valeur a un **type**, qui détermine ce que l'on peut faire avec elle. Les types de base en Python sont les suivants :

Type	Signification	Exemples
`int`	nombre entier (*integer*)	`12, -3, 0`
`float`	nombre à virgule (*floating point number*)	`1.72, -0.5, 12.`
`str`	chaîne de caractères (*string*), c'est-à-dire du texte	`"Aude", 'abc', "12"`
`bool`	booléen : vrai ou faux	`True, False`
`complex`	nombre complexe (peu utilisé dans ce document)	`1 + 2j`

Il existe de plus une valeur spéciale, `None`, qui signifie « pas de valeur » ou « valeur manquante » (son type est `NoneType`).

Erreur fréquente

- En Python, la virgule des nombres décimaux s'écrit avec un **point** : on écrit `1.72` et non `1,72`.
- Les chaînes de caractères peuvent être délimitées par des guillemets doubles (`"abc"`) ou simples (`'abc'`) : les deux sont équivalents, mais il faut utiliser le même au début et à la fin.
- `True` et `False` prennent une majuscule.

Pour connaître le type d'une valeur ou d'une variable, on utilise la fonction `type` :

```
print(type(age))
print(type(taille))
print(type(prenom))
print(type(inscrite))
```

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
```

En Python, on ne déclare pas le type d'une variable : il est déduit de la valeur qu'on y range. Et si l'on range une valeur d'un autre type dans la même variable, son type change (on dit que Python est **typé dynamiquement**) :

```
v = 12          # v est de type entier
print(type(v))
v = 12.         # v change de type : c'est désormais un nombre à␣
 ↪virgule
print(type(v))
```

```
<class 'int'>
<class 'float'>
```

Testez-vous

Quel est le type de chacune des valeurs suivantes ? `42`, `"42"`, `42.0`, `"vrai"`, `False`.

1.2.3 Choisir un nom de variable

Les noms de variables doivent respecter quelques règles :

- ils doivent commencer par une lettre (minuscule ou majuscule) ou par le symbole _ ;
- ils ne doivent contenir que des lettres, des chiffres et des symboles _ (pas d'espace, pas de tiret −, pas d'apostrophe) ;
- ils ne doivent pas être un **mot réservé** du langage Python, dont voici la liste :

```
and del for is raise assert elif from lambda return break else global
not try nonlocal True False class except if or while continue import
pass yield None def finally in as with
```

- enfin, il est vivement conseillé de ne pas utiliser le nom d'une fonction existante de Python (comme `print`, `type`, `list`, `sum`...) : vous apprendrez ces noms au fur et à mesure.

Les noms de variables sont **sensibles à la casse** : `mavariable` et `maVariable` sont deux variables différentes.

```
mavariable = 12
maVariable = 15
print(mavariable)
print(maVariable)
```

```
12
15
```

Bonne pratique

Choisissez des noms **explicites** : `prix_total` est bien plus clair que `pt` ou `x`. Un programme est écrit une fois mais relu de nombreuses fois ! En Python, l'usage est d'écrire les noms de variables en minuscules, avec des `_` pour séparer les mots (`nombre_etudiants`). Les accents sont autorisés mais il est d'usage de les éviter.

Testez-vous

Parmi les noms suivants, lesquels sont des noms de variables valides ? `note1`, `1ere_note`, `note-finale`, `note_finale`, `_temp`, `for`, `Note`.

1.3 L'affectation en détail

La règle de l'affectation est la suivante : « **calculer** la valeur de ce qui est à droite du =, puis **ranger** le résultat dans la variable de gauche ». Il est donc tout à fait valide d'écrire :

```
x = 5
x = x + 2
print(x)
```

```
7
```

Mathématiquement, `x = x + 2` n'a aucun sens. Mais en Python, cette instruction se lit : « `x` prend la valeur (ancienne valeur de `x`) + 2 ».

Le même principe s'applique pour des expressions plus complexes :

```
x = 2
x = 3.9 * x * (1 - x)
print(x)
```

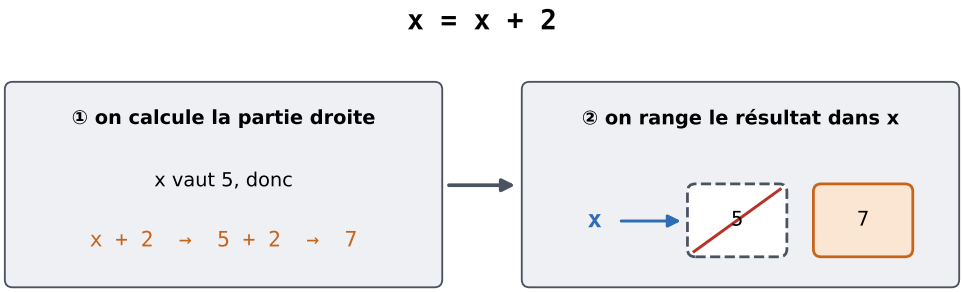


Figure1.2: Les deux temps de l'affectation `x = x + 2` : d'abord le calcul à droite, ensuite le rangement à gauche (l'ancienne valeur est perdue).

```
-7.8
```

Pour exécuter la deuxième instruction, Python commence par évaluer `3.9 * x * (1 - x)` avec la valeur courante de `x` (2), obtient `-7.8`, puis range ce résultat dans `x`.

Comme les instructions sont exécutées dans l'ordre, la valeur d'une variable à un instant donné dépend de tout ce qui a été exécuté avant :

```
a = 3
b = a + 1
a = 10
print(a, b)
```

```
10 4
```

Remarquez que modifier `a` à la ligne 3 ne modifie pas `b` : au moment de la ligne 2, Python a calculé `a + 1` (qui valait 4) et a rangé **le résultat** dans `b`.

Voir ce code s'exécuter pas à pas dans Python Tutor

Testez-vous

Que contiennent les variables `a` et `b` à la fin de ce programme ?

```
a = 5
b = 2
a = b
b = a
```

1.4 Les opérations sur les nombres

1.4.1 Les opérateurs arithmétiques

Voici les opérateurs disponibles pour les nombres (`int` et `float`) :

Opérateur	Opération	Exemple	Résultat
`+`	Addition	`7 + 2`	`9`
`-`	Soustraction	`7 - 2`	`5`
`*`	Multiplication	`7 * 2`	`14`
`/`	Division	`7 / 2`	`3.5`
`**`	Puissance	`7 ** 2`	`49`
`//`	Division entière (quotient)	`7 // 2`	`3`
`%`	Modulo (reste de la division entière)	`7 % 2`	`1`

Les deux derniers opérateurs méritent qu'on s'y arrête : ce sont ceux de la division que vous avez apprise à l'école primaire. Si l'on partage 17 bonbons entre 5 enfants, chaque enfant en reçoit 3 et il en reste 2 :

```
print(17 // 5)  # quotient : nombre de bonbons par enfant
print(17 % 5)   # reste : nombre de bonbons restants
```

```
3
2
```

À retenir

L'opérateur `%` (modulo) est très utile en pratique :

- `n % 2` vaut `0` si `n` est pair et `1` si `n` est impair ;
- plus généralement, `n % k == 0` signifie « `n` est divisible par `k` » ;
- `n % 10` donne le dernier chiffre d'un entier positif `n`.

Notez que la division `/` donne toujours un `float`, même si le résultat « tombe juste » :

```
print(6 / 3)
print(2 / 3)
```

```
2.0
0.6666666666666666
```

De manière générale, lorsqu'une opération mêle un `int` et un `float`, le résultat est un `float`.

Les nombres à virgule sont approximatifs

Les ordinateurs stockent les nombres à virgule avec une précision limitée, ce qui donne parfois des résultats surprenants. Par exemple, `print(0.1 + 0.2)` affiche `0.30000000000000004`. Ce n'est pas un bug de Python : la plupart des langages de

programmation se comportent de la même façon. Retenez simplement qu'il faut éviter de tester l'égalité exacte entre deux nombres à virgule, et utiliser l'affichage avec un nombre fixé de décimales (voir plus bas) pour présenter les résultats.

1.4.2 Priorité des opérateurs

Lorsqu'une expression contient plusieurs opérateurs, Python applique les mêmes règles de priorité qu'en mathématiques (de la priorité la plus forte à la plus faible) :

1. parenthèses ;
2. puissance `**` ;
3. multiplication, divisions et modulo (`*`, `/`, `//`, `%`) ;
4. addition et soustraction ;
5. à priorité égale, de gauche à droite.

```
print(2 + 3 * 4)
print((2 + 3) * 4)
print(2 * 3 ** 2)
```

```
14
20
18
```

En cas de doute, ajoutez des parenthèses : cela ne coûte rien et rend le code plus lisible.

1.4.3 Les opérateurs raccourcis

Il est très fréquent de modifier une variable à partir de sa propre valeur (par exemple pour compter quelque chose). Pour cela, il existe des raccourcis d'écriture : `x += 2` est équivalent à `x = x + 2`.

```
compteur = 0
compteur += 1    # équivaut à compteur = compteur + 1
compteur += 1
print(compteur)
```

```
2
```

Il en va de même pour `-=`, `*=`, `/=`, *etc.* Ce n'est qu'un raccourci : si vous ne vous sentez pas à l'aise avec, vous pouvez toujours écrire `x = x + 2`.

Testez-vous

Que valent les expressions suivantes ? `10 // 3,10 % 3,10 / 4,2 ** 3 + 1,15 % 5`.

1.5 Opérations sur les chaînes et conversions

1.5.1 Concaténation

Certains opérateurs fonctionnent aussi avec les chaînes de caractères, avec un sens différent : `+` met deux chaînes bout à bout (on dit qu'on les **concatène**) et `*` répète une chaîne.

```
prenom = "Aude"
print("Bonjour " + prenom + " !")
print("ha" * 3)
```

```
Bonjour Aude !
hahaha
```

1.5.2 Conversions de type

On ne peut pas additionner directement un texte et un nombre. Il faut d'abord **convertir** l'un des deux dans le type de l'autre, à l'aide des fonctions `int`, `float` et `str` :

```
texte = "12"
nombre = int(texte)       # conversion de str vers int
print(nombre + 2)

age = 19
print("J'ai " + str(age) + " ans")   # conversion de int vers str

print(float("3.5") * 2)   # conversion de str vers float
print(int(3.99))          # int() tronque : on garde la partie␣
 ↪entière
```

```
14
J'ai 19 ans
7.0
3
```

Testez-vous

Qu'affichent les instructions suivantes ?

```
print("5" + "5")
print(5 + 5)
print("5" * 3)
print(int("5") * 3)
```

1.5.3 Afficher joliment : les f-strings

Construire un message avec des + et des `str()` devient vite pénible. Python propose une écriture bien plus pratique, les **f-strings** : on place un `f` juste avant les guillemets et on écrit les noms de variables (ou des calculs) entre accolades `{}` dans le texte.

```
prenom = "Aude"
age = 19
print(f"{prenom} a {age} ans et aura {age + 1} ans l'an prochain.
 ↪")
```

```
Aude a 19 ans et aura 20 ans l'an prochain.
```

On peut aussi contrôler le nombre de décimales affichées avec `:.2f` (2 chiffres après la virgule) :

```
prix = 10 / 3
print(f"Prix : {prix} euros")
print(f"Prix : {prix:.2f} euros")
```

```
Prix : 3.3333333333333335 euros
Prix : 3.33 euros
```

Vous rencontrerez aussi parfois une autre écriture, plus ancienne, qui utilise la méthode `format` : `"{} a {} ans".format(prenom, age)`. Nous reviendrons sur ces deux façons de faire au *chapitre sur les chaînes de caractères*.

1.6 Lire un message d'erreur

En programmant, vous ferez des erreurs : **c'est normal et cela arrive à tout le monde**, débutant·e ou expert·e. Python vous aide en affichant un message d'erreur qu'il faut apprendre à lire. Voici ce qui se passe si l'on essaie d'additionner un texte et un nombre :

```
x = "12"
y = x + 2
```

```
---------------------------------------------------------------------
 ↪-----------
TypeError                                 Traceback (most␣
 ↪recent call last)
Cell In[22], line 2
      1 x = "12"
----> 2 y = x + 2

TypeError: can only concatenate str (not "int") to str
```

Ce message peut sembler effrayant, mais il contient des informations précieuses :

1. **Où ?** Le message indique la ligne du programme où l'erreur s'est produite (ici la ligne 2, `y = x + 2`).
2. **Quel type d'erreur ?** Ici une `TypeError`, c'est-à-dire un problème de type de données.

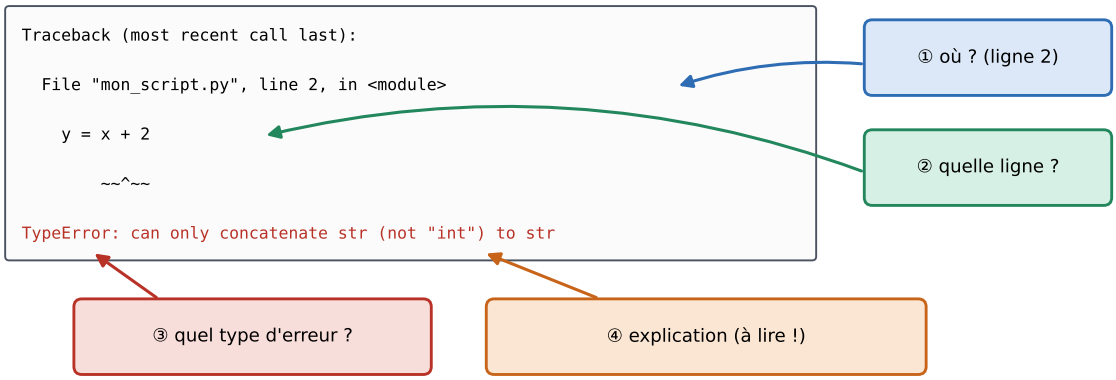


Figure1.3: Anatomie d'un message d'erreur (*traceback*) dans un script. La ligne la plus importante est la **dernière**.

3. **Quelle explication ?** « can only concatenate str (not « int ») to str » : on ne peut concaténer (coller) une chaîne qu'avec une autre chaîne, pas avec un entier.

En effet, `x` contient le texte `"12"` (et non le nombre 12). Pour corriger, il suffit de convertir `x` en entier :

```
x = "12"
y = int(x) + 2
print(y)
```

```
14
```

Erreurs fréquentes de ce chapitre

Message d'erreur	Cause probable	Correction
`NameError: name 'prnom' is not defined`	faute de frappe dans un nom de variable, ou variable utilisée avant d'avoir été créée	vérifier l'orthographe (et les majuscules)
`SyntaxError: invalid syntax`	erreur de « grammaire » : parenthèse ou guillemet oublié, nom de variable invalide...	regarder la ligne indiquée **et la ligne précédente**
`SyntaxError: unterminated string literal`	guillemet fermant oublié	ajouter le guillemet
`TypeError: can only concatenate str (not "int") to str`	+ entre un texte et un nombre	convertir avec `int()` ou `str()`, ou utiliser une f-string
`ValueError: invalid literal for int() with base 10: 'douze'`	conversion impossible	vérifier le contenu de la chaîne
`ZeroDivisionError: division by zero`	division par zéro	vérifier la valeur du diviseur

Voici par exemple une `NameError`, due à une faute de frappe :

```
prenom = "Aude"
print(prnom)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most␣
↪recent call last)
Cell In[24], line 2
      1 prenom = "Aude"
----> 2 print(prnom)

NameError: name 'prnom' is not defined
```

Si vous venez de R ou de SAS

- L'affectation s'écrit uniquement avec = (pas de <−).
- En R, `x <- 5` crée un vecteur de longueur 1 ; en Python, `5` est un simple nombre, et une suite de valeurs est une **liste** (chapitre 5) ou un tableau `numpy` (chapitre 10). Les opérations « vectorisées » auxquelles vous êtes habitué·e n'existent qu'avec `numpy` et `pandas`.
- La division `/` donne toujours un nombre à virgule ; la division entière s'écrit `//` (`%/%` en R) et le reste `%` (`%%` en R).
- Les valeurs logiques s'écrivent `True` et `False` (et non `TRUE` / `FALSE`), et la valeur manquante « générique » est `None` (avec `pandas`, ce sera `NaN`, l'équivalent de `NA` en R et de `.` en SAS).

- Dans un script, seul `print` affiche quelque chose : contrairement à la console R, une expression seule sur une ligne n'est pas affichée (sauf dans la console Python ou en dernière ligne d'une cellule de notebook).

1.7 Exercices

Exercice 1.1 : Conversion de températures ☆

La température en degrés Fahrenheit F s'obtient à partir de la température en degrés Celsius C par la formule $F = \frac{9}{5}C + 32$.

Écrivez un programme qui, à partir d'une température en degrés Celsius stockée dans une variable `celsius`, calcule la température en degrés Fahrenheit, la stocke dans une variable `fahrenheit` et l'affiche sous la forme `20 °C = 68.0 °F`.

Exercice 1.2 : Prix TTC ☆

Un article coûte `prix_ht` euros hors taxes. La TVA est de 20 %. Écrivez un programme qui calcule et affiche le montant de la TVA et le prix toutes taxes comprises (TTC), avec deux chiffres après la virgule. Pour `prix_ht = 49.99`, on doit obtenir :

```
TVA : 10.00 euros
Prix TTC : 59.99 euros
```

Exercice 1.3 : Heures, minutes, secondes ☆ ☆

Une durée est donnée en secondes dans une variable `duree` (par exemple `duree = 4000`). Écrivez un programme qui affiche cette durée en heures, minutes et secondes. Pour 4000 secondes, on doit obtenir : `4000 s = 1 h 6 min 40 s`.

Exercice 1.4 : Chiffres d'un nombre ☆ ☆

Soit `n` un nombre entier positif à trois chiffres (par exemple `n = 472`). Écrivez un programme qui affiche séparément son chiffre des unités, son chiffre des dizaines et son chiffre des centaines, puis la somme de ses chiffres.

Exercice 1.5 : Échanger deux variables ☆ ☆

On dispose de deux variables `a = 5` et `b = 8`. Écrivez un programme qui **échange** leurs valeurs (à la fin, `a` doit valoir 8 et `b` doit valoir 5), sans écrire les nombres 5 et 8 ailleurs que dans les deux premières lignes. Relisez la question « Testez-vous » de la section sur l'affectation : le piège y est décrit !

Exercice 1.6 : Ticket de caisse ☆ ☆ ☆

Vous achetez `nb_croissants` croissants à 1,10 € pièce et `nb_baguettes` baguettes à 0,95 € pièce. Vous payez avec un billet de 20 €. Écrivez un programme qui affiche le montant total et la monnaie rendue, puis le nombre de pièces de 2 € qui seront rendues au maximum dans cette monnaie.

Pour 3 croissants et 2 baguettes, on doit obtenir :

```
Total : 5.20 euros
Monnaie : 14.80 euros
Dont 7 pièce(s) de 2 euros
```

1.8 Bilan du chapitre

À retenir

- `print(...)` affiche des valeurs ; le texte s'écrit entre guillemets.
- `variable = valeur` : on **calcule** ce qui est à droite puis on **range** le résultat dans la variable de gauche.
- Types de base : `int`, `float`, `str`, `bool` (et la valeur spéciale `None`). La fonction `type` donne le type d'une valeur.
- `//` donne le quotient de la division entière, `%` le reste ; `/` donne toujours un `float`.
- On convertit avec `int()`, `float()`, `str()`.
- `f"... {variable} ..."` permet d'insérer des valeurs dans un texte.
- Un message d'erreur se lit en commençant par **la dernière ligne**, puis en allant voir la ligne de code indiquée.

Faites le point : êtes-vous capable de...

- créer une variable, l'afficher et connaître son type ?
- prédire la valeur d'une variable après une suite d'affectations ?
- calculer un quotient et un reste avec `//` et `%` ?
- convertir un texte en nombre et inversement ?
- afficher un message contenant des valeurs de variables avec une f-string ?
- identifier, dans un message d'erreur, la ligne en cause et le type d'erreur ?

Si vous avez coché toutes les cases, passez au chapitre suivant. Sinon, relisez les sections correspondantes et refaites les exercices associés.

CHAPTER 2

LES CONDITIONS

Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- écrire des comparaisons (`<`, `==`, `!=`...) et les combiner (`and`, `or`, `not`) ;
- exécuter des instructions seulement si une condition est vraie (`if`) ;
- choisir entre plusieurs cas (`else`, `elif`) ;
- respecter les règles d'**indentation** de Python.

Prérequis : chapitre 1. **Durée indicative :** 1h30.

Jusqu'à présent, nos programmes exécutaient toutes leurs instructions, dans l'ordre, sans exception. Mais souvent, on souhaite qu'un programme **réagisse différemment selon la situation** : afficher « admis » ou « ajourné » selon une note, appliquer une réduction seulement aux étudiants, *etc.* C'est le rôle des **structures conditionnelles**.

2.1 Les booléens et les comparaisons

Une condition est une expression qui est soit vraie, soit fausse. En Python, son résultat est un **booléen** (type `bool`) : `True` (vrai) ou `False` (faux).

```
note = 12
print(note >= 10)
print(note == 20)
```

```
True
False
```

Voici les opérateurs de comparaison à connaître :

Opérateur	Signification	Exemple (avec `x = 5`)	Résultat
`<`	strictement plus petit que	`x < 5`	`False`
`>`	strictement plus grand que	`x > 0`	`True`
`<=`	plus petit ou égal à	`x <= 5`	`True`
`>=`	plus grand ou égal à	`x >= 6`	`False`
`==`	égal à	`x == 5`	`True`
`!=`	différent de	`x != 5`	`False`
`is`	test d'identité, pour comparer à `None`	`x is None`	`False`
`is not`	contraire de `is`	`x is not None`	`True`
`in`	présence d'une valeur dans une liste	`x in [1, 5, 7]`	`True`

À retenir

Pour **tester** l'égalité, on utilise == (deux signes égal). Un seul = sert à **affecter** une valeur à une variable.

- `x = 5` : « x prend la valeur 5 » (affectation) ;
- `x == 5` : « est-ce que x vaut 5 ? » (comparaison, qui donne `True` ou `False`).

Les comparaisons fonctionnent aussi avec les chaînes de caractères :

```
prenom = "Aude"
print(prenom == "Aude")
print(prenom == "aude")    # attention, la casse compte !
print("a" < "b")           # ordre alphabétique
```

```
True
False
True
```

2.1.1 Combiner des conditions : `and`, `or`, `not`

On peut combiner plusieurs conditions à l'aide des opérateurs logiques :

- `A and B` est vrai si **A et B sont tous les deux vrais** ;
- `A or B` est vrai si **au moins l'un des deux** est vrai ;
- `not A` est vrai si A est faux (et inversement).

```
age = 20
etudiant = True
print(age >= 18 and etudiant)    # majeur ET étudiant
print(age < 12 or age > 65)      # enfant OU senior
print(not etudiant)
```

```
True
False
False
```

Pour tester si une valeur est comprise dans un intervalle, Python autorise une écriture proche des mathématiques :

```
note = 14
print(10 <= note < 20)    # équivaut à 10 <= note and note < 20
```

```
True
```

Testez-vous

Avec `a = 3` et `b = 8`, que valent les expressions suivantes ?

1. `a > 2 and b > 10`
2. `a > 2 or b > 10`
3. `not (a == 3)`
4. `a != b`
5. `a < b < 5`

2.2 L'instruction `if`

Pour n'exécuter des instructions que si une condition est vraie, on utilise le mot-clé `if` (« si » en anglais) :

```
x = 12
if x > 0:
    print("x est positif")
    print("x n'est pas négatif")
print("Fin du programme")
```

```
x est positif
x n'est pas négatif
Fin du programme
```

Observez bien la syntaxe :

- la ligne du `if` se termine par **deux-points `:`** ;
- les instructions à exécuter si la condition est vraie sont **indentées**, c'est-à-dire décalées vers la droite (de 4 espaces, ou d'une tabulation) : elles forment un **bloc** ;
- le bloc se termine dès qu'une ligne revient au niveau d'indentation du `if` : ici, `print("Fin du programme")` est exécutée dans tous les cas.

Si l'on change la valeur de `x` pour une valeur négative, seule la dernière ligne est exécutée :

```
x = -3
if x > 0:
    print("x est positif")
    print("x n'est pas négatif")
print("Fin du programme")
```

```
Fin du programme
```

À retenir : l'indentation

Dans beaucoup de langages, l'indentation n'est qu'une convention de présentation. **En Python, elle fait partie de la syntaxe** : c'est elle qui indique à Python où commence et où se termine un bloc. Un mauvais décalage change le sens du programme, ou provoque une erreur. La plupart des éditeurs (dont Thonny) indentent automatiquement après un `:` ; utilisez la touche Tab et évitez de mélanger tabulations et espaces.

Testez-vous

Qu'affichent ces deux programmes, qui ne diffèrent que par l'indentation de la dernière ligne ?

```
# Programme A
note = 8
if note >= 10:
    print("Admis")
    print("Bravo")
```

```
# Programme B
note = 8
if note >= 10:
    print("Admis")
print("Bravo")
```

2.3 `else` : sinon...

Souvent, on veut exécuter un bloc si la condition est vraie, et **un autre bloc sinon**. On utilise alors le mot-clé `else` (« sinon ») :

```
note = 8
if note >= 10:
    print("Admis")
else:
    print("Ajourné")
```

```
Ajourné
```

Le mot-clé `else` est placé **au même niveau d'indentation que le `if`** auquel il se rapporte, suivi lui aussi de `:`. Un seul des deux blocs est exécuté, jamais les deux.

2.4 `elif` : sinon si...

Lorsqu'il y a plus de deux cas, on enchaîne les tests avec `elif` (contraction de *else if*, « sinon si ») :

```
t = 25
if t <= 0:
    print("L'eau est solide")
elif t < 100:
```

(suite sur la page suivante)

(suite de la page précédente)

```
    print("L'eau est liquide")
else:
    print("L'eau est gazeuse")
```

```
L'eau est liquide
```

Python teste les conditions **dans l'ordre** et exécute **uniquement le bloc de la première condition vraie** ; si aucune n'est vraie, c'est le bloc `else` (facultatif) qui est exécuté.

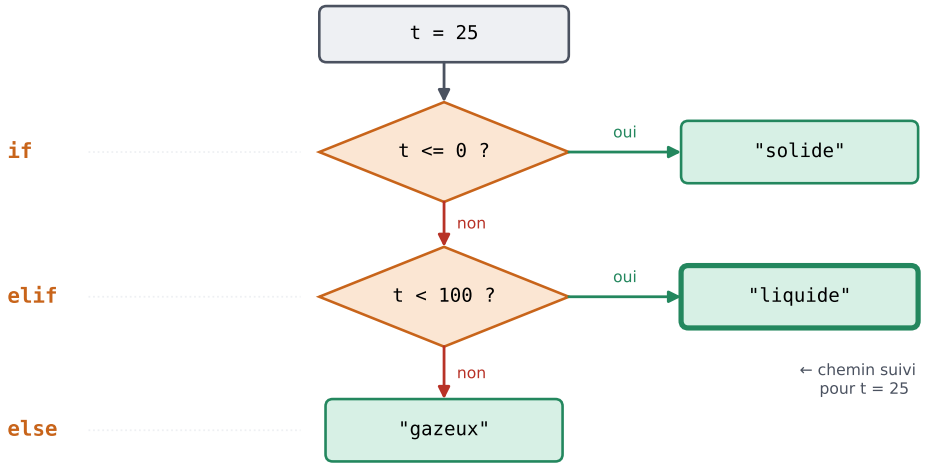


Figure2.1: Le chemin suivi par le programme précédent pour `t = 25` : la première condition est fausse, la deuxième est vraie, on affiche donc « liquide » et on ignore la suite.

Voir ce code s'exécuter pas à pas dans Python Tutor

Remarquez qu'on n'a pas eu besoin d'écrire `elif t > 0 and t < 100` : si l'on arrive au `elif`, c'est que la condition `t <= 0` était fausse, donc que `t > 0`.

Erreur fréquente : l'ordre des tests

Comme seule la **première** condition vraie compte, l'ordre des tests est important. Que penser du programme suivant, censé attribuer une mention ?

```
note = 17
if note >= 10:
    print("Passable")
elif note >= 14:
    print("Bien")
elif note >= 16:
    print("Très bien")
```

Il affiche `Passable` pour une note de 17 ! En effet, la première condition (`note >= 10`) est vraie, donc les suivantes ne sont même pas testées. Il faut tester les conditions **de la plus restrictive à la moins restrictive** : d'abord `note >= 16`, puis `note >= 14`, puis `note >= 10`.

Testez-vous

Qu'affiche ce programme pour `x = 5` ? Pour `x = -1` ? Pour `x = 0` ?

```
if x > 0:
    print("A")
elif x > -2:
    print("B")
else:
    print("C")
print("D")
```

2.5 Conditions imbriquées

Un bloc peut lui-même contenir un `if` : on parle de conditions **imbriquées**. Chaque niveau d'imbrication ajoute un niveau d'indentation.

```
age = 20
etudiant = True
if age >= 18:
    if etudiant:
        print("Tarif étudiant : 5 euros")
    else:
        print("Plein tarif : 9 euros")
else:
    print("Tarif enfant : 4 euros")
```

```
Tarif étudiant : 5 euros
```

Lorsque c'est possible, on préfère toutefois éviter les imbrications trop profondes, qui rendent le code difficile à lire : un `and` ou un `elif` permettent souvent de « mettre à plat » la structure.

Erreurs fréquentes de ce chapitre

Message d'erreur	Cause probable
`SyntaxError: expected ':'`	les deux-points oubliés à la fin d'une ligne `if`, `elif` ou `else`
`IndentationError: expected an indented block`	la ligne qui suit un `:` n'est pas indentée
`IndentationError: unexpected indent`	une ligne est indentée alors qu'elle ne devrait pas l'être
`IndentationError: unindent does not match any outer indentation level`	les décalages ne sont pas cohérents (par exemple 4 espaces puis 3)
`SyntaxError: invalid syntax` sur un `if x = 5:`	`=` utilisé à la place de `==` dans une condition (les versions récentes de Python suggèrent même : *Maybe you meant "==" ...*)
`SyntaxError: invalid syntax` sur un `else`	`else` mal aligné avec son `if`, ou `else` suivi d'une condition (`else x > 5:` est interdit : utiliser `elif`)

Voici par exemple l'erreur obtenue en oubliant d'indenter le contenu d'un bloc :

```
x = 3
if x > 0:
print("positif")
```

```
  Cell In[10], line 3
    print("positif")
    ^
IndentationError: expected an indented block after 'if' statement on line 2
```

Si vous venez de R ou de SAS

- Les blocs sont délimités par l'"**indentation** et non par des accolades (`{ }` en R) ou par `DO; ... END;` (en SAS) : c'est la différence de syntaxe qui surprend le plus au début.
- `else if` s'écrit `elif`, et chaque ligne `if`, `elif`, `else` se termine par `:`.
- Les opérateurs logiques s'écrivent en toutes lettres : `and`, `or`, `not` (au lieu de `&&`, `||`, `!` en R). Attention : avec `pandas`, pour combiner des conditions sur des colonnes entières, on utilisera `&`, `|` et `~` (chapitre 11), comme `&` et `|` en R.
- Le test d'égalité est `==` (comme en R, et non `=` comme dans une instruction `IF` de SAS).

2.6 Exemple guidé : le prix d'un billet de train

Avant de passer aux exercices, voyons sur un exemple comment aborder un problème.

Énoncé : un billet de train coûte 40 €. Les moins de 12 ans paient demi-tarif, les 12–25 ans ont 30 % de réduction, les plus de 60 ans ont 20 % de réduction. Calculer et afficher le prix du billet en fonction de l'âge.

Étape 1 : identifier les cas. Faisons la liste des situations possibles, sur un axe des âges :

- moins de 12 ans : 50 % de réduction ;
- de 12 à 25 ans : 30 % ;
- de 26 à 60 ans : aucune réduction ;
- plus de 60 ans : 20 %.

Les cas ne se chevauchent pas et couvrent tous les âges : une structure `if` / `elif` / `else` convient parfaitement.

Étape 2 : choisir des exemples de test *avant* de programmer. Pour 8 ans, on attend 20 € ; pour 25 ans, 28 € ; pour 40 ans, 40 € ; pour 70 ans, 32 €. Testons aussi les cas « limites » : 12 ans (28 €) et 60 ans (40 €, car « plus de 60 ans » signifie strictement plus).

Étape 3 : écrire le code.

```
age = 25
prix_plein = 40

if age < 12:
    reduction = 0.5
elif age <= 25:
    reduction = 0.3
elif age <= 60:
    reduction = 0
else:
    reduction = 0.2

prix = prix_plein * (1 - reduction)
print(f"À {age} ans, le billet coûte {prix:.2f} euros")
```

```
À 25 ans, le billet coûte 28.00 euros
```

Étape 4 : tester avec toutes les valeurs choisies à l'étape 2 en modifiant la première ligne. Essayez !

Remarquez l'astuce : plutôt que d'écrire quatre fois le calcul et le `print`, on calcule seulement le **taux de réduction** dans les différents cas, puis on fait le calcul une seule fois à la fin.

2.7 Exercices

Exercice 2.1 : Pair ou impair ☆

Écrivez un programme qui, pour un entier stocké dans une variable `n`, affiche `n est pair` ou `n est impair` (avec la valeur de `n` à la place de la lettre `n`).

Exercice 2.2 : Température de l'eau ☆

Écrivez une structure conditionnelle qui, à partir d'une température d'eau stockée dans une variable `t`, affiche si l'eau à cette température est à l'état solide, liquide ou gazeux. Contrairement à l'exemple du cours, testez les conditions en commençant par l'état gazeux.

Exercice 2.3 : Mention ☆ ☆

Écrivez un programme qui, à partir d'une note sur 20 stockée dans `note`, affiche la mention correspondante : « Ajourné » (moins de 10), « Passable » (de 10 inclus à 12 exclu), « Assez bien » (de 12 à 14), « Bien » (de 14 à 16), « Très bien » (16 et plus). Si la note n'est pas comprise entre 0 et 20, le programme doit afficher « Note invalide ».

Exercice 2.4 : Le plus grand de trois nombres ☆ ☆

Écrivez un programme qui affiche le plus grand de trois nombres stockés dans les variables `a`, `b` et `c`, **sans utiliser la fonction `max`** de Python. Testez votre programme en faisant varier la position du plus grand nombre (en `a`, en `b`, puis en `c`) et avec des valeurs égales.

Exercice 2.5 : Année bissextile ☆ ☆ ☆

Une année est bissextile si elle est divisible par 4 mais pas par 100, ou si elle est divisible par 400. Ainsi, 2024 et 2000 sont bissextiles, mais 1900 et 2023 ne le sont pas. Écrivez un programme qui affiche si l'année stockée dans `annee` est bissextile.

2.8 Bilan du chapitre

À retenir

- Une condition vaut `True` ou `False`. Opérateurs : `<`, `>`, `<=`, `>=`, `==`, `!=`, `in`, combinés avec `and`, `or`, `not`.
- == compare, = affecte.
- Syntaxe :

```
if condition1:
    bloc exécuté si condition1 est vraie
elif condition2:
    bloc exécuté si condition1 est fausse et condition2␣
 ↪vraie
else:
    bloc exécuté si aucune condition n'est vraie
```

- Les deux-points `:` et l'**indentation** sont obligatoires.
- Seul le bloc de la **première** condition vraie est exécuté : l'ordre des tests compte.

Faites le point : êtes-vous capable de…

- prédire la valeur (`True` ou `False`) d'une condition combinant `and`, `or` et `not` ?
- écrire un `if` / `elif` / `else` correctement indenté ?
- expliquer pourquoi l'ordre des `elif` est important ?
- choisir des valeurs de test, y compris des cas limites, pour vérifier votre programme ?
- corriger une `IndentationError` ?

CHAPTER 3

LES BOUCLES

Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- répéter des instructions tant qu'une condition est vraie (boucle `while`) ;
- répéter des instructions pour chaque élément d'une liste (boucle `for`) ;
- générer des suites d'entiers avec `range` ;
- utiliser des variables « accumulateurs » (somme, compteur...) ;
- suivre l'exécution d'une boucle à la main, avec un tableau de suivi.

Prérequis : chapitres 1 et 2. **Durée indicative :** 2h.

Imaginons que l'on veuille afficher les nombres de 1 à 100. Écrire cent fois `print` serait fastidieux... et un programme qui doit traiter un million de données ne peut évidemment pas être écrit ainsi. Les **boucles** permettent de **répéter** une série d'instructions autant de fois que nécessaire.

Il existe en Python deux types de boucles :

- la boucle `while`, qui répète des instructions **tant qu'**une condition est vraie ;
- la boucle `for`, qui répète des instructions **pour chaque** valeur d'une séquence (par exemple une liste).

3.1 La boucle `while`

3.1.1 Syntaxe

La boucle `while` (« tant que » en anglais) a une syntaxe très proche de celle du `if` :

```
x = 0
while x <= 10:
    print(x)
    x = 2 * x + 2
print("Fin de la boucle, x vaut", x)
```

```
0
2
6
Fin de la boucle, x vaut 14
```

Son fonctionnement est le suivant :

1. on teste la condition (`x <= 10`) ;
2. si elle est vraie, on exécute le bloc indenté (le **corps** de la boucle), puis **on revient à l'étape 1** ;
3. si elle est fausse, on passe à la suite du programme, après la boucle.

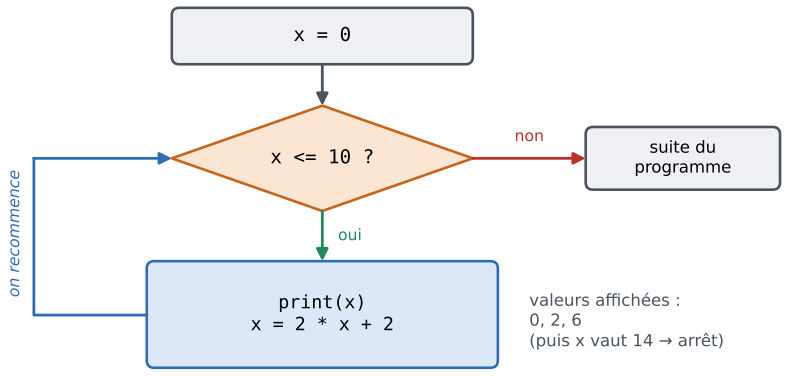


Figure3.1: Fonctionnement de la boucle `while` de l'exemple ci-dessus.

Voir ce code s'exécuter pas à pas dans Python Tutor

3.1.2 Suivre une boucle à la main : le tableau de suivi

Pour comprendre (ou vérifier) ce que fait une boucle, une technique très efficace consiste à dresser un **tableau de suivi** : une ligne par tour de boucle, une colonne par variable. Pour l'exemple précédent :

Moment	`x`	Condition `x <= 10`	Affichage
avant la boucle	0	vraie	`0`
après le tour 1	2	vraie	`2`
après le tour 2	6	vraie	`6`
après le tour 3	14	**fausse** → sortie	

À la sortie de la boucle, `x` vaut 14. Prenez l'habitude de faire ce type de tableau sur papier lorsqu'une boucle ne fait pas ce que vous attendez : c'est exactement ce que fait Python Tutor pour vous.

3.1.3 Attention aux boucles infinies

Si la condition d'une boucle `while` ne devient jamais fausse, la boucle ne s'arrête jamais :

```
x = 2
y = 0
while x > 0:
    y = y - 1
    print(y)
print("Si on arrive ici, on a fini")
```

Ici, `x` vaut 2 et n'est jamais modifiée dans la boucle : la condition `x > 0` reste vraie pour toujours et le programme affiche -1, -2, -3... sans fin.

À retenir

Dans une boucle `while`, le corps de la boucle doit **modifier au moins une des variables de la condition**, de façon à ce que celle-ci finisse par devenir fausse. Si votre programme semble « bloqué », il est probablement pris dans une boucle infinie : interrompez-le avec la combinaison de touches `Ctrl + C` (ou le bouton *Stop* de Thonny).

Testez-vous

Qu'affiche le programme suivant ? Construisez le tableau de suivi.

```
n = 1
while n < 20:
    n = n * 3
print(n)
```

3.2 La boucle `for`

3.2.1 Parcourir une liste

Le second type de boucle utilise une **liste**, c'est-à-dire une suite ordonnée de valeurs écrites entre crochets et séparées par des virgules (nous étudierons les listes en détail au chapitre 5). La boucle `for` exécute son corps **une fois pour chaque élément** de la liste :

```
for x in [1, 5, 7]:
    print(x)
print("Fin de la boucle")
```

```
1
5
7
Fin de la boucle
```

À chaque tour, la variable `x` (appelée **variable de boucle**) prend successivement pour valeur chacun des éléments de la liste, dans l'ordre.

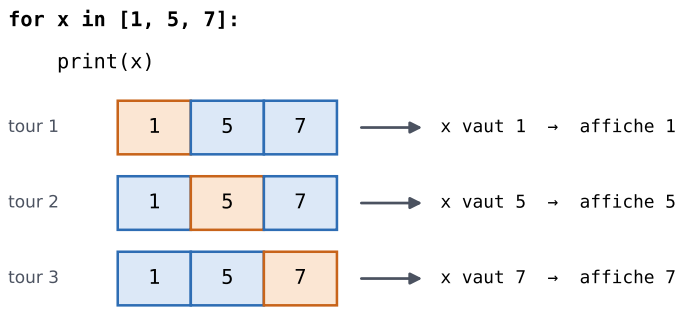


Figure3.2: À chaque tour de boucle, `x` prend la valeur de l'élément suivant de la liste.

Le nom de la variable de boucle est libre, et la liste peut contenir n'importe quel type de valeurs :

```
for prenom in ["Aude", "Ali", "Chloé"]:
    print(f"Bonjour {prenom} !")
```

```
Bonjour Aude !
Bonjour Ali !
Bonjour Chloé !
```

3.2.2 Répéter un nombre donné de fois : `range`

Il est très fréquent de vouloir parcourir une suite d'entiers. C'est le rôle de la fonction `range` :

```
for i in range(5):
    print(i)
```

```
0
1
2
3
4
```

`range` peut prendre un, deux ou trois arguments :

- `range(n)` produit les entiers de `0` à `n - 1` (soit `n` valeurs) ;
- `range(a, b)` produit les entiers de `a` (inclus) à `b` (**exclu**) ;
- `range(a, b, pas)` produit les entiers de `a` à `b` (exclu) en avançant de `pas` à chaque fois.

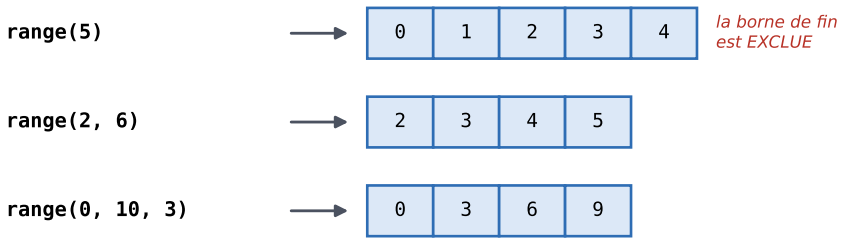


Figure3.3: Les valeurs produites par `range` dans trois cas. La borne de fin n'est jamais atteinte.

```
for i in range(1, 10, 2):
    print(i)
```

```
1
3
5
7
9
```

Pour répéter une action `n` fois, on écrit donc simplement `for i in range(n):` (même si l'on n'utilise pas la valeur de `i`) :

```
for i in range(3):
    print("Hip hip hip, hourra !")
```

```
Hip hip hip, hourra !
Hip hip hip, hourra !
Hip hip hip, hourra !
```

Testez-vous

Quelles valeurs sont produites par `range(3)`, `range(2, 5)`, `range(10, 0, -3)` et `range(5, 5)` ?

Pour info

Si vous avez déjà programmé dans un autre langage, vous avez peut-être l'habitude que les boucles `for` parcourent des entiers (des indices). En Python, la boucle `for` parcourt directement **les éléments** d'une séquence ; `range` n'est qu'un cas particulier de séquence. Nous verrons au chapitre 5 *comment parcourir une liste avec ses indices* lorsque c'est nécessaire.

3.3 Les accumulateurs

Une utilisation très courante des boucles consiste à **accumuler** un résultat au fil des tours : faire une somme, compter des éléments, construire un texte... Le principe est toujours le même :

1. **avant** la boucle, on initialise une variable (à 0 pour une somme ou un compteur) ;
2. **dans** la boucle, on met à jour cette variable ;
3. **après** la boucle, on utilise le résultat.

3.3.1 Exemple guidé : la somme des entiers de 1 à n

Calculons $1 + 2 + \cdots + n$ pour $n = 4$:

```
n = 4
somme = 0                       # 1. initialisation
for i in range(1, n + 1):       # i prend les valeurs 1, 2, ..., n
    somme = somme + i           # 2. mise à jour
print(somme)                    # 3. utilisation
```

```
10
```

Le tableau de suivi permet de voir comment la somme se construit :

Tour	`i`	`somme` après le tour
(avant)	–	0
1	1	1
2	2	3
3	3	6
4	4	10

Voir ce code s'exécuter pas à pas dans Python Tutor

Erreur fréquente

- **Initialiser l'accumulateur dans la boucle** : si l'on place `somme = 0` à l'intérieur de la boucle, la somme est remise à zéro à chaque tour !
- **Oublier le `+ 1` dans `range(1, n + 1)`** : `range(1, n)` s'arrête à `n - 1`.
- **Mettre le `print` dans la boucle** alors qu'on ne veut afficher que le résultat final (ou l'inverse) : vérifiez l'indentation.

3.3.2 Compter

Pour compter, on ajoute 1 à un compteur à chaque fois qu'une condition est vérifiée. On combine alors une boucle et un `if` :

```
notes = [12, 7, 15, 9, 10, 18]
nb_admis = 0
for note in notes:
    if note >= 10:
        nb_admis += 1
print(nb_admis, "étudiants admis")
```

```
4 étudiants admis
```

3.4 Quelle boucle choisir ?

Au début, il peut être difficile de choisir entre `for` et `while`. Voici une règle simple :

- si l'on veut parcourir les éléments d'une liste (ou d'une autre collection), ou si l'on **connaît à l'avance le nombre de tours**, on utilise une boucle `for` (avec `range` si besoin) ;
- si l'on veut répéter une action **tant qu'**une condition est vérifiée, sans savoir à l'avance combien de tours seront nécessaires, on utilise une boucle `while`.

Par exemple, « afficher les 10 premiers multiples de 7 » appelle une boucle `for`, tandis que « combien d'années faut-il pour doubler un capital placé à 3 % ? » appelle une boucle `while`.

3.5 Boucles imbriquées

Le corps d'une boucle peut contenir une autre boucle. La boucle intérieure est alors exécutée entièrement à chaque tour de la boucle extérieure :

```
for i in range(1, 4):
    for j in range(1, 4):
        print(f"{i} x {j} = {i * j}")
    print("---")
```

```
1 x 1 = 1
1 x 2 = 2
1 x 3 = 3
---
2 x 1 = 2
2 x 2 = 4
2 x 3 = 6
---
3 x 1 = 3
3 x 2 = 6
3 x 3 = 9
---
```

Ici, la boucle sur `j` fait 3 tours pour **chacun** des 3 tours de la boucle sur `i` : il y a donc 9 affichages de produits en tout.

Pour aller plus loin : interrompre une boucle

L'instruction `break` permet de sortir immédiatement d'une boucle, et `continue` de passer directement au tour suivant. Elles sont parfois pratiques, mais on peut toujours s'en passer, et un code sans `break` est souvent plus facile à relire. Dans ce document, nous ne les utiliserons pas.

Si vous venez de R ou de SAS

- `range(a, b)` **exclut** la borne `b` : `range(1, 4)` produit 1, 2, 3, alors que `1:4` en R produit 1, 2, 3, 4.
- Comme en R, on évite les boucles explicites pour traiter des **données** : avec `numpy` et `pandas`, les calculs sur des colonnes entières sont « vectorisés » (chapitres 10 à 12). Les boucles restent utiles pour tout le reste (parcourir des fichiers, répéter une simulation, construire un résultat…).
- En SAS, l'étape DATA « boucle » implicitement sur les observations ; en Python, rien de tel : les instructions sont exécutées une fois, sauf boucle explicite.

3.6 Exercices

Exercice 3.1 : Nombres impairs ☆

Écrivez une boucle permettant d'afficher tous les nombres impairs strictement inférieurs à une valeur `n` initialement fixée. Proposez deux versions : l'une utilisant l'opérateur `%`, l'autre non.

Exercice 3.2 : Table de multiplication ☆

Écrivez un programme qui affiche la table de multiplication d'un nombre `n` (de `n x 1` à `n`

`x 10`), sous la forme :

```
7 x 1 = 7
7 x 2 = 14
...
7 x 10 = 70
```

Exercice 3.3 : Somme des carrés ☆

Calculez et affichez la somme $1^2 + 2^2 + \cdots + n^2$ pour une valeur de `n` donnée. Pour `n = 10`, vous devez obtenir 385.

Exercice 3.4 : Factorielle ☆ ☆

La factorielle d'un entier n, notée $n!$, est le produit $1 \times 2 \times \cdots \times n$ (et par convention $0! = 1$). Écrivez un programme qui calcule et affiche $n!$ pour une valeur de `n` donnée. Vous devez obtenir $5! = 120$ et $0! = 1$.

Exercice 3.5 : Doubler son capital ☆ ☆

On place 1000 € sur un compte rémunéré à 3 % par an (chaque année, le capital est multiplié par 1,03). Écrivez un programme qui calcule le nombre d'années nécessaires pour que le capital dépasse 2000 €, et affiche ce nombre ainsi que le capital final (arrondi au centime).

Exercice 3.6 : Suite récurrente ☆ ☆

Écrivez un programme qui affiche tous les termes plus petits que 1000 de la suite (u_n) définie par :

$$\begin{aligned} u_0 &= 2 \\ \forall n \geq 1,\ u_n &= u_{n-1}^2 \end{aligned}$$

Exercice 3.7 : Triangle d'étoiles ☆ ☆

Écrivez un programme qui affiche un triangle de `n` lignes, la ligne numéro `i` contenant `i` étoiles. Pour `n = 4` :

```
*
**
***
****
```

Puis, en bonus, affichez le même triangle aligné à droite.

Exercice 3.8 : Nombre premier ☆ ☆ ☆

Un nombre entier $n \geq 2$ est **premier** s'il n'est divisible que par 1 et par lui-même (2, 3, 5, 7, 11, 13... sont premiers ; $9 = 3 \times 3$ ne l'est pas). Écrivez un programme qui affiche si l'entier `n` est premier.

3.7 Bilan du chapitre

À retenir

- `while condition:` répète le bloc **tant que** la condition est vraie ; la condition doit finir par devenir fausse.
- `for element in liste:` répète le bloc **pour chaque** élément de la liste.
- `range(n)` produit 0, 1, ..., n-1 ; `range(a, b)` produit a, ..., b-1 ; `range(a, b, pas)` avance de `pas` en `pas`.
- Accumulateur : on l'**initialise avant** la boucle, on le **met à jour dans** la boucle, on l'**utilise après**.
- En cas de doute, faites un **tableau de suivi** ou utilisez Python Tutor.

Faites le point : êtes-vous capable de...

- écrire une boucle `for` qui parcourt une liste ou un `range` ?
- prédire les valeurs produites par un `range` à 1, 2 ou 3 arguments ?
- écrire une boucle `while` qui s'arrête bien ?
- calculer une somme, un produit ou un comptage avec un accumulateur ?
- dresser le tableau de suivi d'une boucle ?
- choisir entre `for` et `while` pour un problème donné ?

CHAPTER 4

LES FONCTIONS ET LES MODULES

Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- définir vos propres fonctions avec `def` et les appeler ;
- faire la différence entre **afficher** un résultat (`print`) et **retourner** un résultat (`return`) ;
- utiliser des arguments nommés et des arguments ayant une valeur par défaut ;
- comprendre la notion de variable locale ;
- importer et utiliser des modules (comme `math` ou `random`).

Prérequis : chapitres 1 à 3. **Durée indicative :** 2h.

4.1 Des fonctions, vous en utilisez déjà

Nous avons déjà utilisé plusieurs fonctions sans le dire : `print`, `type`, `int`, `range`... Lorsque l'on écrit :

```
x = 3.7
print(x)
```

```
3.7
```

on **appelle** la fonction `print` en lui donnant un **argument** (ici la variable `x`). La fonction `print` ne fait qu'afficher une information. D'autres fonctions, comme `type` ou `int`, **retournent** (on dit aussi **renvoient**) une valeur que l'on peut réutiliser dans la suite du programme, par exemple en la stockant dans une variable :

```
t1 = type(1)       # on stocke dans t1 la valeur retournée par type
t2 = type(2.)
if t1 == t2:
    print("types identiques")
else:
    print("types différents")
```

```
types différents
```

4.2 Définir ses propres fonctions

4.2.1 Pourquoi ?

Lorsqu'une suite d'instructions est susceptible d'être utilisée à plusieurs reprises, il est recommandé de l'isoler dans une **fonction**. Cela présente plusieurs avantages :

- le code est **plus lisible** : un nom de fonction bien choisi (`calcule_moyenne`, `est_premier`) explique ce que fait le code ;
- le code est **plus facile à corriger et à faire évoluer** : s'il faut corriger une erreur, on ne le fait qu'à un seul endroit (dans la fonction) et non à chaque fois que le code est répété ;
- on peut **découper un problème complexe** en sous-problèmes plus simples, chacun étant résolu par une fonction.

4.2.2 Syntaxe

Pour définir une fonction, on utilise le mot-clé `def` :

```
def carre_plus_un(x):
    y = x ** 2
    return y + 1
```

On a défini ici une fonction :

- dont le **nom** est `carre_plus_un` ;
- qui prend un **argument** (on dit aussi **paramètre**), noté `x` ;
- dont le **corps** (le bloc indenté) calcule une valeur `y` ;
- qui **retourne** une valeur, comme indiqué par le mot-clé `return`.

Cette cellule n'affiche rien : définir une fonction, c'est comme écrire une recette de cuisine, cela ne la réalise pas. Pour exécuter la fonction, il faut l'**appeler**, en écrivant son nom suivi d'arguments entre parenthèses :

```
resultat = carre_plus_un(3)
print(resultat)
print(carre_plus_un(10))
```

```
10
101
```

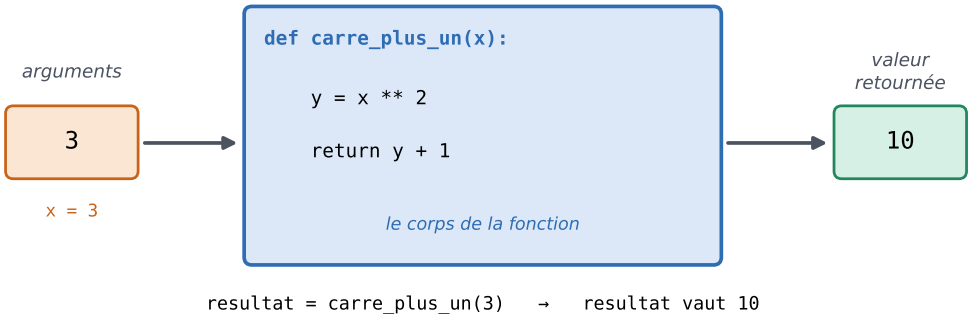


Figure4.1: Une fonction est comme une machine : on lui fournit des arguments en entrée, elle exécute son corps et renvoie une valeur en sortie.

Lors de l'appel `carre_plus_un(3)`, voici ce qui se passe :

1. Python « saute » dans le corps de la fonction, où `x` prend la valeur `3` ;
2. il exécute le corps de la fonction : `y` vaut 9 ;
3. en arrivant au `return`, il calcule `y + 1` (soit 10), **quitte la fonction** et revient là où la fonction avait été appelée ;
4. l'appel `carre_plus_un(3)` est alors remplacé par la valeur retournée, 10, qui est rangée dans `resultat`.

Voir ce code s'exécuter pas à pas dans Python Tutor

Une fonction peut prendre plusieurs arguments, séparés par des virgules, ou aucun argument (on garde alors les parenthèses vides) :

```
def moyenne(a, b):
    return (a + b) / 2

def dire_bonjour():
    print("Bonjour !")

print(moyenne(12, 15))
dire_bonjour()
```

```
13.5
Bonjour !
```

À retenir

- La définition (`def ...:`) doit se trouver **avant** le premier appel de la fonction dans le programme.
- La ligne `def` se termine par `:` et le corps de la fonction est **indenté**.
- Pour appeler une fonction, on écrit **toujours** des parenthèses, même sans argument : `dire_bonjour()`. Sans parenthèses, la fonction n'est pas exécutée.

4.2.3 `print` ou `return` ?

C'est **la** confusion la plus fréquente chez les débutant·e·s. Comparons deux fonctions qui semblent faire la même chose :

```
def double_affiche(x):
    print(x * 2)

def double_retourne(x):
    return x * 2
```

```
a = double_affiche(3)
b = double_retourne(3)
print("a vaut", a)
print("b vaut", b)
```

```
6
a vaut None
b vaut 6
```

La première fonction **affiche** 6 à l'écran mais ne retourne rien : en l'absence de `return`, une fonction retourne la valeur spéciale `None`. La seconde n'affiche rien, mais **retourne** 6, une valeur que l'on peut stocker, afficher ou réutiliser dans un calcul :

```
print(double_retourne(3) + 1)
print(double_retourne(double_retourne(5)))
```

```
7
20
```

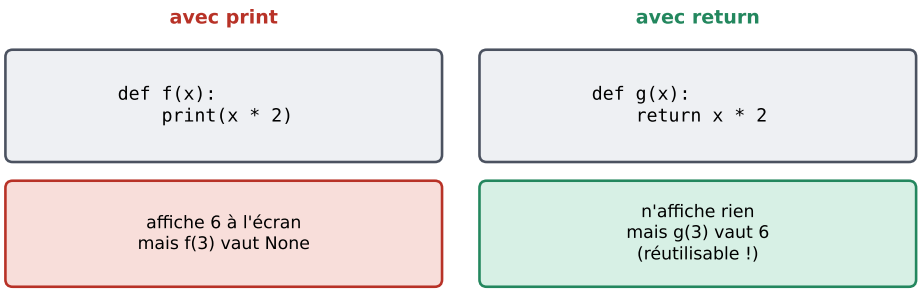


Figure4.2: `print` affiche une valeur pour l'utilisateur ; `return` transmet une valeur au reste du programme.

À retenir

- `print` sert à **communiquer avec l'utilisateur** (afficher un message).
- `return` sert à **communiquer avec le reste du programme** (transmettre un résultat).

Sauf si une fonction a explicitement pour rôle d'afficher quelque chose, elle doit en général **retourner** son résultat avec `return`, et c'est le programme principal qui décide quoi en faire (l'afficher, le stocker…).

Notez aussi que `return` **termine immédiatement** l'exécution de la fonction : les instructions situées après un `return` exécuté ne sont jamais exécutées.

```
def signe(x):
    if x > 0:
        return "positif"
    elif x < 0:
        return "négatif"
    return "nul"    # on n'arrive ici que si aucun return n'a été␣
 ↪exécuté avant

print(signe(-4))
print(signe(0))
```

```
négatif
nul
```

Testez-vous

Qu'affiche le programme suivant ?

```
def f(x):
    return x + 1
    print("coucou")

def g(x):
    print(x + 1)

a = f(1)
b = g(1)
print(a, b)
```

4.2.4 Retourner plusieurs valeurs

Une fonction peut retourner plusieurs valeurs, séparées par des virgules. On peut alors les récupérer dans plusieurs variables :

```
def division(a, b):
    quotient = a // b
    reste = a % b
    return quotient, reste

q, r = division(17, 5)
print("quotient :", q, "reste :", r)
```

```
quotient : 3 reste : 2
```

4.3 Arguments nommés et valeurs par défaut

4.3.1 Nommer les arguments lors de l'appel

Lors de l'appel d'une fonction, on peut préciser le nom des arguments, ce qui évite de se tromper dans leur ordre. Avec la fonction suivante :

```
def affiche_infos_personne(poids, taille):
    print("Poids :", poids)
    print("Taille :", taille)
```

les trois appels suivants sont équivalents :

```
affiche_infos_personne(80, 180)
affiche_infos_personne(taille=180, poids=80)
affiche_infos_personne(poids=80, taille=180)
```

```
Poids : 80
Taille : 180
Poids : 80
Taille : 180
Poids : 80
Taille : 180
```

Quand les arguments sont nommés, leur ordre n'a plus d'importance. Pour que ce soit vraiment utile, il faut bien sûr choisir des **noms d'arguments explicites**.

4.3.2 Arguments optionnels

Certains arguments peuvent avoir une **valeur par défaut**, utilisée si on ne précise pas de valeur lors de l'appel. Par exemple, la fonction `print` a un argument optionnel `end` qui indique ce qui est affiché à la fin (par défaut, un retour à la ligne, noté `"\n"`) :

```
print("La vie est belle")
print("Life is beautiful")
print("La vie est belle", end=" -- ")
print("Life is beautiful", end="*-*")
```

```
La vie est belle
Life is beautiful
La vie est belle -- Life is beautiful*-*
```

Pour donner une valeur par défaut à un argument de vos propres fonctions, on utilise la syntaxe suivante :

```
def prix_ttc(prix_ht, taux=0.2):    # la valeur par défaut de taux␣
↪est 0.2
    return prix_ht * (1 + taux)

print(prix_ttc(100))                # taux vaut 0.2
print(prix_ttc(100, taux=0.055))    # taux vaut 0.055
```

```
120.0
105.5
```

Attention, les arguments ayant une valeur par défaut doivent être placés **après** tous les arguments obligatoires. La définition suivante **n'est pas correcte** :

```
def f(x, y=0, z):
    return x - 2 * y + z
```

4.4 Variables locales

Les variables créées à l'intérieur d'une fonction (y compris ses arguments) sont **locales** : elles n'existent que pendant l'exécution de la fonction et ne sont pas visibles à l'extérieur.

```
def calcule():
    resultat_local = 42
    return resultat_local

valeur = calcule()
print(valeur)
print(resultat_local)  # erreur : cette variable n'existe que␣
 ↪dans la fonction
```

```
42
```

```
---------------------------------------------------------------------
 ↪-----------
NameError                                 Traceback (most␣
 ↪recent call last)
Cell In[15], line 7
      3     return resultat_local
      4
      5 valeur = calcule()
      6 print(valeur)
----> 7 print(resultat_local)  # erreur : cette variable n
 ↪'existe que dans la fonction

NameError: name 'resultat_local' is not defined
```

Cela a une conséquence importante : une variable locale peut porter le même nom qu'une variable du programme principal **sans interférer** avec elle.

```
def f(x):
    x = x * 10
    return x

x = 5
y = f(x)
print(x, y)
```

```
5 50
```

Ici, la variable `x` de la fonction est une autre variable que le `x` du programme principal : modifier l'une ne modifie pas l'autre. Dans Python Tutor, on voit bien que chaque appel de fonction crée son propre « cadre » (*frame*) contenant ses variables locales.

Voir ce code s'exécuter pas à pas dans Python Tutor

Bonne pratique

Une fonction doit recevoir **toutes les données dont elle a besoin par ses arguments**, et transmettre son résultat par `return`. Évitez d'utiliser à l'intérieur d'une fonction des variables définies à l'extérieur : votre fonction sera plus facile à comprendre, à tester et à réutiliser.

4.4.1 Documenter ses fonctions

On peut (et on doit, dès que la fonction n'est pas triviale) décrire ce que fait une fonction avec une chaîne de caractères placée juste sous la ligne `def`, appelée *docstring* :

```
def aire_rectangle(largeur, longueur):
    """Retourne l'aire d'un rectangle de dimensions données."""
    return largeur * longueur

help(aire_rectangle)
```

```
Help on function aire_rectangle in module __main__:

aire_rectangle(largeur, longueur)
    Retourne l'aire d'un rectangle de dimensions données.
```

4.5 Les modules

Les fonctions comme `print` ou `len` sont toujours disponibles. D'autres fonctions sont rangées dans des **modules**, qu'il faut **importer** avant de pouvoir les utiliser.

Par exemple, le module `math` propose des fonctions mathématiques usuelles (`sqrt` pour la racine carrée, `sin`, `cos`, `log`...) et des constantes comme `pi` :

```
import math

print(math.sqrt(2))
print(math.pi)
print(math.cos(2 * math.pi))
print(math.floor(3.7))    # arrondi à l'entier inférieur
```

```
1.4142135623730951
3.141592653589793
1.0
3
```

On accède au contenu d'un module avec la syntaxe `module.nom` : `math.sqrt` signifie « la fonction `sqrt` du module `math` ».

Le module `random` permet quant à lui de générer des nombres au hasard, ce qui est bien pratique pour faire des simulations ou des jeux :

```
import random

de = random.randint(1, 6)    # entier au hasard entre 1 et 6␣
 ↪(inclus)
print("Le dé est tombé sur", de)
print(random.choice(["pile", "face"]))
```

```
Le dé est tombé sur 3
face
```

(Si vous exécutez ce code plusieurs fois, vous obtiendrez des résultats différents.)

On peut aussi **renommer** un module lors de son import, ce qui est pratique pour les modules dont on utilise beaucoup les fonctions. Par exemple, il est d'usage d'importer le module `numpy` (voir *le chapitre qui lui est consacré*) sous le nom `np` :

```
import numpy as np

print(np.zeros(5))
```

```
[0. 0. 0. 0. 0.]
```

Bonne pratique

Placez toutes vos instructions `import` **au tout début** de vos fichiers Python. Ainsi, les modules sont chargés avant toute utilisation, et on voit d'un coup d'œil de quels modules le programme a besoin.

Pour aller plus loin

- On peut aussi importer une fonction précise d'un module avec `from math import sqrt`, puis l'utiliser directement sous le nom `sqrt` (sans `math.`).
- La liste des fonctions d'un module est disponible dans la documentation officielle de Python (par exemple, celle du module `math`).
- N'importe quel fichier `.py` que vous écrivez peut servir de module : si vous avez écrit des fonctions dans un fichier `outils.py`, vous pouvez les utiliser dans un autre fichier du même dossier avec `import outils`.

Erreurs fréquentes de ce chapitre

Symptôme / message	Cause probable
La fonction « ne fait rien »	elle a été définie mais jamais appelée, ou appelée sans parenthèses (`f` au lieu de `f()`)
`None` s'affiche, ou `TypeError: unsupported operand type(s) for +: 'NoneType' and 'int'`	la fonction utilise `print` au lieu de `return`, ou il manque un `return` dans certains cas
`NameError: name 'f' is not defined`	la fonction est appelée avant d'être définie (ou son nom est mal orthographié)
`TypeError: f() missing 1 required positional argument: 'y'`	il manque un argument lors de l'appel
`TypeError: f() takes 1 positional argument but 2 were given`	trop d'arguments lors de l'appel
`NameError: name 'math' is not defined`	le module n'a pas été importé

Si vous venez de R ou de SAS

- `def f(x):` correspond à `f <- function(x) { ... }` en R, ou à une macro SAS (`%macro f(x); ... %mend;`), en beaucoup plus simple à manipuler.
- **Piège important** : en R, une fonction retourne automatiquement la valeur de sa dernière expression. En Python, **il faut écrire `return`**, sinon la fonction retourne `None`.
- Les arguments nommés et les valeurs par défaut fonctionnent comme en R.
- `import module` correspond à `library(package)` ; mais en Python, on préfixe les fonctions par le nom (ou l'alias) du module (`np.mean`, `pd.read_csv`), ce qui évite les conflits de noms entre paquets. `module.fonction` joue le rôle de `paquet::fonction` en R.

4.6 Exercices

Exercice 4.1 : Triangle équilatéral ☆

Écrivez une fonction `aire_equi` qui prend en argument une longueur `cote` et retourne l'aire du triangle équilatéral de côté `cote`. On rappelle que la hauteur d'un tel triangle vaut cote $\times \sin(\pi/3)$.

Exercice 4.2 : Est-ce pair ? ☆

Écrivez une fonction `est_pair` qui prend en argument un entier `n` et **retourne** `True` si `n` est pair et `False` sinon. Utilisez-la ensuite dans une boucle pour afficher les nombres pairs entre 0 et 10.

Exercice 4.3 : Maximum de trois nombres ☆ ☆

1. Écrivez une fonction `max2(a, b)` qui retourne le plus grand de deux nombres (sans utiliser la fonction `max` de Python).
2. Écrivez une fonction `max3(a, b, c)` qui retourne le plus grand de trois nombres, **en utilisant `max2`**.

Exercice 4.4 : Conversion de durée ☆ ☆

Écrivez une fonction `convertit_duree(secondes)` qui retourne **trois** valeurs : le nombre d'heures, de minutes et de secondes correspondant à une durée exprimée en secondes (voir l'exercice 1.3). Écrivez ensuite une fonction `formate_duree(secondes)` qui utilise `convertit_duree` et retourne une chaîne de caractères de la forme `"1 h 6 min 40 s"`.

Exercice 4.5 : Suite récurrente, version fonction ☆ ☆

Écrivez une fonction `affiche_u_n(seuil)` qui affiche tous les termes plus petits que `seuil` de la suite (u_n) définie par :

$$\begin{aligned} u_0 &= 2 \\ \forall n \geq 1,\ u_n &= u_{n-1}^2 \end{aligned}$$

L'argument `seuil` doit valoir 1000 par défaut.

Exercice 4.6 : Nombres premiers ☆ ☆ ☆

1. Écrivez une fonction `est_premier(n)` qui retourne `True` si `n` est premier et `False` sinon (reprenez l'exercice 3.8).
2. Écrivez une fonction `affiche_premiers(n_max)` qui affiche tous les nombres premiers inférieurs ou égaux à `n_max`, en utilisant `est_premier`.
3. Écrivez une fonction `nb_premiers(n_max)` qui **retourne** le nombre de nombres premiers inférieurs ou égaux à `n_max`. Il y en a 25 jusqu'à 100.

Exercice 4.7 : Lancers de dés ☆ ☆ ☆

1. Écrivez une fonction `lance_des(nb_des)` qui simule le lancer de `nb_des` dés à six faces et retourne la somme obtenue.

2. Écrivez une fonction `frequence_somme(somme_visee, nb_lancers)` qui lance `nb_lancers` fois deux dés et retourne la proportion des lancers pour lesquels la somme vaut `somme_visee`.
3. Avec 100 000 lancers, quelle est (approximativement) la probabilité d'obtenir 7 avec deux dés ? Et 12 ? Comparez avec les valeurs théoriques $6/36 \approx 0{,}167$ et $1/36 \approx 0{,}028$.

4.7 Bilan du chapitre

À retenir

- Définition : `def nom_fonction(arg1, arg2):` suivi du corps indenté ; appel : `nom_fonction(valeur1, valeur2)`.
- `return valeur` termine la fonction et transmet `valeur` au programme ; sans `return`, la fonction retourne `None`.
- `print` affiche pour l'utilisateur, `return` transmet au programme : ne pas les confondre !
- Arguments nommés : `f(taille=180, poids=80)` ; valeur par défaut : `def f(x, y=0):`.
- Les variables d'une fonction sont **locales**.
- `import module` (ou `import module as alias`), puis `module.fonction(...)`.

Faites le point : êtes-vous capable de...

- définir une fonction avec des arguments et une valeur de retour, puis l'appeler ?
- expliquer la différence entre une fonction qui affiche et une fonction qui retourne ?
- prédire ce qu'affiche un programme qui appelle des fonctions ?
- écrire une fonction qui en utilise une autre ?
- donner une valeur par défaut à un argument ?
- importer le module `math` ou `random` et utiliser l'une de ses fonctions ?

CHAPTER 5

LES LISTES

Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- créer une liste et accéder à ses éléments grâce à leurs indices (y compris négatifs) ;
- extraire une sous-liste (*slicing*) ;
- parcourir une liste de trois façons différentes ;
- ajouter, supprimer, rechercher et trier des éléments ;
- comprendre pourquoi `liste2 = liste` ne copie pas une liste ;
- (bonus) écrire des listes en compréhension et parcourir plusieurs listes avec `zip`.

Prérequis : chapitres 1 à 4. **Durée indicative :** 2h30.

Définition

Une **liste** est une collection **ordonnée** de valeurs. Dans une liste, chaque valeur occupe une position bien définie que l'on repère par un entier appelé **indice**. La première valeur est associée à l'indice **0**, la seconde à l'indice 1, *etc.* Une liste a une longueur (un nombre d'éléments) finie ; la liste vide a pour longueur 0.

En Python, les éléments d'une liste ne sont pas forcément tous du même type, même si ce sera souvent le cas dans nos exemples.

Vous trouverez des informations complémentaires dans la documentation en ligne de Python.

5.1 Créer une liste

Pour créer une liste contenant des éléments donnés, on les écrit entre crochets, séparés par des virgules :

```
notes = [12, 15, 9]
prenoms = ["Aude", "Ali", "Chloé"]
melange = [1, "deux", 3.0, True]
vide = []
```

La fonction `len` (pour *length*, longueur) retourne le nombre d'éléments d'une liste :

```
print(len(notes))
print(len(vide))
```

```
3
0
```

Pour créer des listes longues (par exemple la liste des 1000 premiers entiers), on peut convertir un `range` en liste avec la fonction `list` :

```
print(list(range(10)))
print(list(range(0, 10, 2)))
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
[0, 2, 4, 6, 8]
```

Listes et itérables

`range(10)` n'est pas exactement une liste : c'est un **itérable**, c'est-à-dire un objet dont on peut parcourir les éléments un à un (par exemple avec une boucle `for`). Une grande partie de ce chapitre s'applique à tous les itérables, mais certaines opérations (comme l'accès au i-ème élément ou la modification d'un élément) sont propres aux listes. Si l'on a un itérable `iterable`, on peut toujours le transformer en liste avec `list(iterable)`.

5.2 Accéder aux éléments

5.2.1 Les indices

Pour accéder à l'élément d'indice `i` d'une liste, on utilise la syntaxe `liste[i]`.

```
liste = ["a", "b", "c", "d", "e"]
print(liste[0])    # le premier élément
print(liste[1])    # le deuxième élément
print(liste[4])    # le dernier élément
```

```
a
b
e
```

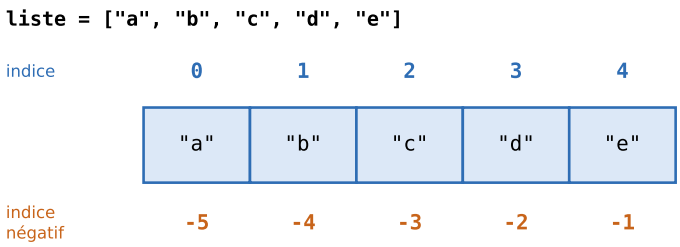


Figure5.1: Les indices d'une liste de 5 éléments : de 0 à 4 en partant du début, de -1 à -5 en partant de la fin.

On peut aussi utiliser des **indices négatifs** pour compter à partir de la fin : `liste[-1]` est le dernier élément, `liste[-2]` l'avant-dernier, *etc.*

```
print(liste[-1])
print(liste[-2])
```

```
e
d
```

Ainsi, pour une liste de taille n, les indices valides sont les entiers compris entre $-n$ et $n-1$ (inclus). Utiliser un autre indice provoque une erreur :

```
liste = ["a", "b", "c", "d", "e"]
print(liste[5])
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most␣
↪recent call last)
Cell In[6], line 2
      1 liste = ["a", "b", "c", "d", "e"]
----> 2 print(liste[5])

IndexError: list index out of range
```

À retenir

Les indices commencent à **0** : dans une liste de `n` éléments, le dernier élément a pour indice `n - 1` (ou `-1`), et non `n`. L'erreur `IndexError: list index out of range` signifie que l'on a utilisé un indice trop grand (ou trop petit).

5.2.2 Modifier un élément

L'accès aux éléments d'une liste peut se faire en lecture (lire l'élément stocké à l'indice `i`) comme en écriture (modifier l'élément stocké à l'indice `i`) :

```
liste = [1, 5, 7]
liste[1] = 2
print(liste)
```

```
[1, 2, 7]
```

5.2.3 Extraire une sous-liste (*slicing*)

On peut extraire une partie d'une liste avec la syntaxe `liste[debut:fin]`, où `debut` est l'indice du premier élément pris et `fin` l'indice du premier élément **non pris** (exclu, comme pour `range`).

Pour s'en souvenir, on peut imaginer que les indices désignent des « piquets » placés **entre** les éléments : `liste[2:5]` correspond à ce qui se trouve entre le piquet 2 et le piquet 5.

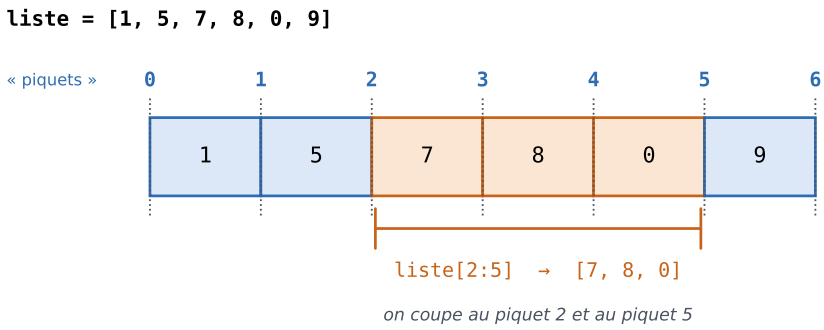


Figure5.2: `liste[2:5]` contient les éléments d'indices 2, 3 et 4 (soit 5 - 2 = 3 éléments).

```
liste = [1, 5, 7, 8, 0, 9]
print(liste[2:5])
```

```
[7, 8, 0]
```

Si l'on omet l'indice de début, la sélection commence au début de la liste ; si l'on omet l'indice de fin, elle s'étend jusqu'à la fin :

```
print(liste[:3])     # les 3 premiers éléments
print(liste[3:])     # tout sauf les 3 premiers
print(liste[-2:])    # les 2 derniers
```

```
[1, 5, 7]
[8, 0, 9]
[0, 9]
```

Testez-vous

Avec `l = [10, 20, 30, 40, 50, 60]`, que valent :

1. `l[2]`
2. `l[-1]`
3. `l[1:3]`
4. `l[:2]`
5. `l[4:]`
6. `len(l[1:4])`

5.3 Parcourir une liste

Lorsque l'on parcourt une liste, on peut vouloir accéder :

- aux éléments de la liste uniquement (c'est le cas le plus fréquent) ;
- aux indices de la liste uniquement ;
- aux indices et aux éléments associés.

Ces trois cas de figure correspondent à trois façons différentes d'écrire la boucle.

5.3.1 Parcours des éléments

Comme nous l'avons vu au chapitre 3, une boucle `for` parcourt directement les éléments :

```
liste = [1, 5, 7]
for elem in liste:
    print(elem)
```

```
1
5
7
```

5.3.2 Parcours par indices

Pour avoir accès aux indices, on utilise le fait que les indices d'une liste sont les entiers de 0 à `len(liste) - 1`, c'est-à-dire `range(len(liste))` :

```
liste = [1, 5, 7]
for i in range(len(liste)):
    print(i, liste[i])
```

```
0 1
1 5
2 7
```

5.3.3 Parcours par indices et éléments

Lorsque l'on a besoin à la fois des indices et des éléments, la fonction `enumerate` est la plus pratique :

```
liste = [1, 5, 7]
for i, elem in enumerate(liste):
    print(i, elem)
```

```
0 1
1 5
2 7
```

À chaque tour, deux variables sont mises à jour : `i` (l'indice courant) et `elem` (l'élément se trouvant à l'indice `i`).

Bonne pratique

Utilisez des noms de variables qui indiquent la nature de ce que vous manipulez : `i` ou `j` pour des indices, `elem`, `val`, ou mieux, un nom parlant (`note`, `prenom`...) pour des éléments. Écrire `for note in notes:` est bien plus clair que `for x in l:`.

Erreur fréquente

- Confondre indice et élément : dans `for elem in liste:`, `elem` est un élément de la liste, pas un indice. Écrire `liste[elem]` n'a alors généralement pas de sens.
- Modifier une liste (lui ajouter ou supprimer des éléments) **pendant** qu'on la parcourt avec une boucle `for` : c'est fortement déconseillé et donne des résultats inattendus. Construisez plutôt une nouvelle liste.

5.3.4 Exemple guidé : la moyenne d'une liste de notes

Calculons la moyenne des notes d'une liste, en réutilisant le principe de l'accumulateur :

```
def moyenne(notes):
    somme = 0
    for note in notes:
        somme += note
    return somme / len(notes)

print(moyenne([12, 15, 9, 14]))
```

```
12.5
```

Python propose d'ailleurs des fonctions prêtes à l'emploi pour les calculs les plus courants sur les listes de nombres : `sum`, `min` et `max`.

```
notes = [12, 15, 9, 14]
print(sum(notes) / len(notes))
print(min(notes), max(notes))
```

```
12.5
9 15
```

Il est néanmoins important de savoir écrire ces fonctions vous-même : c'est un excellent exercice, et les mêmes principes vous serviront pour des calculs que Python ne propose pas directement.

Exercice 5.1 : Argmax ☆ ☆

Écrivez une fonction qui calcule l'**argmax** d'une liste, c'est-à-dire l'indice auquel est stockée la valeur maximale de la liste (sans utiliser les fonctions `max` et `index`). Si cette valeur maximale est présente plusieurs fois dans la liste, on retournera l'indice de sa première occurrence. Si la liste est vide, on retournera `None`.

5.4 Manipuler une liste

5.4.1 Ajouter des éléments

Pour ajouter un élément dans une liste, on peut :

- l'ajouter **à la fin** avec la méthode `append` ;
- l'insérer à l'indice `i` avec la méthode `insert`.

Remarquez qu'il est ici question de **méthodes** et non plus de fonctions : les méthodes sont des fonctions propres à certains types de données (comme les listes), et elles s'appellent avec une syntaxe particulière, `variable.methode(arguments)` :

```
liste = [1, 5, 7]
liste.append(2)
print(liste)
liste.insert(2, 0)    # insère la valeur 0 à l'indice 2
print(liste)
```

```
[1, 5, 7, 2]
[1, 5, 0, 7, 2]
```

La méthode `append` est très souvent utilisée pour **construire une liste petit à petit**, en partant d'une liste vide :

```
carres = []
for i in range(6):
    carres.append(i ** 2)
print(carres)
```

```
[0, 1, 4, 9, 16, 25]
```

5.4.2 Supprimer des éléments

Pour supprimer un élément, on peut :

- supprimer l'élément situé à l'indice `i`, avec l'instruction `del` ;
- supprimer la **première occurrence** d'une valeur donnée, avec la méthode `remove`.

```
liste = [1, 5, 7]
del liste[1]    # supprime l'élément d'indice 1 (le deuxième !)
print(liste)

liste = [7, 1, 5, 1]
liste.remove(1)    # supprime le premier 1 rencontré
print(liste)
```

```
[1, 7]
[7, 5, 1]
```

5.4.3 Rechercher un élément

Pour savoir si une valeur est présente dans une liste, on utilise le mot-clé `in`, qui donne un booléen. Pour trouver l'indice de la première occurrence d'une valeur, on utilise la méthode `index` :

```
liste = [1, 5, 7]
print(5 in liste)
print(3 in liste)
print(liste.index(7))
```

```
True
False
2
```

Attention, `liste.index(valeur)` et `liste.remove(valeur)` provoquent une erreur (`ValueError`) si la valeur n'est pas dans la liste : pensez à tester avec `in` au préalable si vous n'en êtes pas sûr·e.

5.4.4 Concaténer et répéter

Comme pour les chaînes de caractères, l'opérateur + met deux listes bout à bout et l'opérateur * répète une liste :

```
print([1, 5, 7] + [3, 4])
print(3 * [1, 5])
print([0] * 10)
```

```
[1, 5, 7, 3, 4]
[1, 5, 1, 5, 1, 5]
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

5.4.5 Trier une liste

La fonction `sorted` retourne une **nouvelle** liste contenant les éléments triés (la liste de départ n'est pas modifiée) :

```
liste = [4, 5, 2]
liste_triee = sorted(liste)
print(liste_triee)
print(liste)
print(sorted(liste, reverse=True))    # ordre décroissant
```

```
[2, 4, 5]
[4, 5, 2]
[5, 4, 2]
```

On peut trier une liste dès lors que ses éléments sont comparables entre eux (des nombres entre eux, des chaînes de caractères entre elles...) :

```
print(sorted(["poire", "abricot", "kiwi"]))
```

```
['abricot', 'kiwi', 'poire']
```

Sachant que les émoticônes sont des caractères comme les autres, on peut ainsi (enfin) obtenir une réponse à un problème vieux comme le monde :

```
print(sorted(["🐔", "🎻"]))
```

```
['🎻', '🐔']
```

```{admonition} Pour info : `sorted` ou `.sort()` ? :class: seealso

Il existe aussi une méthode `liste.sort()` qui trie la liste **sur place** (elle modifie la liste et ne retourne rien). Attention donc à ne pas écrire `liste = liste.sort()` : `liste` vaudrait alors `None` !

```{admonition} Testez-vous
:class: hint

Que contient `l` à la fin de ce programme ?

    l = [3, 1, 4]
    l.append(1)
    l.insert(0, 5)
    l.remove(1)
    del l[-1]
```
```

5.4.6 Exercices

Exercice 5.2 : Notes au-dessus de la moyenne ☆

Écrivez une fonction `au_dessus(notes, seuil)` qui retourne la liste des notes de `notes` strictement supérieures à `seuil`.

Exercice 5.3 : Inverser une liste ☆ ☆

Écrivez une fonction `inverse(liste)` qui retourne une nouvelle liste contenant les éléments de `liste` dans l'ordre inverse (sans utiliser les outils de Python dédiés comme `reversed` ou `liste[::-1]`).

Exercice 5.4 : Intersection de listes ☆ ☆

Écrivez une fonction qui prend deux listes en entrée et retourne leur intersection, c'est-à-dire une liste contenant (une seule fois chacun) tous les éléments présents dans les deux listes.

Exercice 5.5 : Union de listes ☆ ☆

Écrivez une fonction qui prend deux listes en entrée et retourne leur union **sans doublon**, c'est-à-dire une liste contenant une seule fois chaque élément présent dans au moins une des deux listes.

5.5 Copier une liste

Pour la plupart des variables, en Python, la copie ne pose pas de problème :

```
a = 12
b = a
a = 5
print(a, b)
```

```
5 12
```

Cela ne se passe pas de la même façon pour les listes. Observez :

```
liste = [1, 5, 7]
liste2 = liste
liste[1] = 2
print(liste, liste2)
```

```
[1, 2, 7] [1, 2, 7]
```

Surprise : `liste2` a été modifiée alors que l'on n'y a pas touché ! En effet, l'instruction `liste2 = liste` ne recopie pas le contenu de `liste` : elle crée un **deuxième nom** pour **la**

même liste en mémoire. Toute modification faite via l'un des noms est donc visible via l'autre.

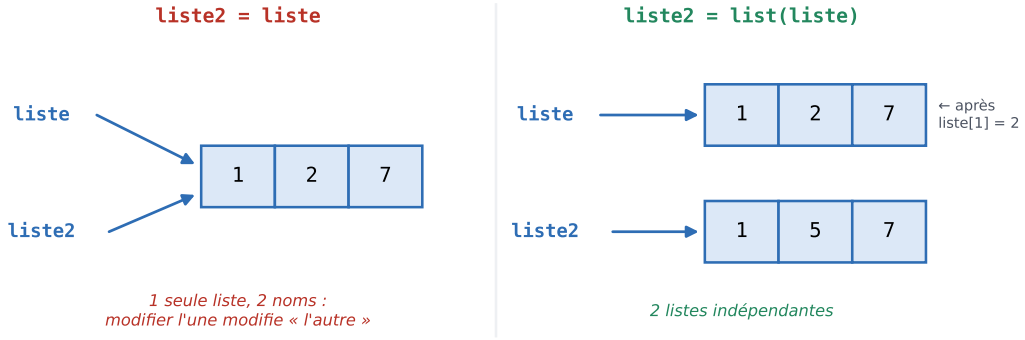


Figure5.3: À gauche, `liste` et `liste2` désignent la même liste. À droite, `list(liste)` a créé une copie indépendante.

Voir ce code s'exécuter pas à pas dans Python Tutor

Pour obtenir une copie indépendante, il faut la demander explicitement, par exemple avec la fonction `list` :

```
liste = [1, 5, 7]
liste2 = list(liste)    # ou liste.copy(), ou liste[:]
liste[1] = 2
print(liste, liste2)
```

```
[1, 2, 7] [1, 5, 7]
```

À retenir

Pour les listes (et plus généralement pour les objets modifiables, comme les dictionnaires que nous verrons bientôt), `b = a` ne copie pas : `a` et `b` désignent alors le même objet. Pour copier une liste, on utilise `list(a)` ou `a.copy()`.

Ce comportement s'applique aussi aux fonctions : si une fonction modifie une liste reçue en argument (avec `append`, `del`...), la modification est visible à l'extérieur de la fonction.

Testez-vous

Qu'affiche ce programme ?

```
def ajoute_zero(l):
    l.append(0)

ma_liste = [1, 2]
ajoute_zero(ma_liste)
print(ma_liste)
```

5.6 Bonus : les listes en compréhension

Python propose une syntaxe compacte pour créer une liste à partir d'une autre liste (ou d'un itérable), en transformant et/ou en filtrant ses éléments. Supposons par exemple que l'on souhaite créer la liste des carrés des 10 premiers entiers. Voici deux façons équivalentes de le faire :

```
# Façon "classique"
liste = []
for i in range(10):
    liste.append(i ** 2)
print(liste)
```

```
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

```
# Avec une liste en compréhension
liste = [i ** 2 for i in range(10)]
print(liste)
```

```
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

La syntaxe se lit presque comme une phrase : « la liste des `i ** 2` pour `i` parcourant `range(10)` ». On peut aussi ne garder que certains éléments, avec un `if` :

```
liste = [i ** 2 for i in range(10) if i % 2 == 0]
print(liste)
```

```
[0, 4, 16, 36, 64]
```

Ici, on n'a considéré que les entiers pairs. Ainsi, l'exercice 5.2 peut se résoudre en une ligne : `[note for note in notes if note > seuil]`.

5.7 Super bonus : parcourir plusieurs listes en même temps

Il peut arriver que l'on ait à parcourir simultanément plusieurs listes. Supposons par exemple que l'on ait une liste des prénoms des enfants d'une classe et une autre liste contenant leurs noms. On peut effectuer un parcours par indices :

```
prenoms = ["Jeanne", "Anne", "Camille"]
noms = ["Papin", "Bakayoko", "Drogba"]

for i in range(len(prenoms)):
    print(prenoms[i], noms[i])
```

```
Jeanne Papin
Anne Bakayoko
Camille Drogba
```

Mais on n'a pas réellement besoin des indices ici : on veut juste parcourir les deux listes en parallèle. C'est ce que permet la fonction `zip` :

```
for p, n in zip(prenoms, noms):
    print(p, n)
```

```
Jeanne Papin
Anne Bakayoko
Camille Drogba
```

On peut même parcourir plus de deux listes simultanément :

```
ages = [12, 11, 12]
for p, n, a in zip(prenoms, noms, ages):
    print(p, n, a)
```

```
Jeanne Papin 12
Anne Bakayoko 11
Camille Drogba 12
```

Pour que cela ait du sens, les listes doivent bien sûr être de même taille.

Si vous venez de R ou de SAS

Trois différences avec les vecteurs de R sont sources d'erreurs fréquentes :

- **Les indices commencent à 0** : le premier élément est `l[0]` (`l[1]` en R) et le dernier `l[-1]` (`l[length(l)]` en R).
- **Les indices négatifs n'ont pas le même sens** : en R, `l[-1]` **retire** le premier élément ; en Python, `l[-1]` **désigne** le dernier.
- **La borne de fin d'une tranche est exclue** : `l[1:3]` contient les 2e et 3e éléments (indices 1 et 2), c'est-à-dire l'équivalent de `l[2:3]` en R. Retenez que le début est décalé de 1 et que la fin est exclue.

Par ailleurs, une liste Python n'est pas un vecteur : `[1, 2] * 2` vaut `[1, 2, 1, 2]` et `[1, 2] + [3, 4]` vaut `[1, 2, 3, 4]`. Les calculs élément par élément nécessitent `numpy` ou `pandas`.

Enfin, R copie les objets lorsqu'on les modifie (`b <- a` puis modifier `b` ne change pas `a`), alors qu'en Python `b = a` crée un **second nom pour la même liste** (section « Copier une liste »).

5.8 Exercices de synthèse

Exercice 5.6 : Compter les occurrences ☆

Écrivez une fonction `compte(liste, valeur)` qui retourne le nombre de fois où `valeur` apparaît dans `liste` (sans utiliser la méthode `count` des listes).

Exercice 5.7 : Liste triée ? ☆ ☆

Écrivez une fonction `est_triee(liste)` qui retourne `True` si la liste est triée par ordre croissant et `False` sinon (sans utiliser `sorted`).

Exercice 5.8 : Moyenne pondérée ☆ ☆

Écrivez une fonction `moyenne_ponderee(notes, coefficients)` qui retourne la moyenne des notes pondérée par les coefficients (les deux listes ont la même longueur). Pour les notes `[12, 15, 8]` avec les coefficients `[2, 1, 3]`, on doit obtenir $(12 \times 2 + 15 \times 1 + 8 \times 3)/(2 + 1 + 3) = 10{,}5$.

Exercice 5.9 : Retirer les doublons en gardant l'ordre ☆ ☆ ☆

Écrivez une fonction `sans_doublons(liste)` qui retourne une nouvelle liste contenant les éléments de `liste` sans répétition, **dans l'ordre de leur première apparition**. Écrivez-en ensuite une version `retire_doublons(liste)` qui **modifie** la liste reçue en argument (et ne retourne rien).

5.9 Bilan du chapitre

À retenir

- Création : `[1, 5, 7]`, `[]`, `list(range(n))` ; taille : `len(liste)`.
- Indices de `0` à `len(liste) - 1` ; indices négatifs depuis la fin (`liste[-1]` est le dernier élément).
- Sous-liste : `liste[debut:fin]` (fin exclue).
- Parcours : `for elem in liste`, `for i in range(len(liste))`, `for i, elem in enumerate(liste)`.
- Méthodes : `append`, `insert`, `remove`, `index` ; instruction `del liste[i]` ; test `valeur in liste`.
- `sorted(liste)` retourne une copie triée ; `sum`, `min`, `max` sur des listes de nombres.
- `liste2 = liste` ne copie pas ! Utiliser `list(liste)`.

Faites le point : êtes-vous capable de…

- accéder au premier, au dernier, au i-ème élément d'une liste ?
- extraire une sous-liste avec la notation `[debut:fin]` ?
- choisir le bon type de parcours (éléments, indices, `enumerate`) ?
- construire une nouvelle liste avec `append` dans une boucle ?

- calculer une somme, une moyenne, un maximum, un comptage sur une liste ?
- expliquer ce qui se passe quand on écrit `liste2 = liste` ?

CHAPTER 6

LES CHAÎNES DE CARACTÈRES

Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- manipuler une chaîne de caractères comme une liste de caractères (indices, sous-chaînes, parcours) ;
- convertir des chaînes en nombres et inversement ;
- utiliser les principales méthodes des chaînes (`split`, `lower`, `replace`, `strip`...) ;
- mettre en forme des affichages avec `format` et les f-strings.

Prérequis : chapitres 1 à 5. **Durée indicative :** 1h30.

Nous nous intéressons maintenant plus en détail à un type de données que nous utilisons depuis le début : les chaînes de caractères (type `str`). Pour créer une chaîne de caractères, il suffit d'utiliser des guillemets, simples ou doubles (les deux sont équivalents) :

```
s1 = "abc"
s2 = 'bde'
s3 = "L'apostrophe passe sans problème entre guillemets doubles"
print(s3)
```

```
L'apostrophe passe sans problème entre guillemets doubles
```

Comme pour les listes, il est conseillé de se reporter à la documentation en ligne lorsque vous avez des doutes sur la manipulation des chaînes de caractères.

6.1 Conversion d'une chaîne en nombre

Si une chaîne de caractères représente une valeur numérique (comme la chaîne `"10.2"`), on peut la transformer en entier ou en nombre à virgule pour l'utiliser ensuite dans des calculs, avec les fonctions de conversion `int` et `float`. C'est indispensable lorsque les données proviennent d'un fichier ou d'une saisie de l'utilisateur, qui sont toujours lues sous forme de texte.

```
s = "10.2"
f = float(s)
print(f + 2)
print(f == s)    # un nombre n'est jamais égal à une chaîne
```

```
12.2
False
```

```
s = "10"
i = int(s)
print(i - 1)
print(s * 2)    # attention : sur une chaîne, * répète !
```

```
9
1010
```

6.2 Analogie avec les listes

Une chaîne de caractères se manipule en partie comme une liste de caractères. On peut ainsi obtenir sa longueur avec `len`, ou accéder à son i-ème caractère avec la notation `s[i]`, y compris avec des indices négatifs :

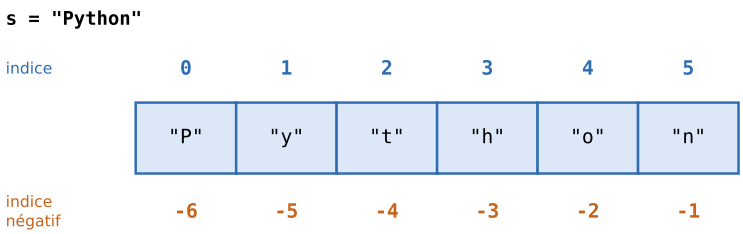


Figure6.1: Les indices des caractères de la chaîne `"Python"`.

```
s = "Python"
print(len(s))
print(s[0])
print(s[-1])
```

```
6
P
n
```

De même, on peut extraire des sous-chaînes à partir des indices de début et de fin (exclu) :

```
print(s[2:4])
print(s[:2])
```

```
th
Py
```

On peut aussi parcourir une chaîne caractère par caractère avec une boucle `for` :

```
for lettre in "abc":
    print(lettre)
```

```
a
b
c
```

Comme pour les listes, on peut concaténer deux chaînes avec l'opérateur + et répéter une chaîne avec l'opérateur * :

```
s = "ab" + ("cde" * 3)
print(s)
```

```
abcdecdecde
```

Enfin, le mot-clé `in` permet de tester la présence d'une sous-chaîne dans une chaîne :

```
s = "abcde"
print("a" in s)
print("bcd" in s)
print("bCd" in s)
```

```
True
True
False
```

À retenir : les chaînes ne sont pas modifiables

L'analogie entre listes et chaînes de caractères n'est pas parfaite. En particulier, on peut **lire** le i-ème caractère d'une chaîne, mais **pas le modifier** : si `s` est une chaîne de caractères, l'instruction `s[2] = "c"` provoque une erreur. On dit que les chaînes sont **non modifiables** (ou *immuables*). Pour « modifier » une chaîne, on en construit une nouvelle, que l'on peut ranger dans la même variable :

```
s = "maison"
s[0] = "r"
```

```
---------------------------------------------------------------------
 ↪-----------
TypeError                                 Traceback (most␣
 ↪recent call last)
Cell In[9], line 2
```

(suite sur la page suivante)

(suite de la page précédente)

```
      1 s = "maison"
----> 2 s[0] = "r"

TypeError: 'str' object does not support item assignment
```

```
s = "maison"
s = "r" + s[1:]    # on construit une nouvelle chaîne
print(s)
```

```
raison
```

Testez-vous

Avec `mot = "bonjour"`, que valent : `mot[3]`, `mot[-2]`, `mot[1:4]`, `len(mot)`, `mot[0] + mot[-1]` et `"jour" in mot` ?

6.2.1 Construire une chaîne petit à petit

De la même façon que l'on construit une liste avec `append`, on peut construire une chaîne en partant d'une chaîne vide `""` et en lui ajoutant des morceaux au fil d'une boucle :

```
mot = "python"
resultat = ""
for lettre in mot:
    resultat = resultat + lettre + "-"
print(resultat)
```

```
p-y-t-h-o-n-
```

6.3 Principales méthodes des chaînes

Les chaînes disposent de nombreuses **méthodes** (des fonctions propres au type `str`, appelées avec la syntaxe `chaine.methode(...)`). Comme les chaînes ne sont pas modifiables, ces méthodes ne modifient jamais la chaîne d'origine : elles **retournent une nouvelle valeur**.

Méthode	Description
`ch.lower()`	retourne la version en minuscules de `ch`
`ch.upper()`	retourne la version en majuscules de `ch`
`ch.strip()`	retourne `ch` sans les espaces (et retours à la ligne) au début et à la fin
`ch.replace(old, new)`	retourne une copie de `ch` où toutes les occurrences de `old` sont remplacées par `new`
`ch.split(sep)`	retourne la **liste** des morceaux de `ch` découpée à chaque occurrence de `sep` (par défaut, à chaque espace ou retour à la ligne)
`sep.join(liste)`	l'inverse de `split` : retourne la chaîne obtenue en collant les éléments de `liste` (des chaînes) séparés par `sep`
`ch.count(sub)`	retourne le nombre d'occurrences de `sub` dans `ch`
`ch.find(sub)`	retourne l'indice de la première occurrence de `sub` dans `ch` (ou -1 si `sub` n'est pas dans `ch`)
`ch.rfind(sub)`	retourne l'indice de la dernière occurrence de `sub` dans `ch`
`ch.startswith(pre`	retourne `True` si `ch` commence par `prefix`
`ch.endswith(suffix`	retourne `True` si `ch` se termine par `suffix`
`ch.islower()`, `ch.isupper()`	retourne `True` si les lettres de `ch` sont toutes en minuscules (resp. majuscules)
`ch.isnumeric()`	retourne `True` si `ch` n'est constituée que de chiffres
`ch.format(...)`	remplace les `{}` de `ch` par les valeurs passées en argument (voir plus bas)

Voici quelques exemples :

```
ch = "  La vie est belle  "
print(ch.strip())
print(ch.upper())
print(ch.replace("belle", "longue"))
print(ch.count("e"))
```

```
La vie est belle
  LA VIE EST BELLE
  La vie est longue
4
```

La méthode `split` est particulièrement utile pour découper un texte en mots, ou une ligne de données en champs :

```
phrase = "la vie est belle"
mots = phrase.split()
print(mots)
print(len(mots), "mots")

ligne = "Lemarchand;John;23"
print(ligne.split(";"))
```

```
['la', 'vie', 'est', 'belle']
4 mots
['Lemarchand', 'John', '23']
```

et `join` fait l'opération inverse :

```
print("-".join(["2024", "09", "01"]))
print(" ".join(mots).upper())
```

```
2024-09-01
LA VIE EST BELLE
```

Erreur fréquente

Les méthodes des chaînes **retournent** un résultat : elles ne modifient pas la chaîne. Le code suivant n'a donc aucun effet visible :

```
nom = "aude"
nom.upper()
print(nom)    # affiche toujours "aude"
```

Il faut récupérer le résultat : `nom = nom.upper()`.

Testez-vous

Que valent les expressions suivantes ?

1. `"Bonjour".lower().count("o")`
2. `"a,b,,c".split(",")`
3. `"Rennes".replace("n", "N")`
4. `" x ".strip() + "!"`
5. `"".join(["a", "b", "c"])`

6.4 Mettre en forme des chaînes de caractères

Nous avons vu au chapitre 1 deux façons d'insérer des valeurs de variables dans une chaîne : la méthode `format` et les f-strings. Revenons-y plus en détail.

6.4.1 La méthode `format`

La méthode `format` recherche dans la chaîne les accolades `{}` et les remplace, dans l'ordre, par les valeurs fournies :

```
age = 12
prenom = "Micheline"
s = "{} a {} ans".format(prenom, age)
print(s)
```

```
Micheline a 12 ans
```

On peut aussi nommer les emplacements, ce qui est utile pour les chaînes longues contenant de nombreuses valeurs, et préciser le format d'affichage après un `:` (ici, 3 chiffres après la virgule) :

```
s = "{prenom} a {age:.3f} ans".format(prenom="Micheline", age=12)
print(s)
```

```
Micheline a 12.000 ans
```

6.4.2 Les f-strings

Les f-strings (pour *formatted strings*) sont une écriture plus concise du même principe : on place un `f` avant la chaîne et on écrit directement les variables, ou même des calculs, entre accolades :

```
age_chat = 12
s = f"Ce chat a {age_chat} ans, ce qui lui fait {age_chat * 6}␣
 ↪ans en âge équivalent humain"
print(s)
```

```
Ce chat a 12 ans, ce qui lui fait 72 ans en âge équivalent␣
 ↪humain
```

6.4.3 Les options de mise en forme

Que ce soit avec `format` ou avec une f-string, on peut ajouter après le nom de la variable un `:` suivi d'options de mise en forme, par exemple pour fixer le nombre de décimales ou aligner des valeurs en colonnes :

Option	Effet	Exemple	Résultat
`:.2f`	2 décimales	`f"{3.14159:.2f}"`	`3.14`
`:8.2f`	2 décimales, largeur 8	`f"{3.14159:8.2f}"`	`  3.14`
`:5d`	entier, largeur 5	`f"{42:5d}"`	`   42`
`:<6`	aligné à gauche, largeur 6	`f"{'ab':<6}"`	`ab    `
`:>6`	aligné à droite, largeur 6	`f"{'ab':>6}"`	`    ab`
`:^6`	centré, largeur 6	`f"{'ab':^6}"`	`  ab  `

Vous trouverez une présentation exhaustive de ces options dans la documentation Python.

6.5 Exercices

Exercice 6.1 : Compter les voyelles ☆

Écrivez une fonction `nb_voyelles(s)` qui retourne le nombre de voyelles (a, e, i, o, u, y, en minuscules ou majuscules) contenues dans la chaîne `s`.

Exercice 6.2 : Mots commençant par un préfixe ☆

Écrivez une fonction qui prend en argument deux chaînes de caractères `s` et `prefix` et retourne le nombre de mots de la chaîne `s` qui débutent par la chaîne `prefix`.

Exercice 6.3 : Compter un mot sans tenir compte de la casse ☆ ☆

Écrivez une fonction qui prend en argument deux chaînes de caractères `s` et `mot_cible` et retourne le nombre d'occurrences du mot `mot_cible` dans la chaîne `s`, sans tenir compte de la casse (majuscules / minuscules). Attention : on compte des **mots** entiers, `"la"` ne doit pas être compté dans `"lac"`.

Exercice 6.4 : Acronyme ☆ ☆

Écrivez une fonction `acronyme(s)` qui retourne l'acronyme d'une expression, formé des premières lettres de chaque mot, en majuscules.

Exercice 6.5 : Palindrome ☆ ☆

Un palindrome est un mot qui se lit de la même façon dans les deux sens, comme « radar » ou « kayak ». Écrivez une fonction `est_palindrome(mot)` qui retourne `True` si `mot` est un palindrome. En bonus, faites en sorte que la fonction ignore les espaces et la casse, pour reconnaître des phrases comme « Esope reste ici et se repose ».

6.6 Bilan du chapitre

À retenir

- Une chaîne se manipule comme une liste de caractères **non modifiable** : `len(s)`, `s[i]`, `s[debut:fin]`, `for lettre in s`, `sous_chaine in s`.

- `int(s)`, `float(s)` convertissent une chaîne en nombre ; `str(x)` convertit en chaîne.
- Méthodes utiles : `lower`, `upper`, `strip`, `replace`, `split`, `join`, `count`, `find`, `startswith`, `endswith`. Elles **retournent** une nouvelle valeur.
- Mise en forme : `f"{x:.2f}"` (2 décimales).

Faites le point : êtes-vous capable de…

- extraire un caractère ou une sous-chaîne d'une chaîne ?
- parcourir une chaîne lettre par lettre, ou mot par mot avec `split` ?
- construire une nouvelle chaîne au fil d'une boucle ?
- expliquer pourquoi `s.upper()` seul ne modifie pas `s` ?
- afficher un nombre avec un nombre fixé de décimales ?

CHAPTER 7

DICTIONNAIRES, *TUPLES* ET *SETS*

Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- créer un dictionnaire et y associer des valeurs à des clés ;
- lire, ajouter, modifier et supprimer des entrées ;
- parcourir un dictionnaire par clés, par valeurs ou par couples clé/valeur ;
- utiliser un dictionnaire pour compter des occurrences ;
- utiliser des *tuples* (et les « déballer » dans plusieurs variables) et des *sets* ;
- choisir entre une liste, un dictionnaire, un *tuple* et un *set*.

Prérequis : chapitres 1 à 6. **Durée indicative :** 2h.

Comme une liste, un dictionnaire est une collection de données. Mais au lieu de repérer chaque valeur par une position (un indice entier), on la repère par une **clé**, que l'on choisit librement. Chaque entrée d'un dictionnaire est donc une association entre une **clé** et une **valeur**, comme dans un dictionnaire papier où l'on associe une définition à chaque mot.

On utilise souvent des chaînes de caractères comme clés : par exemple, pour représenter le stock d'un primeur, on associe à chaque nom de fruit le nombre de fruits disponibles.

Les clés sont des valeurs quelconques (non modifiables : des chaînes, des nombres…), la seule contrainte étant qu'**une même clé ne peut apparaître qu'une fois** dans un dictionnaire.

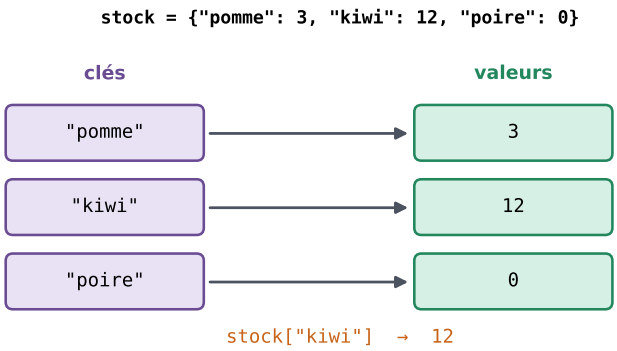


Figure7.1: Un dictionnaire associe des **valeurs** à des **clés**. On accède à une valeur grâce à sa clé.

7.1 Créer un dictionnaire

Pour définir un dictionnaire, on écrit ses paires `clé : valeur` entre accolades, séparées par des virgules :

```
stock = {"pomme": 3, "kiwi": 12, "poire": 0}
print(stock)
```

```
{'pomme': 3, 'kiwi': 12, 'poire': 0}
```

Un dictionnaire vide se crée avec des accolades vides :

```
mon_dico = {}
print(mon_dico)
print(len(stock))    # nombre de paires clé/valeur
```

```
{}
3
```

7.2 Lire le contenu d'un dictionnaire

Pour lire la valeur associée à une clé, on utilise la même syntaxe que pour les listes, en remplaçant l'indice par la clé :

```
stock = {"pomme": 3, "kiwi": 12, "poire": 0}
print(stock["kiwi"])
```

```
12
```

Si la clé demandée n'existe pas, on obtient une erreur `KeyError` :

```
print(stock["banane"])
```

```
---------------------------------------------------------------------
↪-----------
KeyError                                  Traceback (most␣
```

(suite sur la page suivante)

(suite de la page précédente)

```
 ↪recent call last)
Cell In[4], line 1
----> 1 print(stock["banane"])

KeyError: 'banane'
```

Pour éviter cela, on peut d'abord tester si la clé est présente avec le mot-clé `in` (qui teste la présence d'une **clé**, et non d'une valeur) :

```
if "banane" in stock:
    print(stock["banane"])
else:
    print("Pas de bananes")
```

```
Pas de bananes
```

ou bien utiliser la méthode `get`, qui permet de préciser une valeur par défaut à retourner si la clé n'existe pas :

```
print(stock.get("kiwi", 0))
print(stock.get("banane", 0))
```

```
12
0
```

7.3 Modifier le contenu d'un dictionnaire

Pour modifier la valeur associée à une clé, on utilise la même syntaxe que pour les listes :

```
stock = {"pomme": 3, "kiwi": 12, "poire": 0}
stock["pomme"] = 10
print(stock)
```

```
{'pomme': 10, 'kiwi': 12, 'poire': 0}
```

Et si la clé n'existe pas encore, la même syntaxe **crée** une nouvelle paire clé/valeur (contrairement aux listes, pour lesquelles un indice trop grand provoque une erreur) :

```
stock["banane"] = 6
print(stock)
```

```
{'pomme': 10, 'kiwi': 12, 'poire': 0, 'banane': 6}
```

Enfin, pour supprimer une paire clé/valeur, on utilise le mot-clé `del` :

```
del stock["poire"]
print(stock)
```

```
{'pomme': 10, 'kiwi': 12, 'banane': 6}
```

Testez-vous

Que contient `d` à la fin de ce programme ? Qu'affiche-t-il ?

```
d = {"a": 1, "b": 2}
d["c"] = 3
d["a"] = d["a"] + 10
del d["b"]
print(len(d), d.get("b", "absent"), "c" in d, 3 in d)
```

7.4 Parcourir un dictionnaire

Comme pour les listes, il existe trois façons de parcourir un dictionnaire.

7.4.1 Parcours par clés

Pour parcourir les clés, on utilise la méthode `keys` (ou, plus simplement, on parcourt directement le dictionnaire, ce qui revient au même) :

```
stock = {"pomme": 3, "kiwi": 12, "poire": 0}
for fruit in stock.keys():
    print(fruit)
```

```
pomme
kiwi
poire
```

7.4.2 Parcours par valeurs

Pour parcourir uniquement les valeurs, on utilise la méthode `values` :

```
for quantite in stock.values():
    print(quantite)
```

```
3
12
0
```

7.4.3 Parcours par couples clé/valeur

Pour accéder simultanément aux clés et aux valeurs associées, on utilise la méthode `items` :

```
for fruit, quantite in stock.items():
    print(f"{fruit} : {quantite}")
```

```
pomme : 3
kiwi : 12
poire : 0
```

À propos de l'ordre

Depuis Python 3.7, les dictionnaires conservent l'ordre dans lequel les clés ont été insérées, et c'est dans cet ordre qu'ils sont parcourus. Mais ils ne sont pas pour autant « ordonnés » au sens des listes : il n'y a pas de « premier élément » accessible par `d[0]`. Pour que votre code soit clair, évitez de faire reposer sa logique sur l'ordre des clés d'un dictionnaire.

7.5 Exemple guidé : compter des occurrences

Une utilisation très fréquente des dictionnaires est le **comptage** : combien de fois chaque mot apparaît-il dans un texte ? combien d'étudiants ont obtenu chaque note ? Comptons les votes d'un sondage :

```
votes = ["pizza", "sushi", "pizza", "crêpes", "pizza", "crêpes"]

compteurs = {}
for vote in votes:
    if vote in compteurs:
        compteurs[vote] += 1      # on a déjà vu ce vote : on␣
 ↪incrémente
    else:
        compteurs[vote] = 1       # première fois : on crée l
 ↪'entrée
print(compteurs)
```

```
{'pizza': 3, 'sushi': 1, 'crêpes': 2}
```

Le schéma est toujours le même : si la clé existe déjà, on met à jour sa valeur ; sinon, on la crée. Avec la méthode `get`, on peut même écrire les deux cas en une seule ligne : `compteurs[vote] = compteurs.get(vote, 0) + 1`.

Voir ce code s'exécuter pas à pas dans Python Tutor

7.6 Des valeurs plus riches

Les valeurs d'un dictionnaire peuvent être de n'importe quel type, y compris des listes ou d'autres dictionnaires. Cela permet de représenter des données structurées :

```
notes = {
    "Aude": [12, 15, 17],
    "Ali": [9, 14],
    "Chloé": [16, 13, 18, 11]
}
for prenom, liste_notes in notes.items():
    moyenne = sum(liste_notes) / len(liste_notes)
    print(f"{prenom} : moyenne de {moyenne:.2f} sur {len(liste_
 ↪notes)} notes")
```

```
Aude : moyenne de 14.67 sur 3 notes
Ali : moyenne de 11.50 sur 2 notes
Chloé : moyenne de 14.50 sur 4 notes
```

```
etudiant = {
    "nom": "Martin",
    "prenom": "Aude",
    "date_de_naissance": {"jour": 1, "mois": 6, "annee": 2005}
}
print(etudiant["date_de_naissance"]["annee"])
```

```
2005
```

Nous retrouverons ce type de structure au chapitre 9, avec les fichiers JSON.

Liste ou dictionnaire ?

Lorsque l'on doit stocker une série de valeurs, on a donc le choix entre une liste et un dictionnaire.

On utilise un **dictionnaire** dès lors que l'on veut mettre en évidence une **correspondance entre des clés et des valeurs**, comme dans l'exemple suivant, dans lequel on associe à chaque numéro étudiant un entier (pouvant représenter une note, un nombre de photocopies restantes autorisées, ou autre) :

```
{"2170000": 13, "2180000": 23}
```

On utilise une **liste** pour stocker une **série ordonnée de valeurs**, comme une liste de prénoms :

```
["Chloé", "Ali", "Margot", "Antoine"]
```

Si vous venez de R ou de SAS

- Un dictionnaire ressemble à une **liste nommée** de R (`list(pomme = 3, kiwi = 12)`, avec un accès `stock[["kiwi"]]`), ou à un format SAS (`PROC FORMAT`) qui associe des libellés à des codes.
- Avec `pandas`, les dictionnaires servent souvent à créer des tableaux (`pd.DataFrame({...})`, chapitre 11) et à recoder des valeurs (méthode `map`, chapitre 12).

7.7 Deux autres collections : les *tuples* et les *sets*

Pour terminer ce chapitre, voyons rapidement deux autres types de collections, que vous croiserez régulièrement (notamment avec `pandas`).

7.7.1 Les *tuples*

Un *tuple* s'écrit comme une liste, mais avec des parenthèses (facultatives) au lieu des crochets. Il ressemble en de nombreux points à une liste (accès par indice, `len`, boucle `for`...), à une différence majeure près : **un *tuple* ne peut pas être modifié** une fois créé.

```
point = (3, 7)
print(point[0], len(point))
```

```
3 2
```

```
point[0] = 5
```

```
---------------------------------------------------------------------
 ↪-----------
TypeError                                 Traceback (most␣
 ↪recent call last)
Cell In[17], line 1
----> 1 point[0] = 5

TypeError: 'tuple' object does not support item assignment
```

Vous en avez déjà rencontré sans le savoir : quand une fonction retourne plusieurs valeurs séparées par des virgules (`return quotient, reste`), elle retourne en fait un *tuple*. On peut répartir les valeurs d'un *tuple* dans plusieurs variables en une seule instruction (on parle de **déballage**) :

```
x, y = point
print(x, y)
```

```
3 7
```

C'est ce qui se passe dans `for cle, valeur in d.items()` : à chaque tour, `items` fournit un *tuple* `(clé, valeur)`, déballé dans deux variables. Le déballage permet aussi d'échanger deux variables très simplement :

```
a, b = 5, 8
a, b = b, a
print(a, b)
```

```
8 5
```

Avec `pandas`, vous verrez par exemple que la taille d'un tableau de données est un *tuple* `(nombre de lignes, nombre de colonnes)`.

7.7.2 Les *sets*

Un *set* (ou **ensemble**) est une collection **sans ordre et sans doublon**, notée avec des accolades comme les dictionnaires, mais sans valeurs associées. Son utilisation la plus courante consiste à **éliminer les doublons** d'une liste, ou à connaître les **valeurs distinctes** d'une série de données :

```
villes = ["Rennes", "Brest", "Rennes", "Vannes", "Brest", "Rennes
 ↪"]
distinctes = set(villes)
print(distinctes)
print(len(distinctes), "villes différentes")
```

```
{'Rennes', 'Brest', 'Vannes'}
3 villes différentes
```

On peut ajouter un élément avec `add`, tester l'appartenance avec `in`, et calculer intersections (`&`), unions (`|`) et différences (`-`) :

```
a = {1, 2, 3, 4}
b = {3, 4, 5}
print(a & b, a | b, a - b)
```

```
{3, 4} {1, 2, 3, 4, 5} {1, 2}
```

Attention : `{}` crée un dictionnaire vide ; un *set* vide s'écrit `set()`.

7.7.3 Récapitulatif

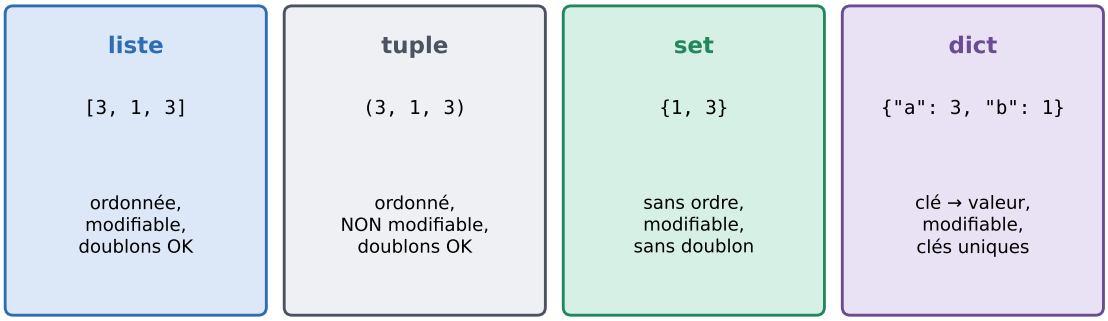


Figure7.2: Les quatre types de collections de base en Python.

Testez-vous

Quelle structure utiliseriez-vous pour stocker :

1. les températures relevées chaque heure pendant une journée ?
2. la population de chaque commune de Bretagne ?
3. la position (latitude, longitude) d'une ville ?

4. la liste des espèces d'oiseaux observées au moins une fois (sans répétition) ?

7.8 Exercices

Exercice 7.1 : Somme des valeurs ☆

Écrivez une fonction qui retourne la somme des **valeurs** d'un dictionnaire fourni en argument, en supposant que toutes ces valeurs soient numériques.

Exercice 7.2 : Occurrences des mots ☆ ☆

Écrivez une fonction qui compte le nombre d'occurrences de chacun des mots d'une chaîne de caractères et retourne le résultat sous forme de dictionnaire :

```
print(compte_occurrences("la vie est belle c'est la vie"))
# Doit afficher : {'la': 2, 'vie': 2, 'est': 1, 'belle': 1, "c
 ↪'est": 1}
```

Exercice 7.3 : Le mot le plus fréquent ☆ ☆

Écrivez une fonction `plus_frequent(d)` qui prend en argument un dictionnaire de comptages (comme celui retourné par la fonction de l'exercice précédent) et retourne la clé associée à la plus grande valeur.

Exercice 7.4 : Annuaire inversé ☆ ☆

On dispose d'un annuaire sous forme de dictionnaire associant à chaque nom un numéro de téléphone. Écrivez une fonction `inverse_annuaire(annuaire)` qui retourne un nouveau dictionnaire associant à chaque numéro le nom correspondant.

Exercice 7.5 : Bulletin de notes ☆ ☆ ☆

On représente les notes d'une classe par un dictionnaire associant à chaque prénom la liste de ses notes (comme dans la section « Des valeurs plus riches »).

1. Écrivez une fonction `moyennes(notes)` qui retourne un dictionnaire associant à chaque prénom sa moyenne.
2. Écrivez une fonction `admis(notes)` qui retourne la liste triée par ordre alphabétique des prénoms dont la moyenne est supérieure ou égale à 10.
3. Écrivez une fonction `ajoute_note(notes, prenom, note)` qui ajoute une note à un étudiant, en créant son entrée s'il n'existe pas encore.

Exercice 7.6 : Lettres communes ☆

Écrivez une fonction `lettres_communes(mot1, mot2)` qui retourne la liste **triée** des lettres présentes à la fois dans `mot1` et dans `mot2`.

7.9 Bilan du chapitre

À retenir

- Création : `d = {"cle1": valeur1, "cle2": valeur2}` ou `d = {}`.
- Lecture : `d[cle]` (erreur `KeyError` si la clé n'existe pas) ou `d.get(cle, valeur_par_defaut)`.
- Écriture : `d[cle] = valeur` (crée ou remplace) ; suppression : `del d[cle]`.
- `cle in d` teste la présence d'une **clé**.
- Parcours : `for cle in d.keys()`, `for valeur in d.values()`, `for cle, valeur in d.items()`.
- Compter : `d[x] = d.get(x, 0) + 1`.
- *Tuple* : `(3, 7)`, non modifiable ; déballage `x, y = point`. *Set* : `set(liste)` pour les valeurs distinctes.

Faites le point : êtes-vous capable de...

- créer un dictionnaire, lire et modifier une valeur à partir de sa clé ?
- éviter une `KeyError` avec `in` ou `get` ?
- parcourir un dictionnaire en récupérant à la fois les clés et les valeurs ?
- compter des occurrences avec un dictionnaire ?
- déballer un *tuple* et obtenir les valeurs distinctes d'une liste ?
- choisir entre une liste, un dictionnaire, un *tuple* et un *set* ?

CHAPTER 8

DÉBOGUER ET TESTER SON CODE

Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- lire un message d'erreur et reconnaître les types d'erreurs les plus courants ;
- adopter une démarche méthodique pour trouver un *bug* ;
- écrire des tests unitaires simples, avec `print` ou avec `assert` ;

Prérequis : chapitres 1 à 5. **Durée indicative :** 1h.

Pour info

Il existe en Python des outils dédiés au test de programmes (comme `pytest` ou `unittest`). Ce chapitre ne traite pas de ces outils, mais de l'intérêt des tests en général et de bonnes habitudes à prendre dès le début.

Dans ce document, nous avons jusqu'à présent supposé que tout se passait bien : que votre code ne produisait jamais d'erreur et ne contenait jamais de *bug*. Quel que soit votre niveau en Python, ces deux hypothèses sont peu réalistes ! Nous allons donc voir comment vérifier si votre code fait bien ce que l'on attend de lui, et comment comprendre son comportement lorsque ce n'est pas le cas.

On distingue deux situations :

- le programme **s'arrête avec un message d'erreur** : c'est la situation la plus simple, car Python vous indique où et pourquoi ;
- le programme **s'exécute sans erreur mais donne un résultat faux** : c'est un *bug* « silencieux », plus difficile à trouver, et c'est là que les tests sont indispensables.

8.1 Les erreurs en Python

8.1.1 Lire un message d'erreur

Étudions ce qu'il se passe lors de l'exécution du code suivant :

```
x = "12"
y = x + 2
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most␣
↪recent call last)
Cell In[1], line 2
      1 x = "12"
----> 2 y = x + 2

TypeError: can only concatenate str (not "int") to str
```

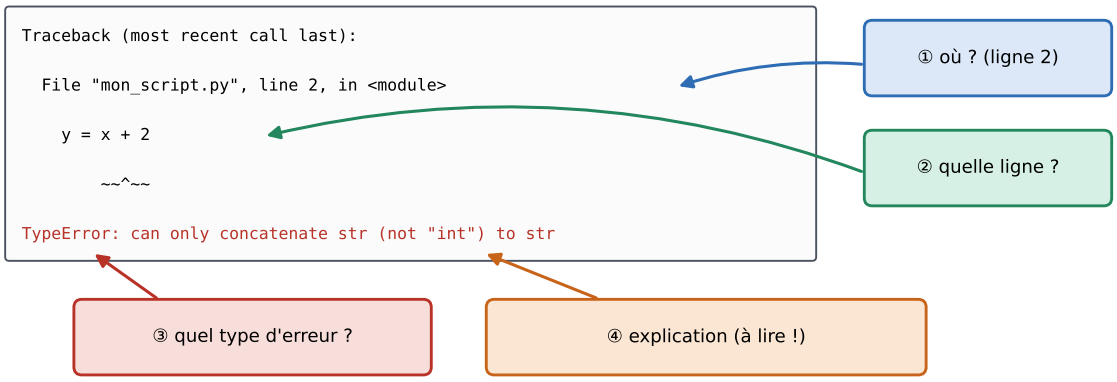


Figure8.1: Rappel du chapitre 1 : anatomie d'un message d'erreur.

Ce type de message ne doit pas vous effrayer, il est là pour vous aider. Il vous fournit de précieuses informations :

1. l'erreur se produit à la ligne 2 du code ;
2. le problème est une `TypeError` : Python ne peut concaténer (`concatenate`) une chaîne (`str`) qu'avec une autre chaîne, et non avec un entier (`int`).

Reste à comprendre pourquoi. Dans la ligne 2 (`y = x + 2`), deux opérations sont effectuées : l'opération + entre `x` et `2`, puis l'affectation du résultat à `y`. Nous avons vu que Python sait effectuer l'opération + avec des opérandes de types variés (nombre + nombre, liste + liste, chaîne + chaîne...). Intéressons-nous donc au type des opérandes : `x`, définie à la ligne 1, est une chaîne de caractères, tandis que `2` est un entier. Or Python ne définit pas d'addition entre une chaîne et un entier. Plus précisément, Python nous dit : « si je pouvais convertir l'entier en chaîne de caractères à la volée, je pourrais faire l'opération +, qui serait alors une concaténation, mais je ne me permets pas de le faire sans que vous l'ayez écrit explicitement ».

Maintenant que nous avons compris le sens de ce *bug*, il nous reste à le corriger. Si nous souhaitons faire la somme du nombre 12 (stocké sous forme de chaîne de caractères dans la variable `x`) et de la valeur 2, nous écrivons :

```
x = "12"
y = int(x) + 2
print(y)
```

```
14
```

Lorsque l'erreur se produit à l'intérieur d'une fonction, le message d'erreur liste toute la **pile d'appels** : la ligne du programme principal qui a appelé la fonction, puis la ligne de la fonction où l'erreur s'est produite. C'est généralement la **dernière** ligne de code mentionnée qui est la plus intéressante.

8.1.2 Les erreurs les plus courantes

Voici un récapitulatif des erreurs rencontrées dans ce document :

Type d'erreur	Signification	Exemple typique
`SyntaxErro`	le code ne respecte pas la « grammaire » de Python	`:` ou parenthèse oubliés, = au lieu de ==
`Indentatio`	décalages incorrects	bloc non indenté après un `if`
`NameError`	nom (variable, fonction) inconnu	faute de frappe, variable utilisée avant d'être définie, module non importé
`TypeError`	opération impossible pour ce type de données	`"12" + 2`, mauvais nombre d'arguments dans un appel
`ValueError`	bon type mais valeur incorrecte	`int("douze")`, `liste.remove(x)` si `x` est absent
`IndexError`	indice hors de la liste	`liste[len(liste)]`
`KeyError`	clé absente d'un dictionnaire	`d["cle_inexistante"]`
`AttributeEr`	attribut ou méthode inexistant pour cet objet	`liste.lower()`, `None.append(...)`
`ZeroDivisio`	division par zéro	`moyenne` d'une liste vide
`FileNotFou`	fichier introuvable	mauvais nom ou mauvais dossier

Testez-vous

Quel type d'erreur produit chacune de ces lignes ?

1. `print(len(5))`
2. `[1, 2, 3][3]`
3. `prnt("bonjour")`
4. `float("3,5")`
5. `{"a": 1}["b"]`
6. `"abc".append("d")`

8.2 Trouver un *bug* : une démarche méthodique

Lorsqu'un programme ne fait pas ce qu'il devrait, il est tentant de modifier le code un peu au hasard jusqu'à ce que « ça marche ». C'est rarement efficace ! Voici une démarche plus fiable.

1. **Reproduire le problème** sur un exemple aussi petit que possible, dont vous connaissez le résultat attendu.
2. **Lire le message d'erreur** s'il y en a un, en commençant par la dernière ligne.
3. **Observer** ce que fait réellement le programme : affichez les valeurs des variables avec `print` à des endroits stratégiques (au début d'une fonction, à chaque tour de boucle...), ou exécutez le code pas à pas avec Python Tutor ou le débogueur de Thonny.
4. **Comparer** avec ce que vous attendiez : à quel moment la valeur d'une variable s'écarte-t-elle de ce que vous aviez prévu ? Le *bug* se trouve juste avant.
5. **Corriger**, puis **re-tester** tous vos exemples (une correction peut en casser une autre !).

8.2.1 Exemple guidé

La fonction suivante est censée calculer la moyenne des valeurs positives d'une liste. Elle s'exécute sans erreur... mais le résultat est faux : la moyenne des valeurs positives de `[4, -2, 6]` devrait valoir 5.

```
def moyenne_positifs(liste):
    somme = 0
    nb = 0
    for x in liste:
        if x > 0:
            somme += x
        nb += 1
    return somme / nb

print(moyenne_positifs([4, -2, 6]))
```

```
3.3333333333333335
```

Ajoutons un affichage dans la boucle pour observer l'évolution des variables :

```
def moyenne_positifs(liste):
    somme = 0
    nb = 0
    for x in liste:
        if x > 0:
            somme += x
        nb += 1
        print(f"x = {x}, somme = {somme}, nb = {nb}")    #␣
 ↪affichage de débogage
    return somme / nb

print(moyenne_positifs([4, -2, 6]))
```

```
x = 4, somme = 4, nb = 1
x = -2, somme = 4, nb = 2
x = 6, somme = 10, nb = 3
3.3333333333333335
```

L'affichage montre que `nb` est incrémenté même lorsque `x = -2` : l'instruction `nb += 1` est mal indentée, elle devrait se trouver **dans** le bloc du `if`. Une fois le *bug* corrigé, on n'oublie pas de retirer l'affichage de débogage :

```
def moyenne_positifs(liste):
    somme = 0
    nb = 0
    for x in liste:
        if x > 0:
            somme += x
            nb += 1
    return somme / nb

print(moyenne_positifs([4, -2, 6]))
```

```
5.0
```

Remarquez au passage que cette fonction produira une `ZeroDivisionError` si la liste ne contient aucune valeur positive : selon l'usage prévu, il faudrait peut-être traiter ce cas à part (en retournant `None`, par exemple).

8.3 Les tests unitaires

8.3.1 Pourquoi tester ?

Pour avoir confiance dans le code que vous écrivez, il faut l'avoir testé sur un ensemble d'exemples qui reflète l'ensemble des situations auxquelles votre programme pourra être confronté. Or, cela représente vite un grand nombre de cas dès que le programme devient un peu complexe. Il est donc recommandé de **découper son code en fonctions** de tailles raisonnables, qui puissent être testées indépendamment les unes des autres. Les tests associés à chacune de ces fonctions sont appelés **tests unitaires**.

Mettre en place de tels tests présente plusieurs avantages :

- vous détectez rapidement un éventuel *bug*, ce qui fait gagner beaucoup de temps ;
- lorsque vous modifiez votre code plus tard (pour le corriger ou lui ajouter une fonctionnalité), vous pouvez relancer les tests pour vous assurer que vous n'avez rien cassé dans les cas qui fonctionnaient déjà.

8.3.2 Choisir ses tests

Prenons un exemple concret. Supposons que l'on souhaite écrire une fonction `bissextile` capable de dire si une année est bissextile. En se renseignant sur le sujet, on apprend qu'une année est bissextile :

- si elle est divisible par 4 et non divisible par 100, ou
- si elle est divisible par 400.

On en déduit un ensemble de tests couvrant les différents cas de figure :

```
print(bissextile(2004))   # True car divisible par 4 et non par 100
print(bissextile(1900))   # False car divisible par 100 et non par␣
 ↪400
print(bissextile(2000))   # True car divisible par 400
print(bissextile(1999))   # False car non divisible par 4
```

À retenir : comment choisir ses tests ?

- Un test par **cas de figure** distinct de l'énoncé (ici, les 4 situations possibles).
- Des **cas limites** : liste vide, liste à un seul élément, valeur nulle, valeurs égales, première et dernière valeur d'un intervalle...
- Pour chaque test, **le résultat attendu doit être connu à l'avance** (calculé à la main, ou trouvé dans l'énoncé).

8.3.3 Tester automatiquement avec `assert`

Avec des `print`, il faut vérifier « à l'œil » que chaque résultat affiché correspond au résultat attendu, ce qui devient fastidieux quand les tests sont nombreux. L'instruction `assert` permet d'automatiser cette vérification : `assert condition` ne fait rien si la condition est vraie, et provoque une erreur (`AssertionError`) si elle est fausse.

```
def bissextile(annee):
    return annee % 4 == 0   # version incomplète, volontairement␣
 ↪fausse

assert bissextile(2004) == True
assert bissextile(1999) == False
assert bissextile(2000) == True
assert bissextile(1900) == False   # ce test échoue
print("Tous les tests sont passés !")
```

```
---------------------------------------------------------------------
 ↪-----------
AssertionError                            Traceback (most␣
 ↪recent call last)
Cell In[6], line 7
      3
      4 assert bissextile(2004) == True
      5 assert bissextile(1999) == False
```

(suite sur la page suivante)

(suite de la page précédente)

```
      6 assert bissextile(2000) == True
----> 7 assert bissextile(1900) == False   # ce test échoue
      8 print("Tous les tests sont passés !")

AssertionError:
```

Le message d'erreur indique précisément quel test a échoué. On peut regrouper les tests d'une fonction dans une fonction de test, que l'on relance après chaque modification :

```
def bissextile(annee):
    if annee % 4 == 0 and annee % 100 != 0:
        return True
    elif annee % 400 == 0:
        return True
    else:
        return False

def test_bissextile():
    assert bissextile(2004) == True     # divisible par 4 et non␣
 ↪par 100
    assert bissextile(1900) == False    # divisible par 100 et non␣
 ↪par 400
    assert bissextile(2000) == True     # divisible par 400
    assert bissextile(1999) == False    # non divisible par 4
    print("test_bissextile : OK")

test_bissextile()
```

```
test_bissextile : OK
```

Attention avec les nombres à virgule

À cause des erreurs d'arrondi, `assert 0.1 + 0.2 == 0.3` échoue ! Pour tester des résultats à virgule, on vérifie plutôt que l'écart est très petit : `assert abs(resultat - attendu) < 1e-9`.

8.4 Exercices

Exercice 8.1 : Trouvez les bugs ☆ ☆

Chacune des fonctions suivantes contient (au moins) un *bug*. Pour chacune, écrivez d'abord un ou plusieurs tests qui mettent le *bug* en évidence, puis corrigez la fonction.

```
# Doit retourner le plus grand élément d'une liste non vide
def maximum(liste):
    m = 0
    for x in liste:
        if x > m:
            m = x
    return m
```

```
# Doit retourner le nombre de voyelles d'un mot
def nb_voyelles(mot):
    n = 0
    for lettre in mot:
        if lettre in "aeiouy":
            n += 1
        return n

# Doit retourner la liste des éléments pairs de la liste
def pairs(liste):
    resultat = []
    for i in range(len(liste)):
        if i % 2 == 0:
            resultat.append(liste[i])
    return resultat
```

Exercice 8.2 : Bissextile en une ligne ☆ ☆

Proposez une réécriture de la fonction `bissextile` qui tienne en une ligne de la forme :

```
def bissextile(annee):
    return CONDITION_COMPLEXE
```

où `CONDITION_COMPLEXE` est un booléen calculé à partir de la valeur de `annee`. Assurez-vous que cette nouvelle fonction passe bien les tests énoncés plus haut.

8.5 Bilan du chapitre

À retenir

- Un message d'erreur se lit en partant de la fin : type d'erreur, explication, puis ligne en cause.
- Pour trouver un *bug* : reproduire sur un petit exemple, observer (`print`, Python Tutor, débogueur), comparer avec l'attendu, corriger, re-tester.
- Tester chaque fonction sur plusieurs cas, y compris les cas limites ; `assert condition` vérifie automatiquement un résultat.

Faites le point : êtes-vous capable de…

- associer un type d'erreur (`TypeError`, `IndexError`…) à sa cause probable ?
- utiliser des `print` pour localiser un *bug* silencieux ?
- choisir un jeu de tests pertinent pour une fonction, avec des cas limites ?
- écrire une fonction de test utilisant `assert` ?

CHAPTER 9

LIRE ET ÉCRIRE DES FICHIERS

Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- lister les fichiers d'un dossier et construire leurs chemins avec le module `os` ;
- comprendre ce qu'est un fichier texte et la notion d'encodage ;
- lire un fichier texte ligne par ligne ;
- lire et écrire des fichiers CSV (module `csv`) et JSON (module `json`) ;
- utiliser `with` pour ne pas oublier de fermer un fichier.

Prérequis : chapitres 1 à 8. **Durée indicative :** 2h.

Avant de commencer

Ce chapitre nécessite de travailler **sur votre ordinateur** (avec Thonny, par exemple) : les fenêtres de code intégrées à la version web n'ont pas accès à vos fichiers. Pour reproduire les exemples, créez un dossier de travail contenant votre script Python et les fichiers de données utilisés dans ce chapitre (`simple.csv`, `entete.csv`, `simple.json` et le sous-dossier `donnees` contenant `fable.txt` et `notes.csv`), dont le contenu est donné au fil du texte. Les chemins de fichiers relatifs (comme `"donnees/fable.txt"`) sont interprétés à partir du **dossier dans lequel le programme est exécuté** : si Python ne trouve pas vos fichiers (`FileNotFoundError`), c'est en général que le script n'est pas lancé depuis le bon dossier.

Un fichier est un ensemble d'informations stocké sur une mémoire (un disque dur, une clé USB…). En lisant des fichiers depuis un programme Python, on va pouvoir traiter des données qui ne sont pas écrites « en dur » dans le programme : un tableau de notes, un texte, des relevés météo… Et en écrivant des fichiers, on pourra conserver les résultats de nos programmes.

9.1 Lister les fichiers d'un dossier avec le module `os`

Il est fréquent de vouloir répéter la même opération sur plusieurs fichiers, par exemple sur tous les fichiers d'extension `.txt` d'un dossier. Le module `os` propose pour cela des fonctions standard de manipulation de fichiers, notamment la fonction `listdir`, qui liste les fichiers et sous-dossiers contenus dans un dossier donné :

```
import os

for nom_fichier in os.listdir("donnees"):
    print(nom_fichier)
```

```
fable.txt
flights.csv
titanic.csv
penguins.csv
machin.csv
truc.txt
notes.csv
```

La fonction `listdir` accepte un chemin absolu (comme `"C:/Users/aude/Documents"` ou `"/home/aude/documents"`) ou relatif (ici, `"donnees"` désigne le sous-dossier `donnees` du dossier de travail courant).

Le nom stocké dans `nom_fichier` ne contient pas le chemin vers le fichier. Or, pour ouvrir ensuite ce fichier, il faudra bien spécifier ce chemin. On utilise pour cela la fonction `os.path.join` :

```
import os

repertoire = "donnees"
for nom_fichier in os.listdir(repertoire):
    nom_complet_fichier = os.path.join(repertoire, nom_fichier)
    print(nom_fichier, "->", nom_complet_fichier)
```

```
fable.txt -> donnees/fable.txt
flights.csv -> donnees/flights.csv
titanic.csv -> donnees/titanic.csv
penguins.csv -> donnees/penguins.csv
machin.csv -> donnees/machin.csv
truc.txt -> donnees/truc.txt
notes.csv -> donnees/notes.csv
```

Il est préférable d'utiliser `os.path.join` plutôt que de concaténer les chaînes de caractères vous-même, car la forme des chemins dépend du système d'exploitation (`\` sous Windows, `/` sous Linux et macOS), ce que `os.path.join` gère pour vous.

9.2 Fichiers textuels et encodage

9.2.1 Qu'est-ce qu'un fichier textuel ?

Dans ce chapitre, nous ne traitons que de fichiers **textuels**, c'est-à-dire de fichiers dont le contenu est lisible en clair lorsqu'on les ouvre avec un éditeur de texte brut (comme le Bloc-notes sous Windows, TextEdit sous macOS, gedit sous Linux... ou Thonny). C'est le cas des fichiers `.txt`, `.csv`, `.json`, et même des fichiers `.py` !

Sous Linux et macOS, la commande `head` affiche le début d'un fichier ; on obtient ici un contenu parfaitement lisible :

```
!head entete.csv
```

```
NOM;PRENOM;AGE
Lemarchand;John;23
Trias;Anne;
```

À l'inverse, un fichier non textuel (une image, un document Word, un fichier Excel...) n'est pas lisible ainsi : ce chapitre ne traite que des fichiers textuels.

9.2.2 L'encodage

Un ordinateur ne stocke que des nombres : pour stocker du texte, il faut donc associer un code à chaque caractère. Il existe plusieurs façons de faire, appelées **encodages**. Nous nous focaliserons ici sur les deux encodages que vous êtes les plus susceptibles de rencontrer :

- l'encodage UTF-8, dont le nom en Python est `"utf-8"` : c'est aujourd'hui l'encodage le plus répandu ;
- l'encodage Latin-1 (ISO-8859-1), dont le nom en Python est `"iso-8859-1"`, que l'on rencontre dans d'anciens fichiers (et dans certains exports de logiciels sous Windows).

La principale différence entre ces deux encodages concerne les caractères accentués. Si vous lisez un fichier avec le mauvais encodage, les accents apparaissent sous une forme étrange (par exemple `Ã©` au lieu de `é`), ou vous obtenez une erreur `UnicodeDecodeError`.

À retenir

Précisez toujours l'encodage lorsque vous ouvrez un fichier texte. En l'absence de contrainte extérieure, utilisez l'encodage UTF-8. Si les accents d'un fichier lu s'affichent mal, essayez l'autre encodage (`"iso-8859-1"`).

9.3 Lire des fichiers textuels

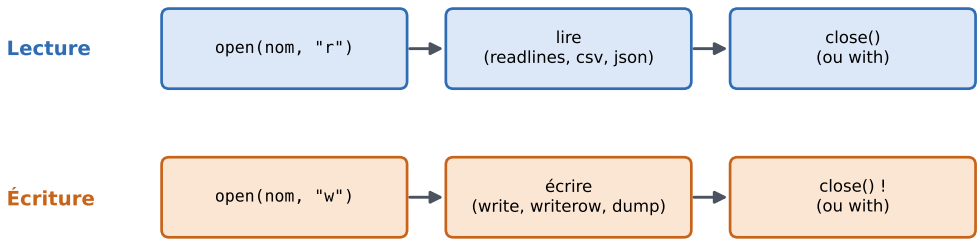


Figure9.1: Les étapes de la lecture et de l'écriture d'un fichier.

La lecture d'un fichier se déroule en deux étapes :

1. **ouvrir** le fichier en lecture ;
2. **parcourir** son contenu.

En supposant que le nom du fichier à ouvrir soit stocké dans la variable `nom_fichier`, le code suivant ouvre un fichier en lecture avec l'encodage UTF-8 :

```
fp = open(nom_fichier, "r", encoding="utf-8")
```

Le second argument (`"r"`, pour *read*) indique que le fichier doit être ouvert en lecture. La variable `fp` (pour *file pointer*) représente le fichier ouvert : elle garde en mémoire l'endroit où nous en sommes dans la lecture du fichier (pour l'instant, le début).

9.3.1 Fichiers textuels génériques

Nous appelons « fichier textuel générique » un fichier dont le contenu n'a pas de structure particulière, comme un texte libre. Une fois ouvert, on peut le lire ligne par ligne :

```
fp = open("donnees/fable.txt", "r", encoding="utf-8")
for ligne in fp.readlines():
    print(ligne)
fp.close()
```

```
La Cigale, ayant chanté

Tout l'été,

Se trouva fort dépourvue

Quand la bise fut venue :

Pas un seul petit morceau

De mouche ou de vermisseau.

Elle alla crier famine
```

(suite sur la page suivante)

(suite de la page précédente)

```
Chez la Fourmi sa voisine,

La priant de lui prêter

Quelque grain pour subsister

Jusqu'à la saison nouvelle.

« Je vous paierai, lui dit-elle,

Avant l'Oût, foi d'animal,

Intérêt et principal. »

La Fourmi n'est pas prêteuse :

C'est là son moindre défaut.

Que faisiez-vous au temps chaud ?

Dit-elle à cette emprunteuse.

- Nuit et jour à tout venant

Je chantais, ne vous déplaise.

- Vous chantiez ? j'en suis fort aise.

Eh bien ! dansez maintenant.
```

La variable `ligne`, de type chaîne de caractères, contient successivement le texte de chacune des lignes du fichier.

Vous remarquez sans doute que des lignes vides sont affichées entre les lignes du texte. En effet, chaque ligne lue se termine par un caractère de retour à la ligne (noté `"\n"`), et `print` ajoute son propre retour à la ligne. On utilise souvent la méthode `strip` pour retirer ce caractère :

```
fp = open("donnees/fable.txt", "r", encoding="utf-8")
for ligne in fp.readlines():
    ligne = ligne.strip()
    print(ligne)
fp.close()
```

```
La Cigale, ayant chanté
Tout l'été,
Se trouva fort dépourvue
Quand la bise fut venue :
Pas un seul petit morceau
De mouche ou de vermisseau.
Elle alla crier famine
Chez la Fourmi sa voisine,
La priant de lui prêter
Quelque grain pour subsister
Jusqu'à la saison nouvelle.
« Je vous paierai, lui dit-elle,
```

(suite sur la page suivante)

(suite de la page précédente)

```
Avant l'Oût, foi d'animal,
Intérêt et principal. »
La Fourmi n'est pas prêteuse :
C'est là son moindre défaut.
Que faisiez-vous au temps chaud ?
Dit-elle à cette emprunteuse.
- Nuit et jour à tout venant
Je chantais, ne vous déplaise.
- Vous chantiez ? j'en suis fort aise.
Eh bien ! dansez maintenant.
```

On peut aussi lire tout le contenu d'un fichier d'un coup, dans une unique chaîne de caractères, avec la méthode `read` :

```
fp = open("donnees/fable.txt", "r", encoding="utf-8")
contenu = fp.read()
fp.close()
print(len(contenu), "caractères")
print(len(contenu.split()), "mots")
```

```
608 caractères
108 mots
```

Une fois la lecture terminée, on **ferme** le fichier avec la méthode `close`.

9.3.2 Mieux : la directive `with`

Il est facile d'oublier de fermer un fichier. Python propose une syntaxe qui s'en charge pour nous, avec le mot-clé `with` :

```
with open("donnees/fable.txt", "r", encoding="utf-8") as fp:
    for ligne in fp.readlines():
        print(ligne.strip())
print("Ici, le fichier est fermé")
```

```
La Cigale, ayant chanté
Tout l'été,
Se trouva fort dépourvue
Quand la bise fut venue :
Pas un seul petit morceau
De mouche ou de vermisseau.
Elle alla crier famine
Chez la Fourmi sa voisine,
La priant de lui prêter
Quelque grain pour subsister
Jusqu'à la saison nouvelle.
« Je vous paierai, lui dit-elle,
Avant l'Oût, foi d'animal,
Intérêt et principal. »
La Fourmi n'est pas prêteuse :
C'est là son moindre défaut.
Que faisiez-vous au temps chaud ?
Dit-elle à cette emprunteuse.
```

(suite sur la page suivante)

(suite de la page précédente)

```
- Nuit et jour à tout venant
Je chantais, ne vous déplaise.
- Vous chantiez ? j'en suis fort aise.
Eh bien ! dansez maintenant.
Ici, le fichier est fermé
```

Le fichier est ouvert et stocké dans la variable `fp` au début du bloc `with`, et il est **automatiquement fermé** dès que l'on sort du bloc (même en cas d'erreur). C'est la façon recommandée d'ouvrir des fichiers en Python.

Testez-vous

Le fichier `donnees/fable.txt` contient 22 lignes. Qu'affiche le programme suivant ?

```
n = 0
with open("donnees/fable.txt", "r", encoding="utf-8") as fp:
    for ligne in fp.readlines():
        if "Fourmi" in ligne:
            n += 1
print(n)
```

9.3.3 Fichiers CSV

Les fichiers CSV (*Comma-Separated Values*) permettent de stocker des données organisées sous la forme de **tableaux** dans des fichiers textuels. Chaque ligne du fichier correspond à une ligne du tableau, et au sein de chaque ligne, les cellules (les colonnes) sont séparées par un délimiteur. À l'origine, ce délimiteur était la virgule (d'où le nom de ce format), mais on rencontre très souvent le point-virgule, notamment dans les fichiers produits par les tableurs en français. Vous pouvez d'ailleurs ouvrir un fichier CSV avec un tableur (LibreOffice Calc, Excel...).

Pour manipuler ces fichiers, Python dispose d'un module dédié, appelé `csv`. Ce module contient notamment une fonction `reader` qui simplifie la lecture des fichiers CSV (remarquez l'argument `delimiter`, qui précise le délimiteur utilisé) :

```
import csv

# Contenu du fichier simple.csv :
# 1;2;3;4;5
# a;b;c;d;e
# xx;xx;xx;xx;xx
# 0.4;0.5;0.7;0.8;0.9

with open("simple.csv", "r", encoding="utf-8") as fp:
    for ligne in csv.reader(fp, delimiter=";"):
        print(ligne)
```

```
['1', '2', '3', '4', '5']
['a', 'b', 'c', 'd', 'e']
['xx', 'xx', 'xx', 'xx', 'xx']
['0.4', '0.5', '0.7', '0.8', '0.9']
```

Contrairement au cas des fichiers textuels génériques, la variable de boucle `ligne` n'est plus une chaîne de caractères mais une **liste** de chaînes de caractères : les cellules de la ligne courante du tableau.

Erreur fréquente

Toutes les valeurs lues dans un fichier CSV sont des **chaînes de caractères**, même celles qui ressemblent à des nombres. Pour faire des calculs, il faut les convertir avec `int` ou `float` : `float(ligne[3])`.

Cas des fichiers à en-tête

Souvent, la première ligne d'un fichier CSV contient les noms des colonnes (on parle d'**en-tête**), comme dans le fichier `entete.csv` suivant :

```
NOM;PRENOM;AGE
Lemarchand;John;23
Trias;Anne;
```

Dans ce cas, il est pratique que chaque ligne lue soit représentée par un **dictionnaire** dont les clés sont les noms des colonnes : c'est ce que permet `csv.DictReader` (à utiliser à la place de `csv.reader`) :

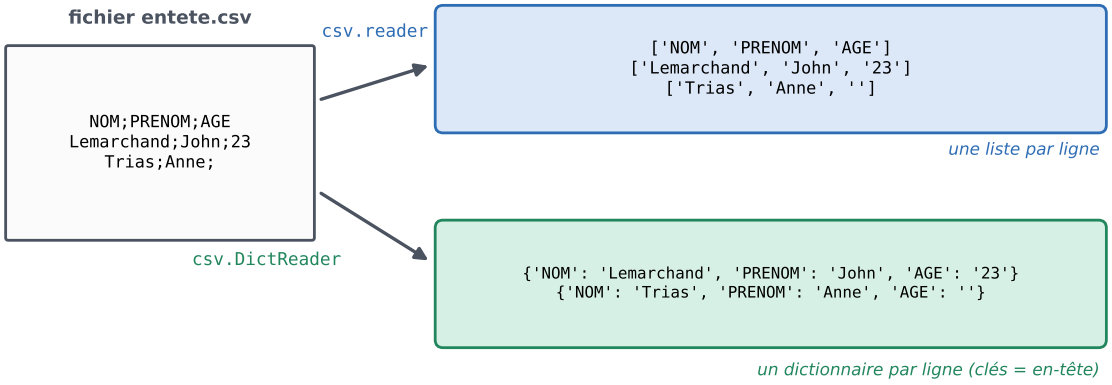


Figure9.2: `csv.reader` produit une liste par ligne ; `csv.DictReader` produit un dictionnaire par ligne, dont les clés sont lues dans l'en-tête.

```
import csv

with open("entete.csv", "r", encoding="utf-8") as fp:
    for ligne in csv.DictReader(fp, delimiter=";"):
        print(ligne["PRENOM"], ligne["NOM"], "a pour âge :",␣
↪ligne["AGE"])
```

```
John Lemarchand a pour âge : 23
Anne Trias a pour âge :
```

Le code est ainsi bien plus lisible : `ligne["AGE"]` est plus parlant que `ligne[2]`.

Exemple guidé : la moyenne d'une colonne

Le fichier `donnees/notes.csv` contient les notes de 8 étudiants dans trois matières :

```
nom;prenom;maths;info;anglais
Martin;Aude;14;17;12
Benali;Yanis;9;12;15
...
```

Calculons la moyenne de la classe en informatique. Il s'agit d'un accumulateur (une somme et un compteur), en n'oubliant pas de convertir les notes en nombres :

```
import csv

somme = 0
nb_etudiants = 0
with open("donnees/notes.csv", "r", encoding="utf-8") as fp:
    for ligne in csv.DictReader(fp, delimiter=";"):
        somme += float(ligne["info"])
        nb_etudiants += 1
print(f"Moyenne en info : {somme / nb_etudiants:.2f}")
```

```
Moyenne en info : 13.00
```

9.3.4 Fichiers JSON

Les fichiers JSON (*JavaScript Object Notation*) permettent de stocker des données **structurées**, par exemple avec une organisation hiérarchique. Un document JSON ressemble beaucoup à un dictionnaire Python (à la nuance près que les clés d'un document JSON sont forcément des chaînes de caractères, écrites entre guillemets doubles). Voici le contenu du fichier `simple.json` :

```
{
    "num_etudiant": "21300000",
    "notes": [12, 5, 14],
    "date_de_naissance": {
        "jour": 1,
        "mois": 1,
        "annee": 1995
    }
}
```

Pour lire de tels fichiers, on dispose du module `json`, qui contient une fonction `load` :

```
import json

with open("simple.json", "r", encoding="utf-8") as fp:
    d = json.load(fp)
print(d)
print(type(d))
print(d["notes"])
print(d["date_de_naissance"]["annee"])
```

```
{'num_etudiant': '21300000', 'notes': [12, 5, 14], 'date_de_
 ↪naissance': {'jour': 1, 'mois': 1, 'annee': 1995}}
<class 'dict'>
[12, 5, 14]
1995
```

Contrairement aux fichiers CSV, les données sont directement converties dans le bon type : `d["notes"]` est une liste d'entiers, et non une chaîne de caractères.

Un fichier JSON peut également contenir une liste de dictionnaires, comme dans l'exemple suivant :

```
[{
    "num_etudiant": "21300000",
    "notes": [12, 5, 14]
},
{
    "num_etudiant": "21300001",
    "notes": [14]
}]
```

Dans ce cas, `json.load` retourne bien évidemment une liste de dictionnaires.

9.4 Écrire des fichiers textuels

L'écriture d'un fichier se déroule en trois étapes :

1. ouvrir le fichier en écriture ;
2. ajouter du contenu dans le fichier ;
3. fermer le fichier (automatiquement si l'on utilise `with`).

Pour ouvrir un fichier en écriture, on utilise le mode `"w"` (pour *write*) :

```
fp = open(nom_fichier, "w", encoding="utf-8", newline="\n")
```

L'argument `newline="\n"` assure que les retours à la ligne seront écrits de la même façon quel que soit le système d'exploitation.

Attention !

Si le fichier existe déjà, ouvrir un fichier en mode `"w"` **efface son contenu** : on repart d'un fichier vide. Si l'on souhaite au contraire ajouter du texte à la fin d'un fichier existant, on utilise le mode `"a"` (pour *append*) :

```
fp = open(nom_fichier, "a", encoding="utf-8", newline="\n")
```

Si l'on n'utilise pas `with`, il ne faut pas oublier de fermer le fichier une fois l'écriture terminée, avec `fp.close()`, sans quoi une partie du contenu pourrait ne pas être écrite sur le disque.

9.4.1 Fichiers textuels génériques

Pour ajouter du contenu à un fichier, on utilise la méthode `write` :

```
with open("ecriture.txt", "w", encoding="utf-8", newline="\n") as␣
↪fp:
    fp.write("La vie est belle\n")
    for i in range(3):
        fp.write(f"Ligne numéro {i}\n")
```

Contrairement à la fonction `print`, la méthode `write` n'ajoute pas de retour à la ligne après la chaîne de caractères : il faut l'inclure vous-même (avec `"\n"`) si vous souhaitez passer à la ligne. De plus, `write` n'accepte que des chaînes de caractères : pour écrire un nombre, il faut le convertir (avec `str` ou dans une f-string).

Vérifions le contenu du fichier en le relisant :

```
with open("ecriture.txt", "r", encoding="utf-8") as fp:
    print(fp.read())
```

```
La vie est belle
Ligne numéro 0
Ligne numéro 1
Ligne numéro 2
```

9.4.2 Fichiers CSV

Le module `csv` contient également une fonction `writer` qui simplifie l'écriture des fichiers CSV :

```
import csv

with open("ecriture.csv", "w", encoding="utf-8", newline="\n") as␣
↪fp:
    csvfp = csv.writer(fp, delimiter=";")
    csvfp.writerow(["nom", "moyenne"])
    csvfp.writerow(["Aude", 14.33])
    csvfp.writerow(["Yanis", 12])
# Après cela, le fichier contient les lignes suivantes :
# nom;moyenne
# Aude;14.33
# Yanis;12
```

La méthode `writerow` prend une liste en argument et écrit dans le fichier les éléments de cette liste, séparés par le délimiteur choisi ; elle se charge elle-même du retour à la ligne.

9.4.3 Fichiers JSON

Le module `json` contient une fonction `dump` qui écrit le contenu d'un dictionnaire (ou d'une liste) dans un fichier JSON :

```
import json

liste = [
    {"a": 5},
    {"b": 3, "a": 7}
]

with open("ecriture.json", "w", encoding="utf-8", newline="\n")␣
 ↪as fp:
    json.dump(liste, fp)
# Après cela, le fichier contient la ligne suivante :
# [{"a": 5}, {"b": 3, "a": 7}]
```

L'argument optionnel `indent` permet d'obtenir un fichier plus lisible, avec des retours à la ligne et une indentation : `json.dump(liste, fp, indent=2)`. Vous pouvez vous référer à la documentation de cette fonction pour plus de détails.

Erreurs fréquentes de ce chapitre

Message d'erreur	Cause probable
`FileNotFoundError: [Errno 2] No such file or directory: 'notes.csv'`	le nom du fichier est mal écrit, ou le programme n'est pas exécuté depuis le dossier qui contient le fichier
`UnicodeDecodeError: 'utf-8' codec can't decode byte ...`	le fichier n'est pas encodé en UTF-8 : essayer `encoding="iso-8859-1"`
`TypeError: write() argument must be str, not int`	`write` n'accepte que des chaînes : convertir avec `str()`
`KeyError: 'Nom'` avec `DictReader`	le nom de colonne ne correspond pas exactement à l'en-tête (majuscules, espaces…), ou le mauvais délimiteur est utilisé
`ValueError: could not convert string to float: ''`	une cellule vide : il faut tester si la valeur est vide avant de la convertir
Le fichier écrit est vide ou incomplet	le fichier n'a pas été fermé : utiliser `with`

9.5 Exercices

Rappel

Ces exercices sont à faire sur votre ordinateur, avec les fichiers de données donnés dans ce chapitre.

Exercice 9.1 : Compter les lignes des fichiers .txt ☆

Écrivez une fonction qui affiche, pour chaque fichier d'extension `.txt` d'un répertoire passé en argument, le nom du fichier ainsi que son nombre de lignes.

Exercice 9.2 : Statistiques sur un texte ☆ ☆

Écrivez une fonction `statistiques(nom_fichier)` qui retourne, pour le fichier texte dont le nom est passé en argument, son nombre de lignes, son nombre de mots et la longueur moyenne d'une ligne (en nombre de mots). Testez-la sur le fichier `donnees/fable.txt`.

Exercice 9.3 : Moyennes des étudiants ☆ ☆

En utilisant le fichier `donnees/notes.csv`, écrivez un programme qui affiche, pour chaque étudiant, son prénom, son nom et sa moyenne dans les trois matières (avec deux décimales), puis le nom de l'étudiant ayant la meilleure moyenne.

Et pandas dans tout ça ?

Au chapitre 11, nous verrons que la bibliothèque `pandas` permet de lire un fichier CSV en une seule ligne (`pd.read_csv(...)`) et de faire des calculs sur ses colonnes sans écrire de boucle. Il reste néanmoins essentiel de comprendre ce qui se passe « sous le capot » (chemins, encodage, délimiteur, conversion des types) : ce sont exactement les mêmes notions qui interviennent, et les mêmes erreurs que l'on rencontre, avec `pandas`.

9.6 Bilan du chapitre

À retenir

- `os.listdir(dossier)` liste un dossier ; `os.path.join(dossier, nom)` construit un chemin.
- Ouvrir : `with open(nom, mode, encoding="utf-8") as fp:` avec le mode `"r"` (lecture), `"w"` (écriture, **écrase** le fichier) ou `"a"` (ajout à la fin).

- Texte : `fp.readlines()` (liste des lignes, à nettoyer avec `strip`), `fp.read()` (tout le texte), `fp.write(chaine)` (ajouter `"\n"` soi-même).
- CSV : `csv.reader(fp, delimiter=";")` (listes), `csv.DictReader(fp, delimiter=";")` (dictionnaires), `csv.writer(fp, delimiter=";").writerow(liste)`. Les valeurs lues sont des chaînes.
- JSON : `json.load(fp)`, `json.dump(donnees, fp)`.

Faites le point : êtes-vous capable de...

- lister les fichiers d'un dossier et construire leur chemin complet ?
- lire un fichier texte ligne par ligne ?
- lire un fichier CSV avec en-tête et faire des calculs sur une colonne ?
- lire un fichier JSON et accéder à une information imbriquée ?
- écrire des résultats dans un fichier CSV ou JSON ?
- diagnostiquer une `FileNotFoundError` ou un problème d'encodage ?

CHAPTER 10

NUMPY ET LE CALCUL MATRICIEL

Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- créer des tableaux `numpy` (vecteurs et matrices) et connaître leur forme ;
- faire la différence entre opérations élément par élément et produit matriciel ;
- extraire des sous-parties d'un tableau ;
- calculer des statistiques (somme, moyenne, écart-type…) sur tout un tableau ou ligne par ligne / colonne par colonne ;
- utiliser des masques booléens pour compter ou filtrer des valeurs.

Prérequis : chapitres 1 à 5 (et quelques notions de calcul matriciel). Ce chapitre et les suivants se travaillent sur votre ordinateur, de préférence dans un *notebook* Jupyter (voir l'introduction) : les fenêtres de code intégrées ne disposent pas de `numpy`. **Durée indicative :** 1h30.

Ce chapitre traite d'un module en particulier : le module `numpy`, dont le nom est un raccourci pour *Numerical Python*. Cette bibliothèque fournit des outils de calcul numérique efficaces : elle permet de manipuler des vecteurs et des matrices (voire des tableaux à plus de deux dimensions) et constitue la base de la plupart des outils d'analyse de données en Python (comme `pandas` et `matplotlib`, que nous verrons dans les chapitres suivants, ou `scikit-learn` pour l'apprentissage automatique).

Installer numpy

Le module `numpy` ne fait pas partie de la bibliothèque standard : installez-le si besoin (dans Thonny : *Outils* > *Gérer les paquets*… ; en ligne de commande : `pip install numpy` ; il est inclus dans Anaconda).

Avant toute chose, il faut importer ce module :

```
import numpy as np
```

Le module est renommé en `np` lors de l'import : c'est une convention très répandue, que vous retrouverez dans tous les exemples en ligne.

Ce chapitre est très loin de couvrir l'ensemble des fonctionnalités de `numpy` : n'hésitez pas à consulter la documentation `numpy` pour trouver ce qui pourrait vous être utile.

10.1 Les tableaux numpy

L'objet de base de `numpy` est le **tableau multi-dimensionnel** (de type `ndarray`). On peut créer un vecteur (tableau à une dimension) à partir d'une liste :

```
vec = np.array([1, 4, 6, 7])
print(vec)
print(vec.ndim)     # nombre de dimensions
print(vec.shape)    # forme : ici 4 éléments
print(vec.dtype)    # type des données stockées
```

```
[1 4 6 7]
1
(4,)
int64
```

et une matrice (tableau à deux dimensions) à partir d'une liste de listes, chaque sous-liste correspondant à une ligne :

```
A = np.array([[0, 1, 2],
              [3, 4, 5]])
print(A)
print(A.ndim)
print(A.shape)    # (nombre de lignes, nombre de colonnes)
print(A.T)        # transposée
```

```
[[0 1 2]
 [3 4 5]]
2
(2, 3)
[[0 3]
 [1 4]
 [2 5]]
```

Contrairement aux listes Python, un tableau `numpy` a **un type unique** pour tous ses éléments (ici des entiers) : on ne peut pas mélanger des types différents dans un même tableau.

On accède aux éléments d'une matrice avec la syntaxe `A[i, j]` (ligne `i`, colonne `j`, toujours en commençant à 0) :

```
print(A[1, 2])
A[0, 0] = 10
print(A)
```

```
5
[[10  1  2]
 [ 3  4  5]]
```

Testez-vous

Avec `M = np.array([[1, 2], [3, 4], [5, 6]])`, que valent `M.shape`, `M[2, 0]`, `M.T.shape` et `M[0, 1] + M[1, 0]` ?

10.2 Les opérations élément par élément

La grande force de `numpy` est que les opérations arithmétiques s'appliquent **à tous les éléments d'un tableau à la fois**, sans avoir à écrire de boucle :

```
vec = np.array([1, 4, 6, 7])
print(2.5 * vec)
print(vec + 10)
print(vec ** 2)
```

```
[ 2.5 10.  15.  17.5]
[11 14 16 17]
[ 1 16 36 49]
```

Les opérations entre deux tableaux de même forme sont également effectuées **élément par élément** :

```
u = np.array([1, 2, 3])
v = np.array([10, 20, 30])
print(u + v)
print(u * v)
```

```
[11 22 33]
[10 40 90]
```

`numpy` propose aussi des versions « vectorisées » des fonctions mathématiques usuelles, qui s'appliquent à chaque élément : `np.sqrt`, `np.exp`, `np.log`, `np.sin`...

```
print(np.sqrt(np.array([1, 4, 9, 16])))
```

```
[1. 2. 3. 4.]
```

À comparer avec les listes

Avec des **listes**, `[1, 2] * 2` vaut `[1, 2, 1, 2]` (répétition) et `[1, 2] + [3, 4]` vaut `[1, 2, 3, 4]` (concaténation). Avec des **tableaux numpy**, `np.array([1, 2]) * 2` vaut `[2, 4]` et `np.array([1, 2]) + np.array([3, 4])` vaut `[4, 6]`. Le même opérateur n'a donc pas du tout le même sens selon le type des données !

10.3 Produit matriciel

Conséquence de ce qui précède : le produit * entre deux matrices est le produit **élément par élément**, et non le produit matriciel que l'on utilise en mathématiques. Pour effectuer un produit matriciel, on utilise l'opérateur `@`.

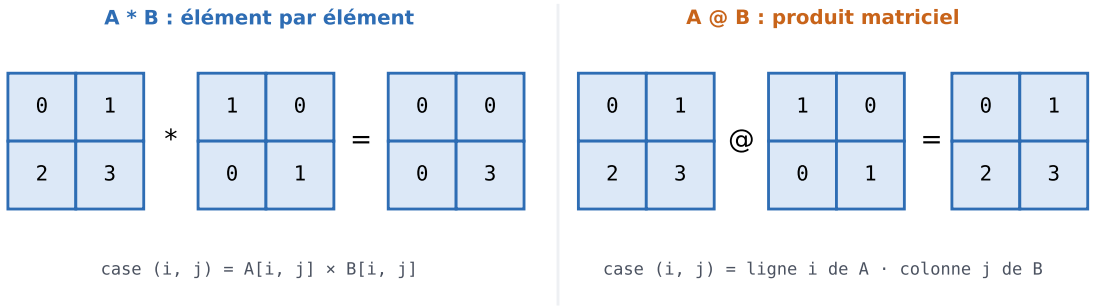


Figure10.1: Le produit élément par élément `A * B` et le produit matriciel `A @ B`, avec `B` la matrice identité.

```
A = np.array([[0, 1], [2, 3]])
I = np.array([[1, 0], [0, 1]])
print(A * I)    # élément par élément
print(A @ I)    # produit matriciel : on retrouve A
```

```
[[0 0]
 [0 3]]
[[0 1]
 [2 3]]
```

De même, `A ** 2` élève au carré chacun des éléments de `A`, ce qui est différent du produit de `A` par elle-même :

```
print(A ** 2)
print(A @ A)
```

```
[[0 1]
 [4 9]]
[[ 2  3]
 [ 6 11]]
```

Pour l'addition, en revanche, il n'y a pas de confusion possible : `A + A` est bien la somme matricielle.

10.4 Créer des tableaux usuels

numpy permet de créer très simplement des tableaux remplis de 0, de 1, la matrice identité, ou des séquences de valeurs :

```
print(np.zeros((2, 3)))         # (2, 3) est la forme du tableau à␣
↪produire
print(np.ones((2, 3)))
print(np.eye(2))                # eye -> matrice identité
print(np.arange(10))            # équivalent de range, mais␣
↪produit un tableau
print(np.linspace(0, 1, 11))    # 11 valeurs régulièrement␣
↪espacées entre 0 et 1 (inclus)
```

```
[[0. 0. 0.]
 [0. 0. 0.]]
[[1. 1. 1.]
 [1. 1. 1.]]
[[1. 0.]
 [0. 1.]]
[0 1 2 3 4 5 6 7 8 9]
[0.  0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9 1. ]
```

La méthode reshape permet de changer la forme d'un tableau (sans changer le nombre total d'éléments) :

```
m = np.arange(10)
print(m.reshape((5, 2)))    # 5 lignes et 2 colonnes
```

```
[[0 1]
 [2 3]
 [4 5]
 [6 7]
 [8 9]]
```

Enfin, le sous-module np.random permet de générer des tableaux de nombres aléatoires, par exemple np.random.randint(1, 7, size=10) pour simuler 10 lancers de dé.

10.5 Accéder à des sous-parties d'un tableau

Comme pour les listes, on peut extraire des « tranches » (*slices*) d'un tableau, avec la syntaxe debut:fin (fin exclue) **dans chaque dimension** :

```
M = np.array([[0, 1, 2, 3, 4],
              [5, 6, 7, 8, 9],
              [10, 11, 12, 13, 14],
              [15, 16, 17, 18, 19]])
print(M[:2, 3:])    # lignes 0 et 1, colonnes à partir de 3
```

```
[[3 4]
 [8 9]]
```

```
print(M[1:3, :])    # lignes 1 et 2, toutes les colonnes
print(M[:, 0])      # la première colonne (toutes les lignes)
print(M[-1])        # la dernière ligne
```

```
[[ 5  6  7  8  9]
 [10 11 12 13 14]]
[ 0  5 10 15]
[15 16 17 18 19]
```

On peut également utiliser une liste (ou un tableau) d'indices pour accéder aux éléments situés à ces positions :

```
v = np.array([1, 5, 7, 9, 12])
print(v[[0, 4]])    # les éléments d'indices 0 et 4
```

```
[ 1 12]
```

Testez-vous

Avec la matrice `M` ci-dessus, comment extraire :

1. la troisième colonne ?
2. le bloc formé des deux dernières lignes et des deux dernières colonnes ?
3. l'élément 13 ?

10.6 Statistiques sur les tableaux

Une fois un tableau défini, on peut très facilement calculer la somme de ses éléments, son minimum, son maximum, sa moyenne ou son écart-type :

```
print(np.sum(M))     # peut aussi s'écrire M.sum()
print(np.min(M))     # M.min()
print(np.max(M))     # M.max()
print(np.mean(M))    # M.mean()
print(np.std(M))     # M.std()
```

```
190
0
19
9.5
5.766281297335398
```

Pour toutes ces opérations, deux syntaxes co-existent (`np.min(M)` et `M.min()`), qui donnent le même résultat.

10.6.1 Calculs par ligne ou par colonne : l'argument `axis`

On peut aussi effectuer ces opérations **ligne par ligne** ou **colonne par colonne**, grâce à l'argument `axis`. Prenons l'exemple d'un tableau de notes, avec une ligne par examen et une colonne par étudiant :

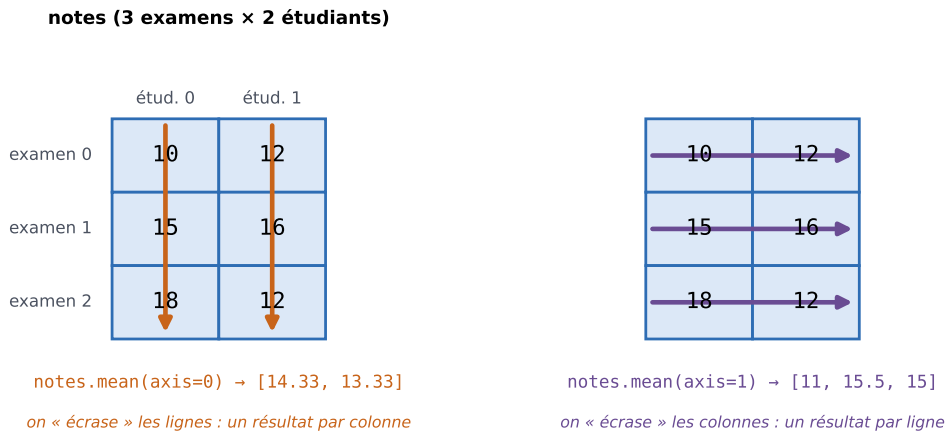


Figure10.2: Avec `axis=0`, on agrège le long des lignes (on obtient un résultat par colonne) ; avec `axis=1`, on agrège le long des colonnes (un résultat par ligne).

```
notes = np.array([[10, 12],
                  [15, 16],
                  [18, 12]])
print(notes.mean(axis=0))    # moyenne de chaque étudiant (colonne)
print(notes.mean(axis=1))    # moyenne de chaque examen (ligne)
```

```
[14.33333333 13.33333333]
[11.  15.5 15. ]
```

```{admonition} Astuce pour retenir le sens de `axis` :class: tip

`axis=0` fait « disparaître » la dimension 0 (les lignes) : le résultat a autant de valeurs qu'il y a de **colonnes**. `axis=1` fait disparaître la dimension 1 (les colonnes) : le résultat a autant de valeurs qu'il y a de **lignes**. En cas de doute, vérifiez la forme du résultat avec `.shape` !

Masques booléens

Enfin, appliquer une comparaison à un tableau produit un tableau de booléens, appelé **masque** :

```{code-cell} ipython3
print(M > 5)   # True à chaque position où l'élément de M est > 5
```

Comme `True` compte pour 1 et `False` pour 0, la somme d'un masque donne le nombre de valeurs qui vérifient la condition :

```
print(np.sum(M > 5))
```

```
14
```
```

Et un masque permet de sélectionner les valeurs qui vérifient la condition :

```
print(M[M > 15])
```

```
[16 17 18 19]
```

10.7 Bonnes pratiques : éviter les boucles

Utilisez autant que possible les fonctions prédéfinies de `numpy` plutôt que de recoder les opérations vous-même avec des boucles. C'est plus court à écrire, et surtout **beaucoup plus rapide** à l'exécution, comme le montre la comparaison suivante :

```
tableau = np.ones((100, 10))
```

```
%%timeit
# %%timeit (spécifique à Jupyter) mesure le temps d'exécution d'un morceau de code
tableau.sum()
```

```
909 ns ± 73 ns per loop (mean ± std. dev. of 7 runs, 1,000,000 loops each)
```

```
%%timeit
s = 0
for v in tableau:  # À éviter !
    s += v.sum()
```

```
75.2 µs ± 6.18 µs per loop (mean ± std. dev. of 7 runs, 10,000 loops each)
```

La version avec une boucle est beaucoup plus lente (regardez les unités : µs pour *microsecondes*, ns pour *nanosecondes*). Sur de gros volumes de données, la différence se compte en secondes, voire en minutes.

Si vous venez de R ou de SAS

- Un tableau `numpy` à une dimension est l'équivalent d'un vecteur R, et à deux dimensions d'une matrice : les opérations `+`, `*`, `**`... s'appliquent élément par élément, comme en R.
- Le produit matriciel `%*%` s'écrit `@` ; la transposée `t(A)` s'écrit `A.T`.
- `apply(M, 2, mean)` (par colonne) s'écrit `M.mean(axis=0)`, et `apply(M, 1, mean)` (par ligne) `M.mean(axis=1)`.
- `seq(0, 1, length.out = 11)` s'écrit `np.linspace(0, 1, 11)` ; `rep(0, 5)` s'écrit `np.zeros(5)`.
- `M[M > 5]` fonctionne comme en R ; `sum(M > 5)` s'écrit `np.sum(M > 5)`.

10.8 Exercices

Exercice 10.1 : Somme des entiers ☆

Calculez en `numpy`, sans boucle, la somme des `n` premiers entiers (de 0 à `n - 1`), puis la somme de leurs carrés, pour `n` fixé.

Exercice 10.2 : Notes d'examens ☆

Supposons qu'on ait stocké dans le tableau suivant les notes reçues par 2 étudiants à 3 examens :

```
notes = np.array(
  [[10, 12],
   [15, 16],
   [18, 12]]
)
```

1. Calculez la moyenne de chacun des deux étudiants.
2. Calculez le nombre de notes supérieures à 12 contenues dans ce tableau.
3. Calculez la meilleure note obtenue à chaque examen.

Exercice 10.3 : Centrer et réduire ☆ ☆

En statistique, **centrer-réduire** une série de valeurs consiste à lui soustraire sa moyenne puis à diviser le résultat par son écart-type. Écrivez une fonction `centre_reduit(x)` qui retourne la version centrée-réduite d'un vecteur `numpy`, puis vérifiez que le résultat a bien une moyenne nulle (aux erreurs d'arrondi près) et un écart-type égal à 1.

Exercice 10.4 : Simulation de lancers de dés ☆ ☆

À l'aide de `np.random.randint`, simulez 10 000 lancers de deux dés (un tableau de forme `(10000, 2)`), calculez la somme de chaque lancer, puis la proportion des lancers dont la somme vaut 7. Comparez avec l'exercice 4.7 : le code est-il plus simple ? Plus rapide ?

10.9 Bilan du chapitre

À retenir

- `import numpy as np` ; `np.array(liste)` ; attributs `.shape`, `.ndim`, `.dtype` ; transposée `.T`.
- Les opérations (`+`, `*`, `**`, fonctions `np.sqrt`...) s'appliquent **élément par élément** ; le produit matriciel s'écrit `@`.
- Constructeurs : `np.zeros`, `np.ones`, `np.eye`, `np.arange`, `np.linspace`, `.reshape`.
- Accès : `M[i, j]`, `M[debut:fin, debut:fin]`, `M[:, j]` (colonne `j`).
- Statistiques : `sum`, `min`, `max`, `mean`, `std`, avec `axis=0` (par colonne) ou `axis=1` (par ligne).
- Masques : `M > 5`, `np.sum(M > 5)`, `M[M > 5]`.
- Pas de boucle sur un tableau `numpy` quand une fonction existe !

Faites le point : êtes-vous capable de...

- créer un vecteur ou une matrice et connaître sa forme ?
- prédire le résultat de `A * B` et de `A @ B` ?
- extraire une ligne, une colonne ou un bloc d'une matrice ?
- calculer une moyenne par ligne ou par colonne ?
- compter les valeurs vérifiant une condition avec un masque ?

CHAPTER 11

DÉCOUVRIR SES DONNÉES AVEC PANDAS

Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- expliquer ce que sont un `DataFrame` et une `Series` ;
- charger un fichier CSV avec `pd.read_csv` (y compris un fichier « à la française ») ;
- obtenir une vue d'ensemble d'un jeu de données (`head`, `shape`, `info`, `describe`, `value_counts`) ;
- sélectionner des colonnes, filtrer des lignes et trier un tableau ;
- utiliser `loc` et `iloc`.

Prérequis : chapitres 1 à 10 (en particulier les listes, les dictionnaires et les masques `numpy`).
Durée indicative : 2h30.

En analyse de données, on travaille presque toujours sur des **tableaux** : une ligne par individu observé (un étudiant, un client, une commune...) et une colonne par caractéristique mesurée (âge, revenu, population...). La bibliothèque `pandas` est l'outil de référence en Python pour manipuler de tels tableaux : on peut la voir comme un **tableur programmable**, avec lequel chaque manipulation est écrite dans du code, et donc reproductible, vérifiable et réutilisable sur de nouvelles données.

Où travailler ?

À partir de ce chapitre, travaillez dans un **notebook Jupyter** (avec Anaconda, VS Code, ou en ligne sur Google Colab, voir l'introduction) : on y voit directement les tableaux et les graphiques sous chaque cellule de code, ce qui est idéal pour explorer des données. Les fenêtres de code intégrées à la version web de ce document ne disposent pas de `pandas`.

Si `pandas` n'est pas installé, installez-le avec `pip install pandas` (il est inclus dans Anaconda et dans Google Colab).

Par convention, on importe `pandas` sous le nom `pd` :

```
import pandas as pd
```

Si vous venez de R ou de SAS

Un `DataFrame` est l'équivalent d'un `data.frame` (ou d'un *tibble*) en R, et d'une table SAS. Voici les correspondances des opérations de ce chapitre :

Opération	R (base / `dplyr`)	SAS	`pandas`
Lire un CSV	`read.csv("f.csv"), read.csv2`	`PROC IMPORT`	`pd.read_csv("f.csv")(sep=";", decimal=",")`
Premières lignes	`head(df)`	`PROC PRINT (OBS=5)`	`df.head()`
Dimensior	`dim(df), nrow(df)`	`PROC CONTENTS`	`df.shape,len(df)`
Structure	`str(df)`	`PROC CONTENTS`	`df.info(),df.dtypes`
Statistique	`summary(df)`	`PROC MEANS`	`df.describe()`
Effectifs	`table(df$x)`	`PROC FREQ`	`df["x"].value_counts()`
Une colonne	`df$x, df[["x"]]`		`df["x"]`
Plusieurs colonnes	`select(df, x, y)`	`KEEP x y;`	`df[["x", "y"]]`
Filtrer	`filter(df, x > 5 & y == "a")`	`WHERE x > 5 AND y = "a";`	`df[(df["x"] > 5) & (df["y"] == "a")]`
Trier	`arrange(df, desc(x))`	`PROC SORT; BY DESCENDING x;`	`df.sort_values("x", ascending=False)`
Valeur manquant	`NA, is.na(x)`	`.,MISSING(x)`	`NaN,df["x"].isna()`

Deux différences importantes :

- les lignes d'un `DataFrame` ont un **index** (par défaut 0, 1, 2...), qui est conservé lors d'un filtre : après un filtre, les numéros de lignes ne sont donc plus consécutifs (utilisez `reset_index(drop=True)` pour les renuméroter) ;
- les colonnes se désignent par des **chaînes de caractères** (`df["x"]`), et non par des noms nus comme dans `dplyr` ou SAS.

11.1 Le jeu de données de ce chapitre

Nous allons travailler sur un jeu de données réel, très utilisé pour apprendre l'analyse de données : les **manchots de l'archipel Palmer** (*Palmer penguins*), collectés en Antarctique par la Dr. Kristen Gorman entre 2007 et 2009 et diffusés librement. Il décrit 344 manchots de trois espèces (Adélie, à jugulaire — *Chinstrap* — et papou — *Gentoo*), vivant sur trois îles, avec pour chacun :

Colonne	Signification
`species`	espèce
`island`	île
`bill_length_mm`, `bill_depth_mm`	longueur et épaisseur du bec (en mm)
`flipper_length_mm`	longueur de la nageoire (en mm)
`body_mass_g`	masse (en g)
`sex`	sexe (`MALE` ou `FEMALE`)

Le fichier se trouve dans `donnees/penguins.csv`.

Si vous n'avez pas le fichier

La bibliothèque `seaborn` (que nous utiliserons au chapitre 13) peut télécharger ce jeu de données pour vous : `import seaborn as sns` puis `df = sns.load_dataset("penguins")` (une connexion internet est nécessaire). Le tableau obtenu est le même, aux majuscules près dans la colonne `sex` (`Male` / `Female`).

11.2 `DataFrame` et `Series`

Chargeons le fichier avec la fonction `pd.read_csv` et affichons ses premières lignes avec la méthode `head` :

```
df = pd.read_csv("donnees/penguins.csv")
df.head()
```

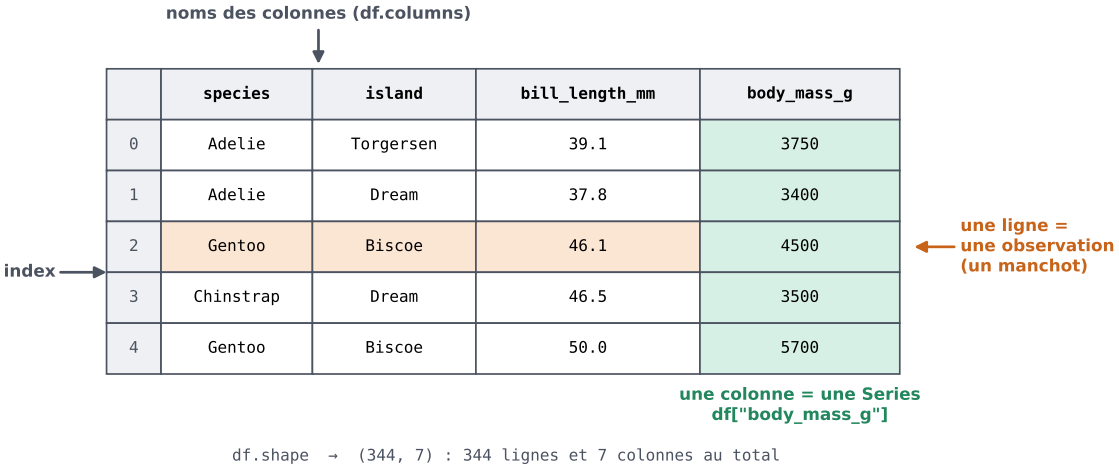
```
  species     island  bill_length_mm  bill_depth_mm  \
0  Adelie  Torgersen            39.1           18.7
1  Adelie  Torgersen            39.5           17.4
2  Adelie  Torgersen            40.3           18.0
3  Adelie  Torgersen             NaN            NaN
4  Adelie  Torgersen            36.7           19.3

   flipper_length_mm  body_mass_g     sex
0              181.0       3750.0    MALE
1              186.0       3800.0  FEMALE
2              195.0       3250.0  FEMALE
3                NaN          NaN     NaN
4              193.0       3450.0  FEMALE
```

Remarque

Dans un notebook, la dernière expression d'une cellule est affichée automatiquement : c'est pourquoi on écrit simplement `df.head()` sans `print`. Les tableaux sont alors joliment mis en forme. Dans un script `.py`, il faut écrire `print(df.head())`.

L'objet `df` est un **`DataFrame`** : un tableau à deux dimensions, dont chaque colonne a un nom et chaque ligne une étiquette (l'"**index**, ici simplement les entiers 0, 1, 2...). Chaque colonne, prise isolément, est une **`Series`** : une suite de valeurs du même type, accompagnée de l'index.



	species	island	bill_length_mm	body_mass_g
0	Adelie	Torgersen	39.1	3750
1	Adelie	Dream	37.8	3400
2	Gentoo	Biscoe	46.1	4500
3	Chinstrap	Dream	46.5	3500
4	Gentoo	Biscoe	50.0	5700

Figure11.1: Anatomie d'un `DataFrame` (ici un extrait de 5 lignes et 4 colonnes du jeu de données).

```
masses = df["body_mass_g"]
print(type(df))
print(type(masses))
masses.head()
```

```
<class 'pandas.DataFrame'>
<class 'pandas.Series'>
```

```
0    3750.0
1    3800.0
2    3250.0
3       NaN
4    3450.0
Name: body_mass_g, dtype: float64
```

On peut aussi créer un `DataFrame` « à la main », à partir d'un dictionnaire associant à chaque nom de colonne la liste de ses valeurs :

```
petit = pd.DataFrame({
    "prenom": ["Aude", "Yanis", "Maëlle"],
    "maths": [14, 9, 16],
    "info": [17, 12, 13]
})
petit
```

```
   prenom  maths  info
0    Aude     14    17
1   Yanis      9    12
2  Maëlle     16    13
```

À retenir

- Un `DataFrame` est un tableau : **une ligne par observation, une colonne par variable**.
- Une colonne d'un `DataFrame` est une `Series`, que l'on obtient avec `df["nom_de_colonne"]`.
- Une `Series` ressemble à un tableau `numpy` à une dimension (on peut faire des calculs sur toute la colonne d'un coup, sans boucle), avec en plus un index.

11.3 Charger un fichier CSV

`pd.read_csv` accepte de nombreux arguments pour s'adapter au format du fichier. Les plus utiles sont :

Argument	Rôle	Exemple
`sep`	délimiteur des colonnes (par défaut `","`)	`sep=";"`
`decimal`	séparateur décimal (par défaut `"."`)	`decimal=","`
`encoding`	encodage du fichier (par défaut UTF-8)	`encoding="iso-8859-1"`
`index_col`	colonne à utiliser comme index	`index_col="prenom"`
`na_values`	valeurs à considérer comme manquantes	`na_values=["NA", "-"]`

Les fichiers produits par un tableur réglé en français utilisent souvent le point-virgule comme délimiteur et la virgule comme séparateur décimal. Le fichier `donnees/notes.csv`, déjà utilisé au chapitre 9, utilise des points-virgules :

```
notes = pd.read_csv("donnees/notes.csv", sep=";")
notes
```

```
       nom  prenom  maths  info  anglais
0   Martin    Aude     14    17       12
1   Benali   Yanis      9    12       15
2  Le Goff  Maëlle     16    13       18
3   Dupont    Hugo      7    10        9
4   Nguyen    Linh     12    19       14
5  Kerbrat   Erwan     11     8       13
6   Diallo   Fatou     15    14       16
7   Moreau    Inès     10    11        8
```

Comparez avec le chapitre 9 : une seule ligne suffit à lire le fichier, et les notes sont **automatiquement converties en nombres** !

Erreur fréquente

Si vous oubliez `sep=";"` pour un fichier séparé par des points-virgules, `pandas` ne voit qu'**une seule colonne**, dont le nom est toute la ligne d'en-tête (`"nom;prenom;maths;info;anglais"`). Premier réflexe après un chargement : **toujours regarder le résultat** avec `head()` et `shape`.

11.4 Explorer un jeu de données

Avant toute analyse, il faut prendre connaissance des données : combien de lignes ? quelles colonnes ? quels types ? y a-t-il des valeurs manquantes ? Voici les outils de base.

11.4.1 Dimensions et colonnes

```
print(df.shape)      # (nombre de lignes, nombre de colonnes) : un␣
 ↪tuple
print(df.columns)    # noms des colonnes
```

```
(344, 7)
Index(['species', 'island', 'bill_length_mm', 'bill_depth_mm',
       'flipper_length_mm', 'body_mass_g', 'sex'],
      dtype='str')
```

Les méthodes `head(n)` et `tail(n)` affichent les `n` premières et dernières lignes (5 par défaut) ; `sample(n)` affiche `n` lignes tirées au hasard.

11.4.2 Types et valeurs manquantes : `info`

```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 344 entries, 0 to 343
Data columns (total 7 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   species            344 non-null    str
 1   island             344 non-null    str
 2   bill_length_mm     342 non-null    float64
 3   bill_depth_mm      342 non-null    float64
 4   flipper_length_mm  342 non-null    float64
 5   body_mass_g        342 non-null    float64
 6   sex                333 non-null    str
dtypes: float64(4), str(3)
memory usage: 18.9 KB
```

`info` indique, pour chaque colonne, le nombre de valeurs non manquantes (*non-null*) et le type des données : `float64` pour des nombres à virgule, `int64` pour des entiers, `object` ou `str` pour du texte. On voit ici que le sexe n'est pas renseigné pour 11 manchots (333 valeurs

sur 344), et que 2 manchots n'ont pas de mesures. Dans `pandas`, une valeur manquante est notée `NaN` (*Not a Number*) ; nous verrons au chapitre suivant comment les traiter.

11.4.3 Statistiques descriptives : `describe`

```
df.describe()
```

```
       bill_length_mm  bill_depth_mm  flipper_length_mm  body_
↪mass_g
count      342.000000     342.000000         342.000000   342.
↪000000
mean        43.921930      17.151170         200.915205  4201.
↪754386
std          5.459584       1.974793          14.061714   801.
↪954536
min         32.100000      13.100000         172.000000  2700.
↪000000
25%         39.225000      15.600000         190.000000  3550.
↪000000
50%         44.450000      17.300000         197.000000  4050.
↪000000
75%         48.500000      18.700000         213.000000  4750.
↪000000
max         59.600000      21.500000         231.000000  6300.
↪000000
```

Pour chaque colonne numérique, `describe` donne le nombre de valeurs (`count`), la moyenne (`mean`), l'écart-type (`std`), le minimum, les quartiles (25 %, 50 % = médiane, 75 %) et le maximum. C'est un excellent moyen de repérer des valeurs aberrantes (une masse négative, un âge de 300 ans...).

11.4.4 Variables qualitatives : `value_counts` et `unique`

Pour une colonne contenant des catégories, on compte les effectifs de chaque modalité avec `value_counts` :

```
df["species"].value_counts()
```

```
species
Adelie       152
Gentoo       124
Chinstrap     68
Name: count, dtype: int64
```

```
print(df["island"].unique())     # les valeurs distinctes
print(df["island"].nunique())    # leur nombre
print(df["species"].value_counts(normalize=True))    # proportions␣
↪plutôt qu'effectifs
```

```
<StringArray>
['Torgersen', 'Biscoe', 'Dream']
```

(suite sur la page suivante)

(suite de la page précédente)

```
Length: 3, dtype: str
3
species
Adelie       0.441860
Gentoo       0.360465
Chinstrap    0.197674
Name: proportion, dtype: float64
```

11.4.5 Calculs sur une colonne

Comme pour les tableaux `numpy`, les méthodes `mean`, `median`, `std`, `min`, `max`, `sum`... s'appliquent à toute une colonne (les valeurs manquantes sont automatiquement ignorées) :

```
print(df["body_mass_g"].mean())
print(df["body_mass_g"].median())
print(df["flipper_length_mm"].max())
```

```
4201.754385964912
4050.0
231.0
```

Testez-vous

1. Quel est le type de `df.shape` ? Comment obtenir uniquement le nombre de lignes ?
2. Que retourne `df["species"]` ? Et `df["species"].value_counts()` ?
3. Pourquoi `df["sex"].mean()` n'a-t-il pas de sens ?

11.5 Sélectionner des colonnes

Nous l'avons vu : `df["nom"]` sélectionne **une** colonne et retourne une `Series`. Pour sélectionner **plusieurs** colonnes, on donne une **liste** de noms (d'où les doubles crochets) ; on obtient alors un `DataFrame` :

```
df[["species", "body_mass_g"]].head(3)
```

```
  species  body_mass_g
0  Adelie       3750.0
1  Adelie       3800.0
2  Adelie       3250.0
```

Erreur fréquente

- `df["species", "island"]` (sans les doubles crochets) provoque une `KeyError` : pandas cherche une colonne dont le nom serait le *tuple* `("species", "island")`.

- Un nom de colonne mal orthographié (majuscules, espaces, accents…) provoque aussi une `KeyError`. En cas de doute, affichez `df.columns`.

11.6 Filtrer des lignes

11.6.1 Avec un masque booléen

Le filtrage des lignes reprend exactement le principe des **masques** vus avec `numpy` : une comparaison appliquée à une colonne produit une `Series` de booléens, que l'on utilise ensuite entre crochets pour ne garder que les lignes où le masque vaut `True`.

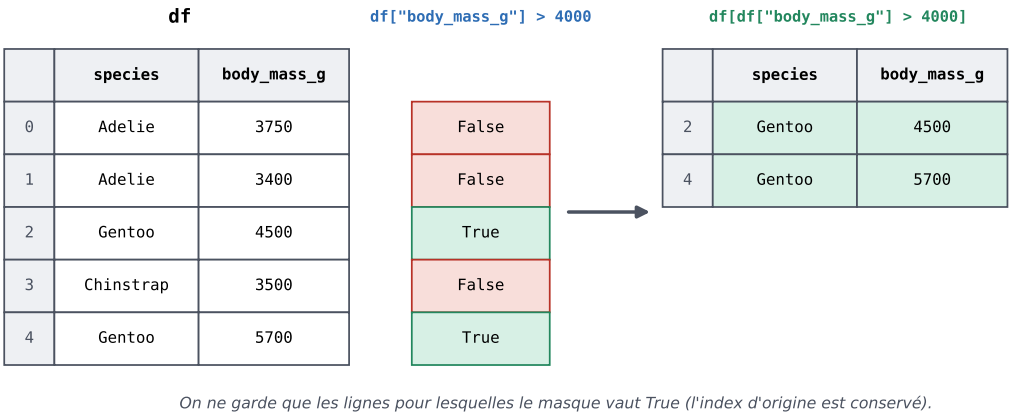


Figure11.2: Filtrer les lignes d'un `DataFrame` avec un masque booléen.

```
masque = df["body_mass_g"] > 5500
print(masque.head())
lourds = df[masque]
print(len(lourds), "manchots de plus de 5,5 kg")
lourds.head()
```

```
0    False
1    False
2    False
3    False
4    False
Name: body_mass_g, dtype: bool
28 manchots de plus de 5,5 kg
```

```
    species  island  bill_length_mm  bill_depth_mm  \
221  Gentoo  Biscoe            50.0           16.3
223  Gentoo  Biscoe            50.0           15.2
231  Gentoo  Biscoe            49.0           16.1
233  Gentoo  Biscoe            48.4           14.6
235  Gentoo  Biscoe            49.3           15.7

     flipper_length_mm  body_mass_g   sex
221              230.0       5700.0  MALE
223              218.0       5700.0  MALE
```

(suite sur la page suivante)

(suite de la page précédente)

```
231              216.0       5550.0  MALE
233              213.0       5850.0  MALE
235              217.0       5850.0  MALE
```

On écrit généralement le filtre directement entre crochets : `df[df["body_mass_g"] > 5500]`. Pour une colonne de texte, on teste l'égalité avec ==, ou l'appartenance à une liste de valeurs avec la méthode `isin` :

```
adelie = df[df["species"] == "Adelie"]
print(adelie.shape)
deux_iles = df[df["island"].isin(["Dream", "Biscoe"])]
print(deux_iles.shape)
```

```
(152, 7)
(292, 7)
```

11.6.2 Combiner des conditions

Pour combiner plusieurs conditions, on utilise **&** (et), **|** (ou) et **~** (non), en mettant **chaque condition entre parenthèses** :

```
femelles_gentoo = df[(df["species"] == "Gentoo") & (df["sex"] ==
↪"FEMALE")]
print(len(femelles_gentoo))
extremes = df[(df["body_mass_g"] < 3000) | (df["body_mass_g"] >␣
↪6000)]
extremes
```

```
58
```

```
       species     island  bill_length_mm  bill_depth_mm  \
47      Adelie      Dream            37.5           18.9
54      Adelie     Biscoe            34.5           18.1
58      Adelie     Biscoe            36.5           16.6
64      Adelie     Biscoe            36.4           17.1
98      Adelie      Dream            33.1           16.1
104     Adelie     Biscoe            37.9           18.6
116     Adelie  Torgersen            38.6           17.0
174  Chinstrap      Dream            43.2           16.6
190  Chinstrap      Dream            46.9           16.6
237     Gentoo     Biscoe            49.2           15.2
253     Gentoo     Biscoe            59.6           17.0

     flipper_length_mm  body_mass_g     sex
47               179.0       2975.0     NaN
54               187.0       2900.0  FEMALE
58               181.0       2850.0  FEMALE
64               184.0       2850.0  FEMALE
98               178.0       2900.0  FEMALE
104              193.0       2925.0  FEMALE
116              188.0       2900.0  FEMALE
174              187.0       2900.0  FEMALE
```

(suite sur la page suivante)

(suite de la page précédente)

```
190              192.0       2700.0  FEMALE
237              221.0       6300.0    MALE
253              230.0       6050.0    MALE
```

À retenir

Avec `pandas` (et `numpy`), on combine des masques avec `&`, `|` et `~`, **et non** avec `and`, `or` et `not` (qui provoquent l'erreur `ValueError: The truth value of a Series is ambiguous`). Les parenthèses autour de chaque condition sont **obligatoires** : `df["a"] > 1 & df["b"] < 2` ne fait pas ce que l'on croit, car `&` est prioritaire sur `>`.

Testez-vous

Écrivez le filtre qui sélectionne :

1. les manchots de l'île Torgersen ;
2. les manchots dont la nageoire mesure au moins 200 mm, sauf ceux de l'espèce Gentoo ;
3. les manchots dont le sexe n'est pas renseigné (indice : la méthode `isna()` d'une `Series` retourne un masque).

11.7 `loc` et `iloc`

Pour sélectionner à la fois des lignes **et** des colonnes, on utilise les accesseurs `loc` et `iloc`, avec la syntaxe `[lignes, colonnes]` :

- `df.loc[...]` sélectionne par **étiquettes** : noms des lignes (l'index) et noms des colonnes ;
- `df.iloc[...]` sélectionne par **positions** : numéros de lignes et de colonnes, comptés à partir de 0, comme pour les listes.

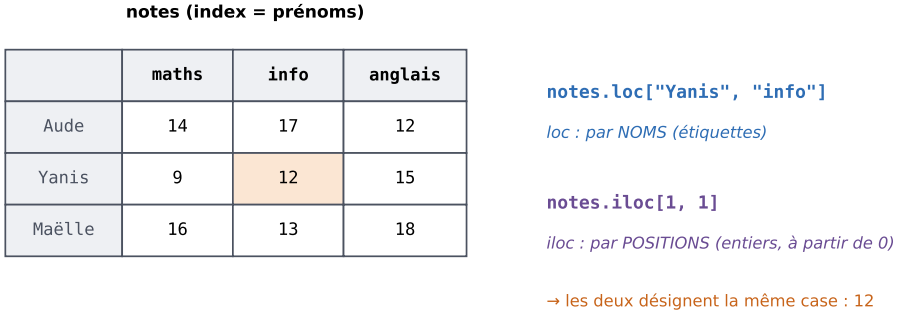


Figure11.3: `loc` utilise les noms, `iloc` les positions.

```
notes = pd.read_csv("donnees/notes.csv", sep=";", index_col=
↪"prenom")
notes
```

prenom	nom	maths	info	anglais
Aude	Martin	14	17	12
Yanis	Benali	9	12	15
Maëlle	Le Goff	16	13	18
Hugo	Dupont	7	10	9
Linh	Nguyen	12	19	14
Erwan	Kerbrat	11	8	13
Fatou	Diallo	15	14	16
Inès	Moreau	10	11	8

```
print(notes.loc["Yanis", "info"])
print(notes.iloc[1, 2])
notes.loc[["Aude", "Inès"], ["maths", "info"]]
```

```
12
12
```

prenom	maths	info
Aude	14	17
Inès	10	11

Ici, la colonne `nom` est la colonne de position 0 : la colonne `info` a donc la position 2 (contrairement au schéma, qui ne contient pas la colonne `nom`).

Comme pour les listes, on peut utiliser des tranches (`debut:fin`), et `:` seul signifie « tout ». Attention : avec `loc`, la borne de fin est **incluse** (puisque l'on travaille avec des noms), alors qu'avec `iloc` elle est exclue :

```
notes.loc["Aude":"Maëlle", "maths":"info"]    # Maëlle et info␣
↪inclus
```

prenom	maths	info
Aude	14	17
Yanis	9	12
Maëlle	16	13

```
notes.iloc[0:3, 1:3]    # lignes 0 à 2, colonnes 1 et 2
```

prenom	maths	info
Aude	14	17
Yanis	9	12
Maëlle	16	13

L'utilisation la plus fréquente de `loc` consiste à combiner un **filtre sur les lignes** et une **sélection de colonnes** :

```
notes.loc[notes["maths"] >= 14, ["nom", "maths"]]
```

```
            nom  maths
prenom
Aude     Martin     14
Maëlle  Le Goff     16
Fatou    Diallo     15
```

11.8 Trier

La méthode `sort_values` trie les lignes selon une ou plusieurs colonnes (ordre croissant par défaut) :

```
df.sort_values("body_mass_g", ascending=False).head(3)
```

```
    species  island  bill_length_mm  bill_depth_mm  \
237  Gentoo  Biscoe            49.2           15.2
253  Gentoo  Biscoe            59.6           17.0
337  Gentoo  Biscoe            48.8           16.2

     flipper_length_mm  body_mass_g   sex
237              221.0       6300.0  MALE
253              230.0       6050.0  MALE
337              222.0       6000.0  MALE
```

```
# Tri selon l'espèce, puis par masse décroissante au sein de␣
↪chaque espèce
df.sort_values(["species", "body_mass_g"], ascending=[True,␣
↪False]).head()
```

```
    species     island  bill_length_mm  bill_depth_mm  \
109  Adelie     Biscoe            43.2           19.0
101  Adelie     Biscoe            41.0           20.0
81   Adelie  Torgersen            42.9           17.6
7    Adelie  Torgersen            39.2           19.6
39   Adelie      Dream            39.8           19.1

     flipper_length_mm  body_mass_g   sex
109              197.0       4775.0  MALE
101              203.0       4725.0  MALE
81               196.0       4700.0  MALE
7                195.0       4675.0  MALE
39               184.0       4650.0  MALE
```

Comme les méthodes des chaînes de caractères, la plupart des méthodes de `pandas` **ne modifient pas** le `DataFrame` : elles en retournent un nouveau. Pour conserver le résultat, il faut l'affecter à une variable : `df_trie = df.sort_values(...)`.

11.9 Exercices

Rappel

Ces exercices sont à faire dans un notebook Jupyter, avec le fichier `donnees/penguins.csv` (sauf mention contraire). Commencez chaque notebook par `import pandas as pd` et `df = pd.read_csv("donnees/penguins.csv")`.

Exercice 11.1 : Premier contact ☆

1. Combien y a-t-il de manchots de chaque sexe ? Combien n'ont pas de sexe renseigné ?
2. Quelle est la longueur moyenne du bec ? La longueur minimale et maximale de la nageoire ?
3. Sur combien d'îles différentes les manchots ont-ils été observés ?

Exercice 11.2 : Filtres ☆

1. Combien de manchots de l'espèce Adelie vivent sur l'île Dream ?
2. Affichez l'espèce, l'île et la masse des 5 manchots les plus légers.
3. Quelle est la masse moyenne des manchots Gentoo ? Et celle des mâles Gentoo ?

Exercice 11.3 : Le fichier de notes ☆ ☆

Chargez le fichier `donnees/notes.csv` dans un `DataFrame` (attention au délimiteur !), puis :

1. affichez les étudiants ayant plus de 15 en informatique ;
2. affichez le nom et le prénom des étudiants ayant au moins 10 dans **les trois** matières ;
3. affichez le tableau trié par note d'anglais décroissante.

Exercice 11.4 : Comparer les espèces ☆ ☆

Sans utiliser de boucle explicite sur les lignes, mais en utilisant une boucle sur les **noms d'espèces** (obtenus avec `unique`), affichez pour chaque espèce son nombre d'individus, sa masse moyenne et la longueur moyenne de son bec, avec une décimale. (Le chapitre suivant présentera un outil, `groupby`, qui fait cela en une ligne.)

Exercice 11.5 : Le Titanic ☆ ☆

Le fichier `donnees/titanic.csv` contient des informations sur 891 passagers du Titanic : `survived` (1 si le passager a survécu, 0 sinon), `pclass` (classe de 1 à 3), `sex`,

`age`, `fare` (prix du billet), `embark_town` (port d'embarquement), etc.

1. Chargez le fichier et explorez-le (`shape`, `info`, `describe`). Quelles colonnes ont des valeurs manquantes ?
2. Combien de passagers ont survécu ? Quelle proportion cela représente-t-il ? (Astuce : quelle est la moyenne d'une colonne qui ne contient que des 0 et des 1 ?)
3. Quel est le taux de survie des femmes ? Des hommes ?
4. Quel était le prix moyen d'un billet en 1re classe ? En 3e classe ?

11.10 Bilan du chapitre

À retenir

- `import pandas as pd;df = pd.read_csv("fichier.csv", sep=";", decimal=",")`.
- Un `DataFrame` est un tableau (lignes = observations, colonnes = variables) ; une colonne `df["col"]` est une `Series`.
- Explorer : `head()`, `shape`, `columns`, `info()`, `describe()`, `value_counts()`,`unique()`,`nunique()`.
- Colonnes : `df["col"]` (une `Series`), `df[["col1", "col2"]]` (un `DataFrame`).
- Filtrer : `df[df["col"] > 10]` ; combiner avec `&`, `|`, `~` et des parenthèses ; `isin([...])`.
- `df.loc[lignes, colonnes]` (par noms, fin incluse), `df.iloc[lignes, colonnes]` (par positions, fin exclue).
- Trier : `df.sort_values("col", ascending=False)`.

Faites le point : êtes-vous capable de...

- charger un fichier CSV, y compris avec un délimiteur `;` ?
- décrire un jeu de données inconnu : taille, colonnes, types, valeurs manquantes, statistiques ?
- compter les effectifs d'une variable qualitative ?
- sélectionner les lignes vérifiant plusieurs conditions ?
- expliquer la différence entre `loc` et `iloc` ?

CHAPTER 12

PRÉPARER ET RÉSUMER SES DONNÉES AVEC `pandas`

Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- créer, transformer, renommer et supprimer des colonnes ;
- repérer et traiter les valeurs manquantes ;
- manipuler des colonnes de texte et de dates ;
- calculer des statistiques par groupe avec `groupby` et des tableaux croisés ;
- fusionner deux tableaux et enregistrer le résultat dans un fichier.

Prérequis : chapitre 11. **Durée indicative :** 2h30.

Dans la réalité, les données sont rarement prêtes à l'emploi : il faut créer de nouvelles variables, corriger des types, gérer des valeurs manquantes… avant de pouvoir calculer des résumés. On dit souvent que cette phase de **préparation** occupe la plus grande partie du temps d'une analyse de données !

```
import pandas as pd
import numpy as np

df = pd.read_csv("donnees/penguins.csv")
```

Si vous venez de R ou de SAS

Opération	R (`dplyr`, `tidyr`)	SAS	`pandas`
Nouvelle colonne	`mutate(df, z = x / y)`	étape DATA : `z = x / y;`	`df["z"] = df["x"] / df["y"]`
Condition	`ifelse(x > 0, "a", "b")`	`IF x > 0 THEN ...; ELSE ...;`	`np.where(df["x"] > 0, "a", "b")`
Recoder	`recode(x, a =`	`PROC FORMAT`	`df["x"].`

Contrairement à SAS (`MERGE ... BY`), il n'est pas nécessaire de trier les tables avant une jointure. Et contrairement à l'opérateur `|>` (ou `%>%`) de R, on enchaîne les opérations `pandas` avec des points : `df.dropna().groupby("g")["x"].mean().round(1)`.

12.1 Créer et modifier des colonnes

12.1.1 Calculs sur des colonnes

Pour créer une nouvelle colonne, il suffit de lui affecter une valeur, comme pour une nouvelle clé de dictionnaire. Les calculs entre colonnes se font **élément par élément**, comme avec `numpy`, sans boucle :

```
df["body_mass_kg"] = df["body_mass_g"] / 1000
df["bill_ratio"] = df["bill_length_mm"] / df["bill_depth_mm"]
df[["body_mass_g", "body_mass_kg", "bill_ratio"]].head(3)
```

```
   body_mass_g  body_mass_kg  bill_ratio
0       3750.0          3.75    2.090909
1       3800.0          3.80    2.270115
2       3250.0          3.25    2.238889
```

Si la colonne existe déjà, son contenu est remplacé.

12.1.2 Recoder des valeurs : `map` et `replace`

Pour remplacer des valeurs par d'autres (traduire des modalités, regrouper des catégories...), on utilise la méthode `map` avec un **dictionnaire** de correspondance :

```
traduction = {"Adelie": "Adélie", "Chinstrap": "à jugulaire",
 ↪"Gentoo": "papou"}
df["espece"] = df["species"].map(traduction)
df[["species", "espece"]].sample(4, random_state=0)
```

```
    species  espece
141  Adelie  Adélie
6    Adelie  Adélie
60   Adelie  Adélie
249  Gentoo   papou
```

Attention : avec `map`, une valeur absente du dictionnaire devient une valeur manquante. Pour ne remplacer que certaines valeurs et laisser les autres intactes, utilisez plutôt `replace` avec le même dictionnaire.

12.1.3 Découper en classes : pd.cut

Pour transformer une variable numérique en catégories (des tranches d'âge, des classes de masse…), on utilise `pd.cut` en précisant les bornes des classes :

```
df["gabarit"] = pd.cut(df["body_mass_g"], bins=[0, 3500, 4500,␣
↪7000],
                       labels=["petit", "moyen", "grand"])
df["gabarit"].value_counts()
```

```
gabarit
moyen    149
grand    115
petit     78
Name: count, dtype: int64
```

Par défaut, chaque intervalle est ouvert à gauche et fermé à droite : une masse de 3500 g est classée « petit ».

12.1.4 Appliquer une fonction : apply

Pour des transformations plus complexes, on peut appliquer à chaque valeur d'une colonne une fonction que l'on a écrite soi-même, avec `apply` :

```
def categorie_bec(longueur):
    if longueur < 40:
        return "court"
    elif longueur < 48:
        return "moyen"
    return "long"

df["bec"] = df["bill_length_mm"].apply(categorie_bec)
df["bec"].value_counts()
```

```
bec
moyen    148
court    100
long      96
Name: count, dtype: int64
```

Bonne pratique

`apply` est très souple mais plus lent que les opérations vectorisées (`+`, `/`, `map`, `pd.cut`…), car il appelle la fonction Python une fois par ligne. Utilisez-le seulement quand il n'existe pas d'outil plus direct.

12.1.5 Renommer et supprimer des colonnes

```
df = df.rename(columns={"bill_length_mm": "bec_long", "bill_depth_
↪mm": "bec_epais"})
df = df.drop(columns=["body_mass_kg", "bill_ratio", "bec"])
df.columns
```

```
Index(['species', 'island', 'bec_long', 'bec_epais',
       'flipper_length_mm', 'body_mass_g', 'sex', 'espece',
       'gabarit'],
      dtype='str')
```

Comme `sort_values`, les méthodes `rename` et `drop` retournent un **nouveau** `DataFrame` : pour conserver le résultat, on le réaffecte à `df`.

12.2 Les valeurs manquantes

Les valeurs manquantes (`NaN`) sont omniprésentes dans les données réelles : capteur en panne, question sans réponse, information inconnue... Il faut toujours commencer par les **compter** :

```
df.isna().sum()
```

```
species               0
island                0
bec_long              2
bec_epais             2
flipper_length_mm     2
body_mass_g           2
sex                  11
espece                0
gabarit               2
dtype: int64
```

`isna()` produit un tableau de booléens (`True` là où la valeur manque), et `sum()` compte les `True` dans chaque colonne.

Plusieurs stratégies sont possibles :

- **supprimer** les lignes concernées, avec `dropna` (éventuellement en ne regardant que certaines colonnes grâce à l'argument `subset`) ;
- **remplacer** les valeurs manquantes par une valeur choisie, avec `fillna` (une valeur fixe, la moyenne ou la médiane de la colonne...).

```
complet = df.dropna(subset=["sex"])    # on retire les 11 manchots␣
↪de sexe inconnu
print(df.shape, complet.shape)

df["sex"] = df["sex"].fillna("inconnu")
df["sex"].value_counts()
```

```
(344, 9) (333, 9)
```

```
sex
MALE       168
FEMALE     165
inconnu     11
Name: count, dtype: int64
```

Attention

Aucune stratégie n'est neutre : supprimer des lignes réduit (et peut biaiser) l'échantillon, et remplacer par la moyenne invente des valeurs. Le choix dépend de la question posée. Dans tous les cas, **notez ce que vous avez fait**, et vérifiez combien de lignes sont concernées.

Testez-vous

Après `df2 = df.dropna()`, combien de lignes reste-t-il si une seule colonne contient des valeurs manquantes, sur 10 lignes ? Et si deux colonnes contiennent chacune 2 valeurs manquantes, sur des lignes différentes ?

12.3 Texte et dates

12.3.1 Les colonnes de texte : l'accesseur `.str`

Les méthodes des chaînes de caractères (`lower`, `upper`, `replace`, `startswith`...) s'appliquent à toute une colonne de texte grâce à l'accesseur `.str` :

```
df["sex"] = df["sex"].str.lower()
print(df["sex"].unique())
print(df["island"].str.upper().head(3))
print(df[df["island"].str.startswith("T")].shape)
```

```
<StringArray>
['male', 'female', 'inconnu']
Length: 3, dtype: str
0    TORGERSEN
1    TORGERSEN
2    TORGERSEN
Name: island, dtype: str
(52, 9)
```

La méthode `.str.contains("motif")` est particulièrement utile pour filtrer les lignes dont un texte contient un mot donné.

12.3.2 Les dates : pd.to_datetime et l'accesseur .dt

Le fichier donnees/flights.csv contient le nombre mensuel de passagers (en milliers) d'une compagnie aérienne entre 1949 et 1960, un grand classique de l'étude des séries temporelles :

```
vols = pd.read_csv("donnees/flights.csv")
vols.head(3)
```

```
   year     month  passengers
0  1949   January         112
1  1949  February         118
2  1949     March         132
```

Pour exploiter les dates, il faut les convertir dans le type date de pandas, avec pd.to_datetime. Ici, on construit d'abord une chaîne "1949-January", puis on indique son format, à l'aide de codes : %Y pour l'année sur 4 chiffres, %B pour le nom du mois (en anglais) :

```
vols["date"] = pd.to_datetime(vols["year"].astype(str) + "-" +␣
 ↪vols["month"], format="%Y-%B")
vols.dtypes
```

```
year                    int64
month                     str
passengers              int64
date           datetime64[us]
dtype: object
```

Formats de date courants

- pd.to_datetime("2024-10-10") : le format international (année-mois-jour) est reconnu automatiquement ;
- pour des dates à la française ("10/10/2024"), précisez le format : pd.to_datetime(colonne, format="%d/%m/%Y") (ou l'argument dayfirst=True) ;
- %d jour, %m mois (numéro), %Y année sur 4 chiffres, %H heure, %M minutes.

Une fois convertie, une colonne de dates offre l'accesseur .dt, qui donne accès à l'année, au mois, au jour de la semaine... et on peut comparer ou soustraire des dates :

```
print(vols["date"].dt.year.head(3))
recents = vols[vols["date"] >= "1960-01-01"]
print(len(recents), "mois en 1960")
print(vols["date"].max() - vols["date"].min())
```

```
0    1949
1    1949
2    1949
Name: date, dtype: int32
```

(suite sur la page suivante)

(suite de la page précédente)

```
12 mois en 1960
4352 days 00:00:00
```

12.4 Résumer par groupes : groupby

12.4.1 Le principe

Une question revient sans cesse en analyse de données : « comment telle variable varie-t-elle **selon** telle catégorie ? » (la masse selon l'espèce, le taux de survie selon la classe, le salaire selon le diplôme...). L'outil pour y répondre est `groupby`, qui suit le schéma « séparer, calculer, combiner » (*split-apply-combine*) :

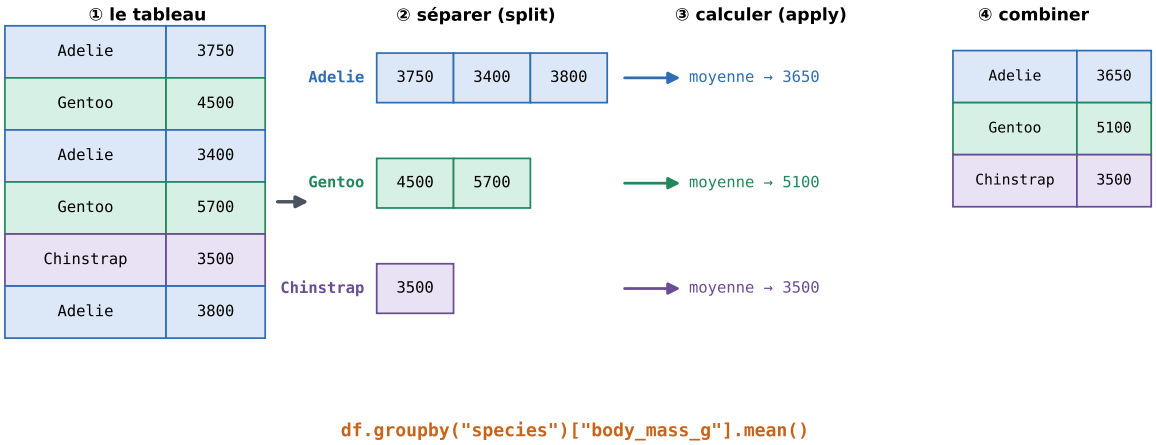


Figure12.1: `groupby` sépare les lignes selon une colonne, calcule un résumé pour chaque groupe, puis rassemble les résultats.

```
df.groupby("species")["body_mass_g"].mean()
```

```
species
Adelie       3700.662252
Chinstrap    3733.088235
Gentoo       5076.016260
Name: body_mass_g, dtype: float64
```

La syntaxe se lit ainsi : « grouper `df` par espèce, prendre la colonne de la masse, et en calculer la moyenne dans chaque groupe ». Toutes les méthodes de résumé vues précédemment fonctionnent : `mean`, `median`, `std`, `min`, `max`, `sum`, `count`... ainsi que `size()`, qui compte le nombre de lignes de chaque groupe.

```
df.groupby("island").size()
```

```
island
Biscoe       168
Dream        124
Torgersen     52
dtype: int64
```

12.4.2 Plusieurs colonnes, plusieurs statistiques

On peut grouper selon **plusieurs** colonnes (en donnant une liste), et résumer plusieurs colonnes à la fois :

```
df.groupby(["species", "sex"])[["body_mass_g", "flipper_length_mm
 ↪"]].mean().round(1)
```

```
                    body_mass_g  flipper_length_mm
species   sex
Adelie    female         3368.8              187.8
          inconnu        3540.0              185.6
          male           4043.5              192.4
Chinstrap female         3527.2              191.7
          male           3939.0              199.9
Gentoo    female         4679.7              212.7
          inconnu        4587.5              215.8
          male           5484.8              221.5
```

La méthode `agg` permet de calculer plusieurs statistiques à la fois :

```
df.groupby("species")["body_mass_g"].agg(["count", "mean", "min",
 ↪"max"])
```

```
           count         mean     min     max
species
Adelie       151  3700.662252  2850.0  4775.0
Chinstrap     68  3733.088235  2700.0  4800.0
Gentoo       123  5076.016260  3950.0  6300.0
```

Le résultat d'un `groupby` est un `DataFrame` (ou une `Series`) dont l'index est formé des groupes. Pour retrouver un tableau « à plat », avec les groupes dans une colonne ordinaire, on ajoute `.reset_index()`.

12.4.3 Tableaux croisés

Pour croiser deux variables qualitatives, `pd.crosstab` construit le tableau des effectifs (ou des proportions, avec l'argument `normalize`) :

```
pd.crosstab(df["species"], df["island"])
```

```
island     Biscoe  Dream  Torgersen
species
Adelie         44     56         52
Chinstrap       0     68          0
Gentoo        124      0          0
```

```
# Proportions par ligne : répartition de chaque espèce entre les␣
 ↪îles
pd.crosstab(df["species"], df["island"], normalize="index").
 ↪round(2)
```

```
island     Biscoe  Dream  Torgersen
species
Adelie       0.29   0.37       0.34
Chinstrap    0.00   1.00       0.00
Gentoo       1.00   0.00       0.00
```

Testez-vous

Que calculent les instructions suivantes ?

1. `df.groupby("island")["body_mass_g"].max()`
2. `df.groupby("species")["island"].nunique()`
3. `df.groupby("sex")["flipper_length_mm"].mean()["male"]`

12.5 Combiner des tableaux

12.5.1 Fusionner : `merge`

Il arrive souvent que les informations soient réparties dans plusieurs tableaux, qui partagent une colonne commune (un identifiant, un nom...). La fonction `pd.merge` les réunit, à la manière d'une jointure en base de données ou d'une RECHERCHEV dans un tableur :

```
infos_especes = pd.DataFrame({
    "species": ["Adelie", "Chinstrap", "Gentoo"],
    "nom_scientifique": ["Pygoscelis adeliae", "Pygoscelis␣
 ↪antarcticus", "Pygoscelis papua"]
})
fusion = pd.merge(df, infos_especes, on="species", how="left")
fusion[["species", "nom_scientifique", "body_mass_g"]].head(3)
```

```
  species    nom_scientifique  body_mass_g
0  Adelie  Pygoscelis adeliae       3750.0
1  Adelie  Pygoscelis adeliae       3800.0
2  Adelie  Pygoscelis adeliae       3250.0
```

L'argument `how="left"` conserve toutes les lignes du premier tableau, même celles qui n'ont pas de correspondance dans le second (elles reçoivent alors des valeurs manquantes).

12.5.2 Empiler : `concat`

Pour mettre bout à bout des tableaux ayant les mêmes colonnes (par exemple des données de plusieurs années stockées dans plusieurs fichiers), on utilise `pd.concat` :

```
debut = vols[vols["year"] == 1949]
fin = vols[vols["year"] == 1960]
pd.concat([debut, fin]).shape
```

```
(24, 4)
```

12.6 Enregistrer le résultat

Enfin, on enregistre un `DataFrame` dans un fichier CSV avec la méthode `to_csv` (l'argument `index=False` évite d'écrire l'index comme une colonne supplémentaire) :

```
resume = df.groupby("species")["body_mass_g"].mean().round(1).
↪reset_index()
resume.to_csv("resume_especes.csv", index=False)
print(open("resume_especes.csv", encoding="utf-8").read())
```

```
species,body_mass_g
Adelie,3700.7
Chinstrap,3733.1
Gentoo,5076.0
```

Pour un fichier destiné à un tableur réglé en français, on peut utiliser `sep=";"` et `decimal=","`. La méthode `to_excel("fichier.xlsx", index=False)` produit directement un fichier Excel (elle nécessite le paquet `openpyxl`) ; symétriquement, `pd.read_excel` lit un fichier Excel.

Erreurs fréquentes de ce chapitre

Symptôme / message	Cause probable
La colonne créée ou modifiée « disparaît »	le résultat de `rename`, `drop`, `fillna`, `sort_values`... n'a pas été réaffecté (`df = df.drop(...)`)
Une colonne entière de `NaN` après `map`	les valeurs de la colonne ne correspondent pas exactement aux clés du dictionnaire (majuscules, espaces...)
`TypeError: unsupported operand type(s)` lors d'un calcul	la colonne est lue comme du texte (`object` ou `str`) : vérifier `df.dtypes`, convertir avec `pd.to_numeric(df["col"], errors="coerce")`
`ValueError: time data ... doesn't match format`	le format passé à `pd.to_datetime` ne correspond pas aux données
Une moyenne de groupe inattendue	des valeurs manquantes, des doublons ou des modalités mal orthographiées (`"Male"` / `"male"`) : vérifier avec `value_counts()`

12.7 Exercices

Rappel

Ces exercices utilisent le fichier `donnees/titanic.csv` (voir l'exercice 11.5 pour la description des colonnes). Commencez votre notebook par `import pandas as pd` et `titanic = pd.read_csv("donnees/titanic.csv")`.

Exercice 12.1 : Survie par groupe ☆

1. Calculez le taux de survie selon le sexe, puis selon la classe (`class`), avec `groupby`.
2. Calculez le taux de survie selon le sexe **et** la classe. Quel groupe a le taux de survie le plus élevé ? Le plus faible ?
3. Construisez le tableau croisé des effectifs du sexe et de la classe.

Exercice 12.2 : Les âges manquants ☆ ☆

1. Combien de passagers n'ont pas d'âge renseigné ? Quelle proportion cela représente-t-il ?
2. Calculez l'âge médian des passagers selon leur classe.
3. Créez une colonne `age_complete` dans laquelle les âges manquants sont remplacés par l'âge médian **de la classe du passager**.

Exercice 12.3 : Enfants et adultes ☆ ☆

1. Avec `pd.cut`, créez une colonne `tranche_age` découpant l'âge en quatre classes : « enfant » (de 0 à 12 ans), « adolescent » (12 à 18 ans), « adulte » (18 à 60 ans) et « senior » (plus de 60 ans).
2. Calculez le nombre de passagers et le taux de survie de chaque tranche d'âge.
3. Le principe « les femmes et les enfants d'abord » semble-t-il avoir été appliqué ?

Exercice 12.4 : Rapport par port d'embarquement ☆ ☆

Produisez un tableau qui donne, pour chaque port d'embarquement (`embark_town`), le nombre de passagers, l'âge moyen, le prix moyen du billet et le taux de survie (arrondis à 2 décimales), trié par nombre de passagers décroissant, puis enregistrez-le dans un fichier `rapport_ports.csv` lisible par un tableur français.

Exercice 12.5 : Saisonnalité du trafic aérien ☆ ☆

Avec le fichier `donnees/flights.csv` :

1. calculez le nombre total de passagers par année ;
2. calculez le nombre moyen de passagers pour chaque mois de l'année (toutes années confondues). Quel est le mois le plus chargé ? Le moins chargé ?
3. de combien de pourcents le trafic annuel a-t-il augmenté entre 1949 et 1960 ?

12.8 Bilan du chapitre

À retenir

- Nouvelle colonne : `df["c"] = df["a"] / df["b"]` ; recodage : `map(dico)`, `replace(dico)` ; classes : `pd.cut(col, bins=[...], labels=[...])` ; fonction : `apply(f)`.
- `rename(columns={...})`, `drop(columns=[...])` : penser à réaffecter le résultat.
- Valeurs manquantes : `isna().sum()`, `dropna(subset=[...])`, `fillna(valeur)`.
- Texte : `df["col"].str.lower()`, `.str.contains(...)` ; dates : `pd.to_datetime(..., format=...)`, `.dt.year`.
- Par groupe : `df.groupby("cat")["num"].mean()`, `.size()`, `.agg([...])` ; `pd.crosstab(a, b)`.
- Combiner : `pd.merge(df1, df2, on="cle", how="left")`, `pd.concat([df1, df2])` ; enregistrer : `df.to_csv("f.csv", index=False)`.

Faites le point : êtes-vous capable de...

- créer une variable calculée ou une variable en classes ?
- compter et traiter les valeurs manquantes d'un tableau ?
- convertir une colonne en dates et en extraire l'année ou le mois ?
- calculer une moyenne ou un effectif par groupe, selon une ou deux variables ?
- fusionner deux tableaux qui partagent une colonne ?
- enregistrer un tableau de résultats dans un fichier CSV ?

CHAPTER 13

VISUALISER SES DONNÉES AVEC `MATPLOTLIB` ET `SEABORN`

Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- choisir un type de graphique adapté à la question posée ;
- tracer des courbes, nuages de points, histogrammes et diagrammes en barres avec `matplotlib` ;
- soigner un graphique (titre, axes, légende) et l'enregistrer dans un fichier ;
- produire rapidement des graphiques statistiques à partir d'un `DataFrame` avec `seaborn`.

Prérequis : chapitres 11 et 12. **Durée indicative :** 2h.

Un graphique bien choisi permet de comprendre en un coup d'œil ce que des tableaux de chiffres cachent : la forme d'une distribution, une différence entre groupes, une tendance, une valeur aberrante... En Python, la bibliothèque de base pour tracer des graphiques est `matplotlib`. La bibliothèque `seaborn`, construite sur `matplotlib`, est spécialisée dans les graphiques statistiques et s'utilise très simplement avec des `DataFrame`.

Installation

`matplotlib` et `seaborn` sont inclus dans Anaconda et dans Google Colab. Sinon : `pip install matplotlib seaborn`.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

(suite sur la page suivante)

(suite de la page précédente)

```
df = pd.read_csv("donnees/penguins.csv")
vols = pd.read_csv("donnees/flights.csv")
```

Par convention, on importe le module `matplotlib.pyplot` sous le nom `plt` et `seaborn` sous le nom `sns`.

13.1 Quel graphique pour quelle question ?

Avant de tracer un graphique, demandez-vous **quelle question il doit éclairer**. Le type de graphique en découle, selon la nature des variables (numériques ou qualitatives) :

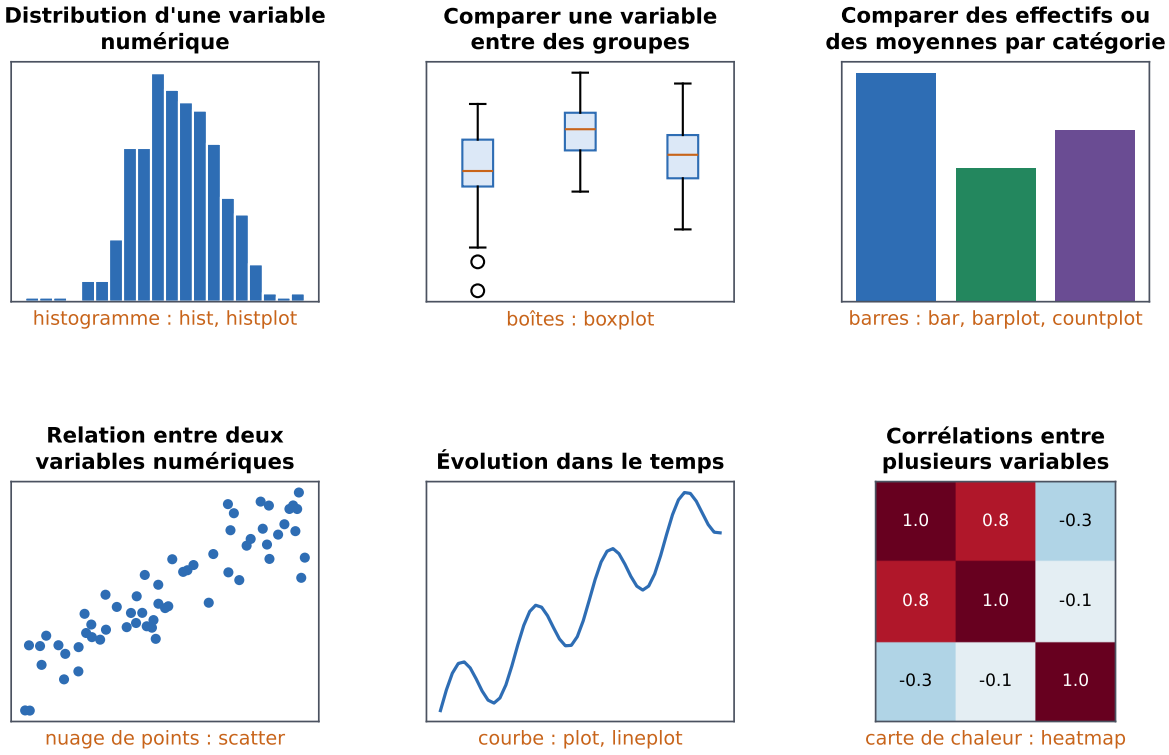


Figure13.1: Les principaux types de graphiques et la question à laquelle chacun répond (en orange, les fonctions `matplotlib` / `seaborn` correspondantes).

13.2 Les bases de `matplotlib`

13.2.1 Un premier graphique

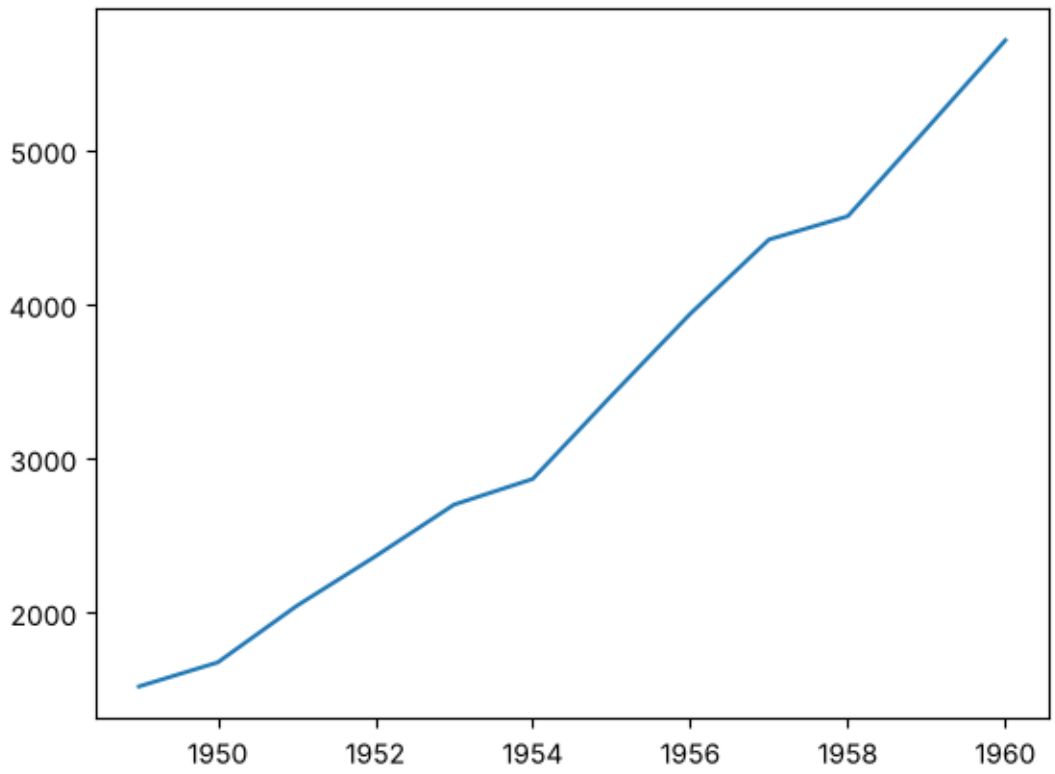
Traçons l'évolution du nombre de passagers aériens. La fonction `plt.plot(x, y)` trace une courbe reliant les points de coordonnées `(x[i], y[i])` :

```
par_an = vols.groupby("year")["passengers"].sum()
```

(suite sur la page suivante)

(suite de la page précédente)

```
plt.plot(par_an.index, par_an.values)
plt.show()
```



`plt.show()` affiche le graphique (dans un notebook, il s'affiche aussi automatiquement à la fin de la cellule).

13.2.2 Soigner son graphique

Un graphique sans titre ni nom d'axes est difficile à lire. La façon recommandée de construire un graphique avec `matplotlib` consiste à créer une **figure** (`fig`) contenant un **système d'axes** (`ax`), puis à appeler les méthodes de `ax` pour tracer et annoter :

```
fig, ax = plt.subplots(figsize=(7, 4))    # largeur et hauteur en␣
↪pouces
ax.plot(par_an.index, par_an.values, marker="o", label="passagers
↪")
ax.set_title("Nombre de passagers par an")
ax.set_xlabel("Année")
ax.set_ylabel("Passagers (milliers)")
ax.legend()
ax.grid(alpha=0.3)
plt.show()
```

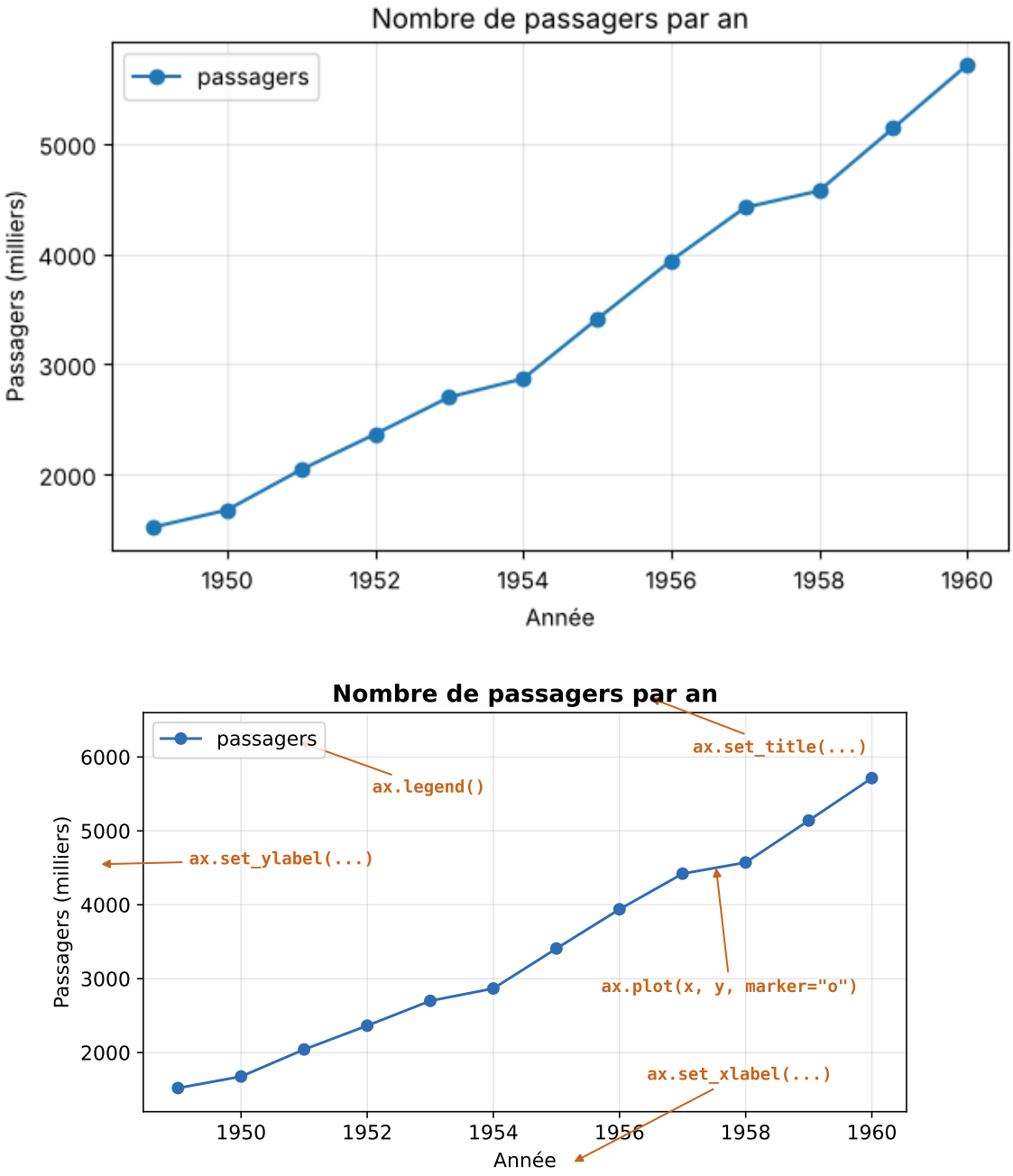


Figure13.2: Les éléments d'un graphique et les méthodes qui les définissent.

À retenir

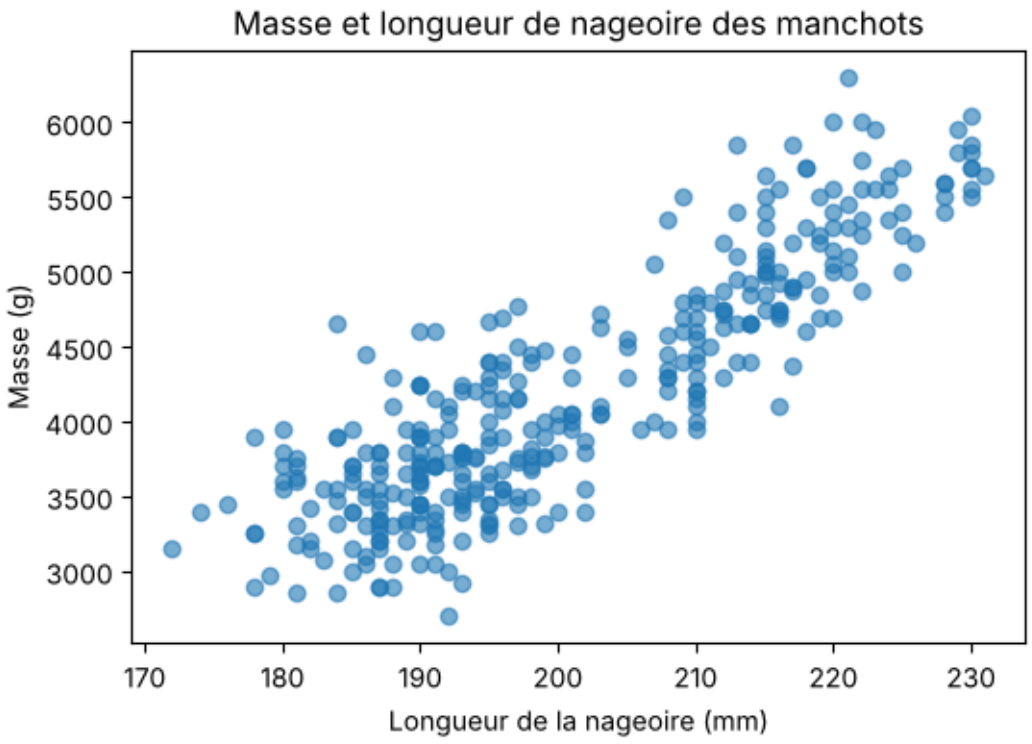
Un bon graphique a toujours **un titre, des noms d'axes avec leurs unités**, et une **légende** s'il contient plusieurs séries. Posez-vous la question : « quelqu'un qui n'a pas vu mon code comprendrait-il ce graphique ? »

13.2.3 Les principaux types de graphiques

Voici les fonctions de base, à appeler sur un système d'axes `ax` :

Méthode	Graphique	Usage
`ax.plot(x, y)`	courbe	évolution (dans le temps...)
`ax.scatter(x, y)`	nuage de points	relation entre deux variables numériques
`ax.hist(valeurs, bins=20)`	histogramme	distribution d'une variable numérique
`ax.bar(categories, hauteurs)`	diagramme en barres	comparer des valeurs entre catégories
`ax.boxplot([serie1, serie2])`	boîtes à moustaches	comparer des distributions

```
fig, ax = plt.subplots(figsize=(6, 4))
ax.scatter(df["flipper_length_mm"], df["body_mass_g"], alpha=0.6)
ax.set_xlabel("Longueur de la nageoire (mm)")
ax.set_ylabel("Masse (g)")
ax.set_title("Masse et longueur de nageoire des manchots")
plt.show()
```



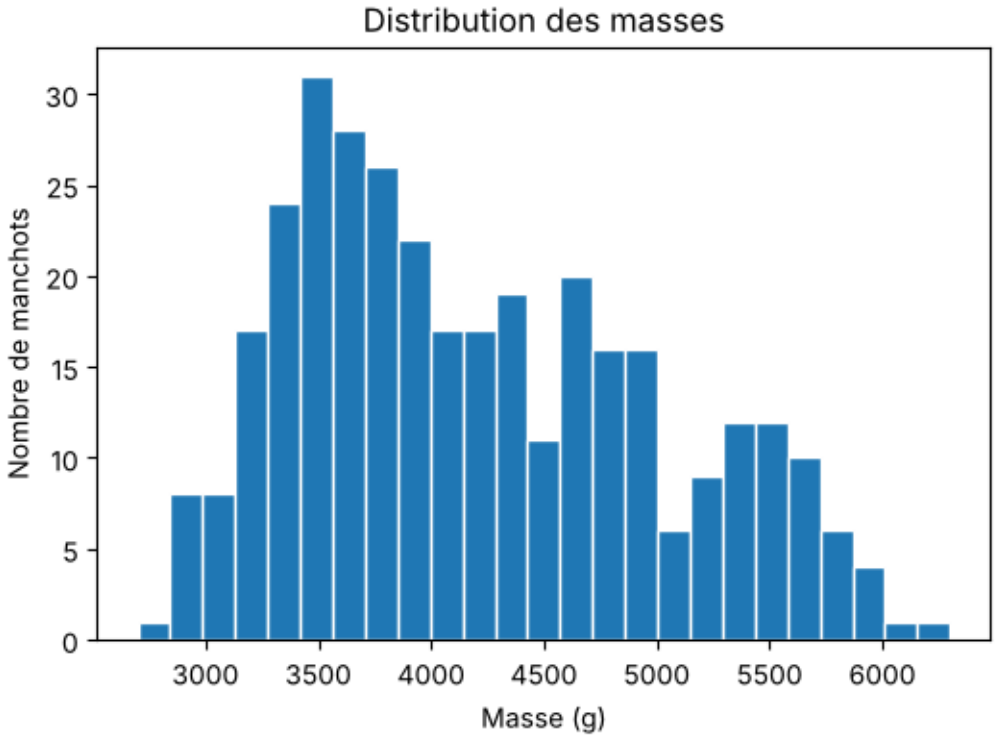
L'argument `alpha` (entre 0 et 1) rend les points semi-transparents, ce qui permet de voir les zones où ils se superposent.

```
fig, ax = plt.subplots(figsize=(6, 4))
ax.hist(df["body_mass_g"].dropna(), bins=25, color="tab:blue",␣
↪edgecolor="white")
ax.set_xlabel("Masse (g)")
```

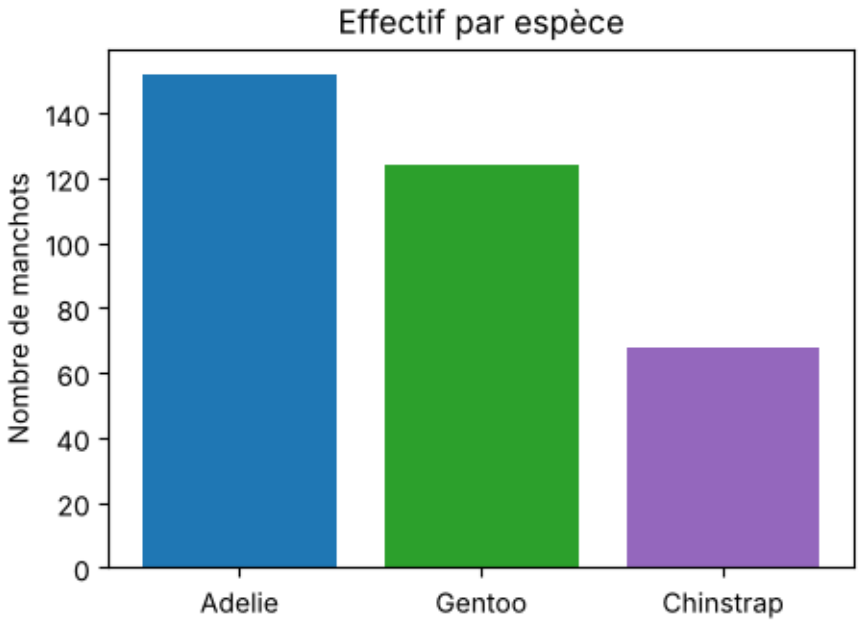
(suite sur la page suivante)

(suite de la page précédente)

```
ax.set_ylabel("Nombre de manchots")
ax.set_title("Distribution des masses")
plt.show()
```



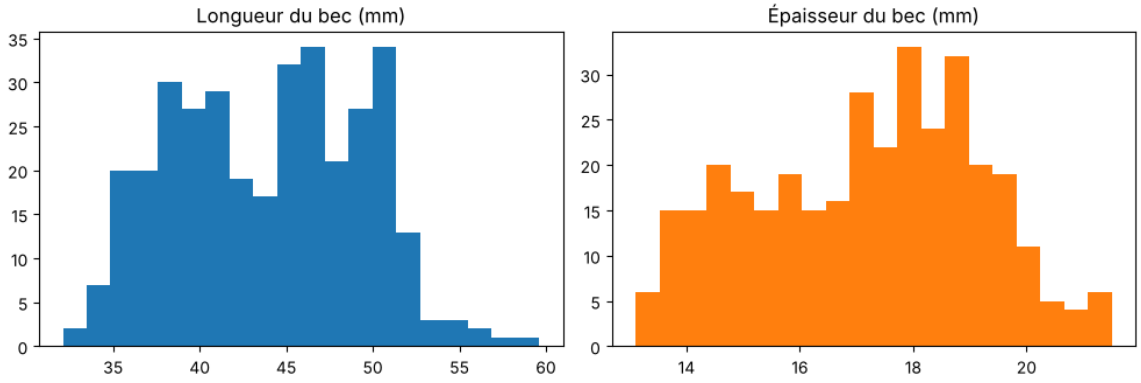

```
effectifs = df["species"].value_counts()
fig, ax = plt.subplots(figsize=(5, 3.5))
ax.bar(effectifs.index, effectifs.values, color=["tab:blue",
 ↪"tab:green", "tab:purple"])
ax.set_ylabel("Nombre de manchots")
ax.set_title("Effectif par espèce")
plt.show()
```



13.2.4 Plusieurs graphiques côte à côte

`plt.subplots(nb_lignes, nb_colonnes)` crée une grille de systèmes d'axes ; `axes` est alors un tableau `numpy`, dont on sélectionne chaque élément avec des indices :

```
fig, axes = plt.subplots(1, 2, figsize=(10, 3.5))
axes[0].hist(df["bill_length_mm"].dropna(), bins=20)
axes[0].set_title("Longueur du bec (mm)")
axes[1].hist(df["bill_depth_mm"].dropna(), bins=20, color=
 ↪"tab:orange")
axes[1].set_title("Épaisseur du bec (mm)")
fig.tight_layout()    # ajuste les espacements
plt.show()
```



13.2.5 Enregistrer un graphique

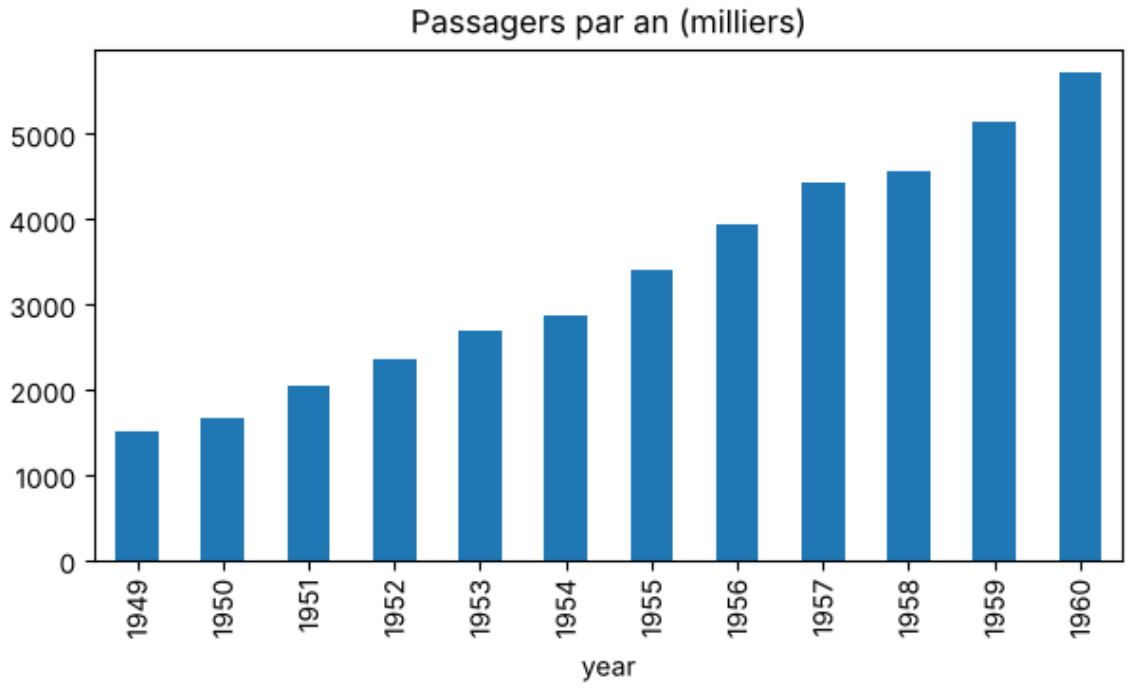
Pour enregistrer un graphique dans un fichier (pour l'insérer dans un rapport ou une présentation), on utilise `fig.savefig` **avant** `plt.show()` ; le format est déduit de l'extension :

```
fig.savefig("masses.png", dpi=150)    # image (dpi = résolution)
fig.savefig("masses.pdf")             # format vectoriel, idéal pour l
 ↪'impression
```

13.2.6 Le raccourci de pandas

Les `Series` et `DataFrame` disposent d'une méthode `plot` qui appelle `matplotlib` pour vous : pratique pour un coup d'œil rapide pendant l'exploration.

```
ax = par_an.plot(kind="bar", figsize=(7, 3.5), title="Passagers␣
 ↪par an (milliers)")
plt.show()
```



L'argument `kind` peut valoir `"line"` (par défaut), `"bar"`, `"barh"`, `"hist"`, `"box"`, `"scatter"`...

Testez-vous

Quel type de graphique utiliseriez-vous pour répondre à chacune de ces questions ?

1. Les mâles sont-ils plus lourds que les femelles ?
2. Comment le nombre de passagers évolue-t-il au cours d'une année ?
3. Quelle est la répartition des passagers du Titanic entre les trois classes ?

4. Les manchots qui ont un long bec ont-ils aussi une longue nageoire ?

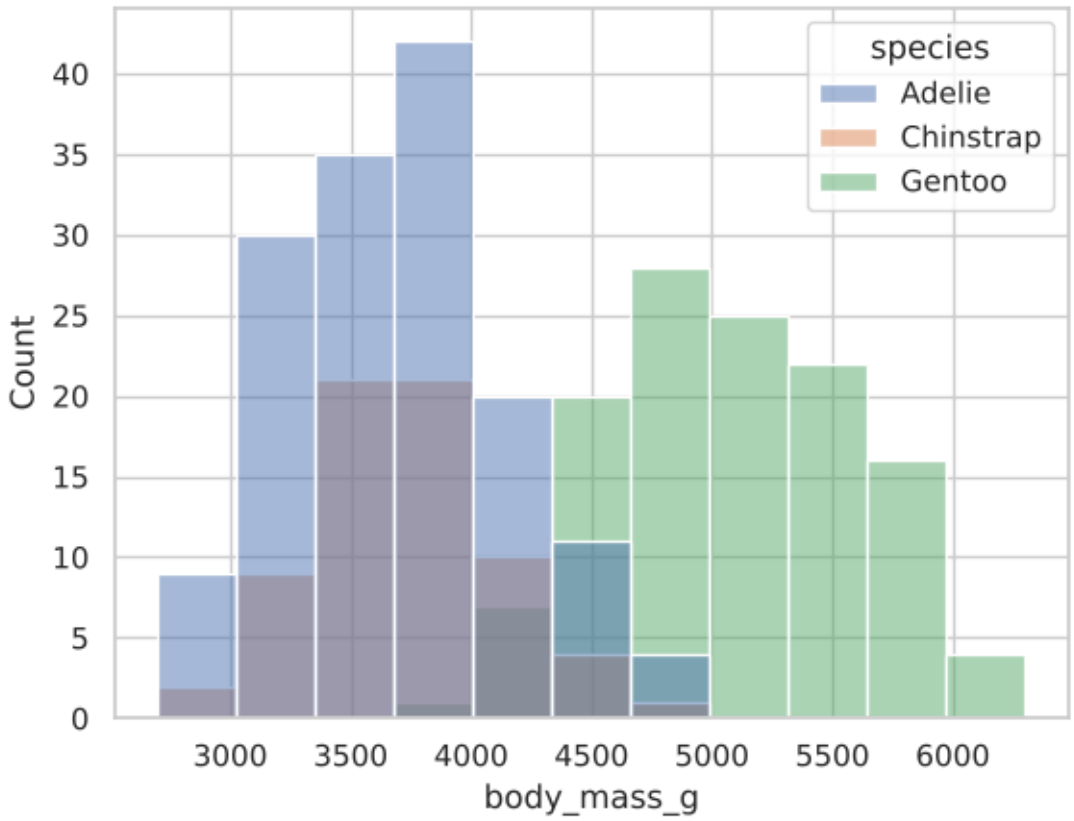
13.3 Des graphiques statistiques avec seaborn

`seaborn` simplifie énormément la création de graphiques statistiques : on lui donne le `DataFrame` (argument `data`) et les **noms des colonnes** à représenter (`x`, `y`, et souvent `hue` pour colorer selon une catégorie). Elle se charge de la légende, des couleurs, des noms d'axes, et des calculs statistiques (moyennes, intervalles de confiance, densités...).

```
sns.set_theme(style="whitegrid")    # un style plus agréable pour␣
↪tous les graphiques suivants
```

13.3.1 Distribution d'une variable : histplot

```
sns.histplot(data=df, x="body_mass_g", hue="species")
plt.show()
```

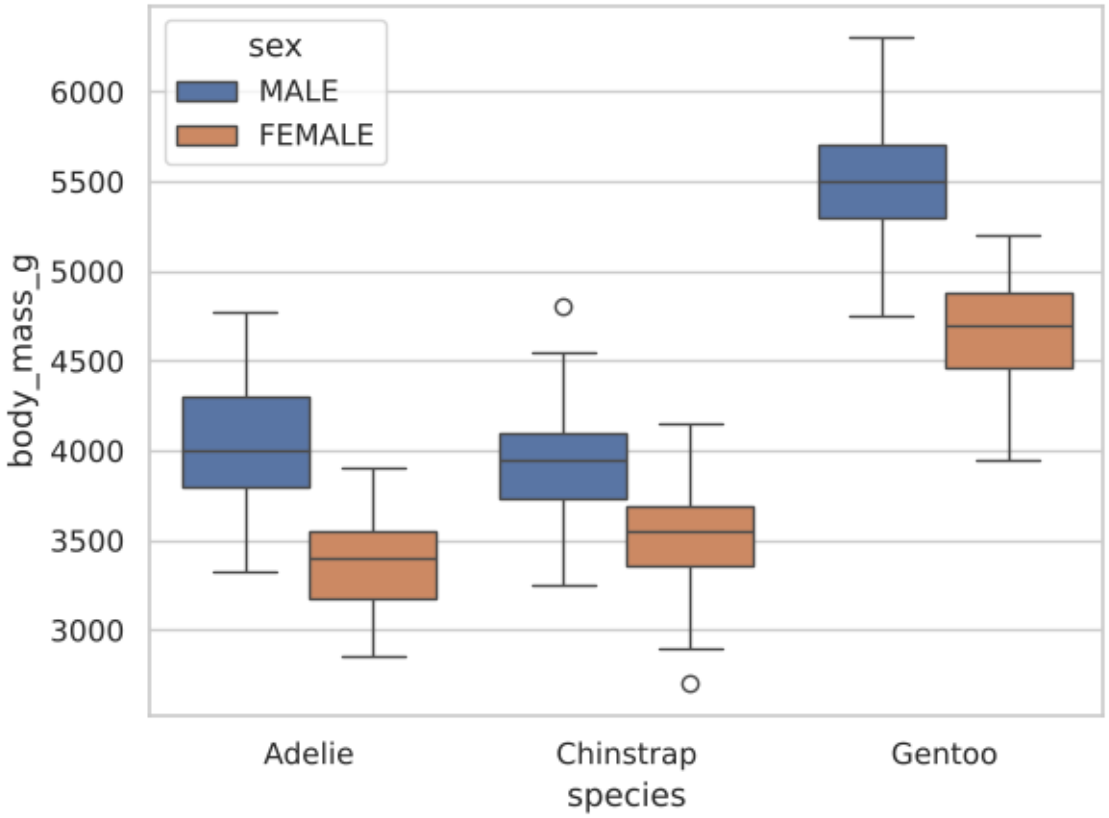


En une ligne, on obtient trois histogrammes superposés, un par espèce, avec la légende. L'argument `kde=True` ajoute une estimation lisse de la densité, et `multiple="stack"` empile les histogrammes au lieu de les superposer.

13.3.2 Comparer des groupes : boxplot

La **boîte à moustaches** résume une distribution : la boîte va du premier au troisième quartile (elle contient la moitié centrale des observations), le trait central est la médiane, et les points isolés sont des valeurs éloignées des autres.

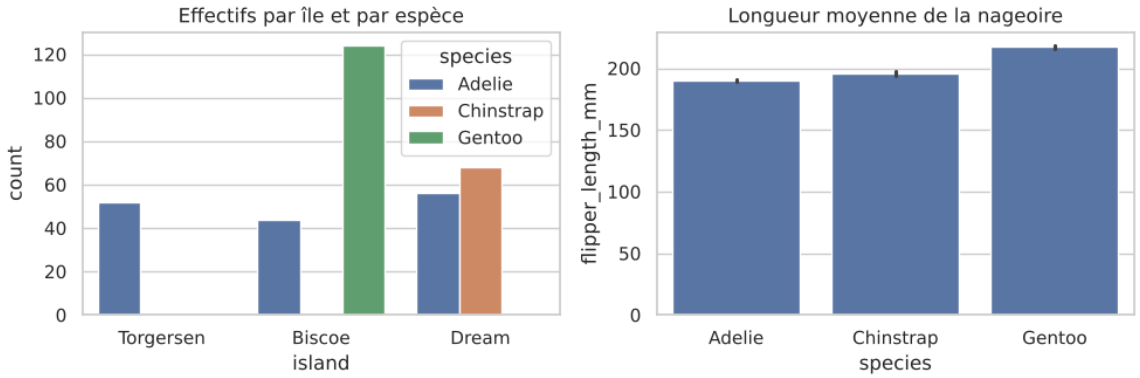
```
sns.boxplot(data=df, x="species", y="body_mass_g", hue="sex")
plt.show()
```



On voit immédiatement que les Gentoo sont plus lourds que les deux autres espèces, et que, dans chaque espèce, les mâles sont plus lourds que les femelles. `sns.violinplot`, avec les mêmes arguments, donne une représentation plus détaillée de la forme des distributions.

13.3.3 Moyennes et effectifs : barplot et countplot

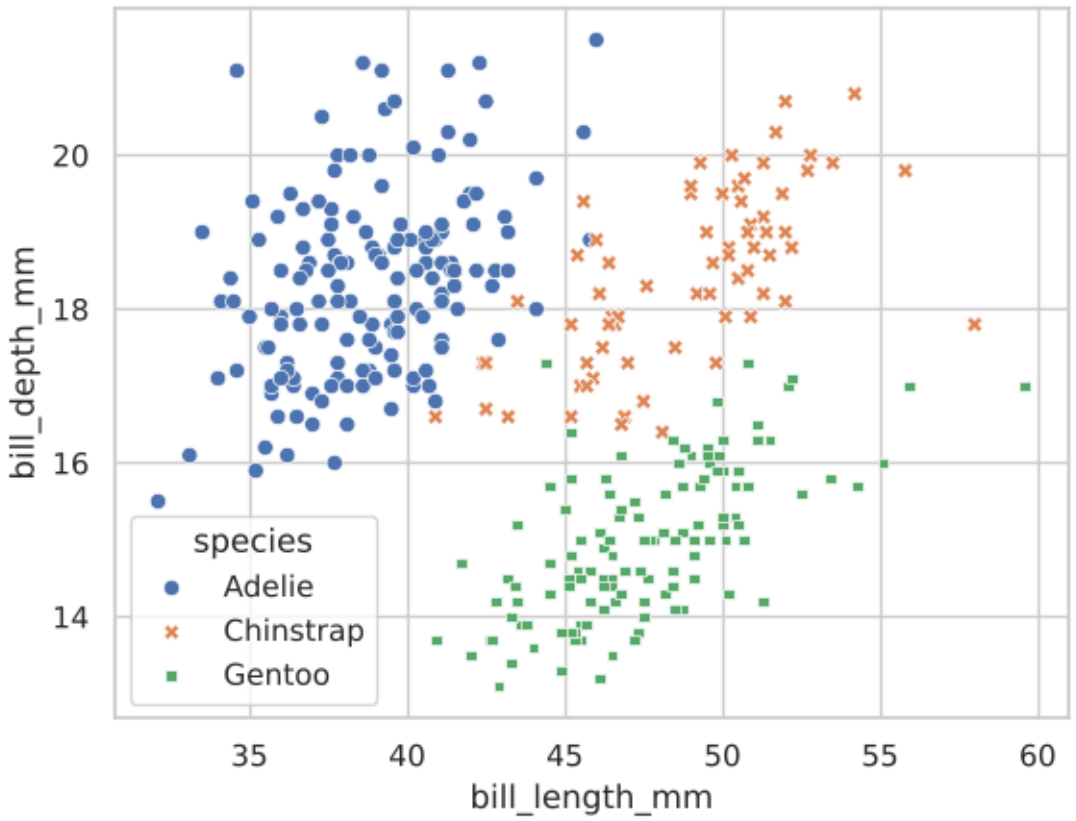
```
fig, axes = plt.subplots(1, 2, figsize=(10, 3.5))
sns.countplot(data=df, x="island", hue="species", ax=axes[0])
axes[0].set_title("Effectifs par île et par espèce")
sns.barplot(data=df, x="species", y="flipper_length_mm",␣
↪ax=axes[1])
axes[1].set_title("Longueur moyenne de la nageoire")
fig.tight_layout()
plt.show()
```



`countplot` compte les lignes de chaque catégorie ; `barplot` représente la **moyenne** de `y` pour chaque catégorie, avec un trait vertical qui indique l'incertitude sur cette moyenne (un intervalle de confiance à 95 %). Remarquez l'argument `ax=...`, qui permet de placer un graphique `seaborn` dans un système d'axes créé avec `plt.subplots`.

13.3.4 Relation entre deux variables : `scatterplot` et `regplot`

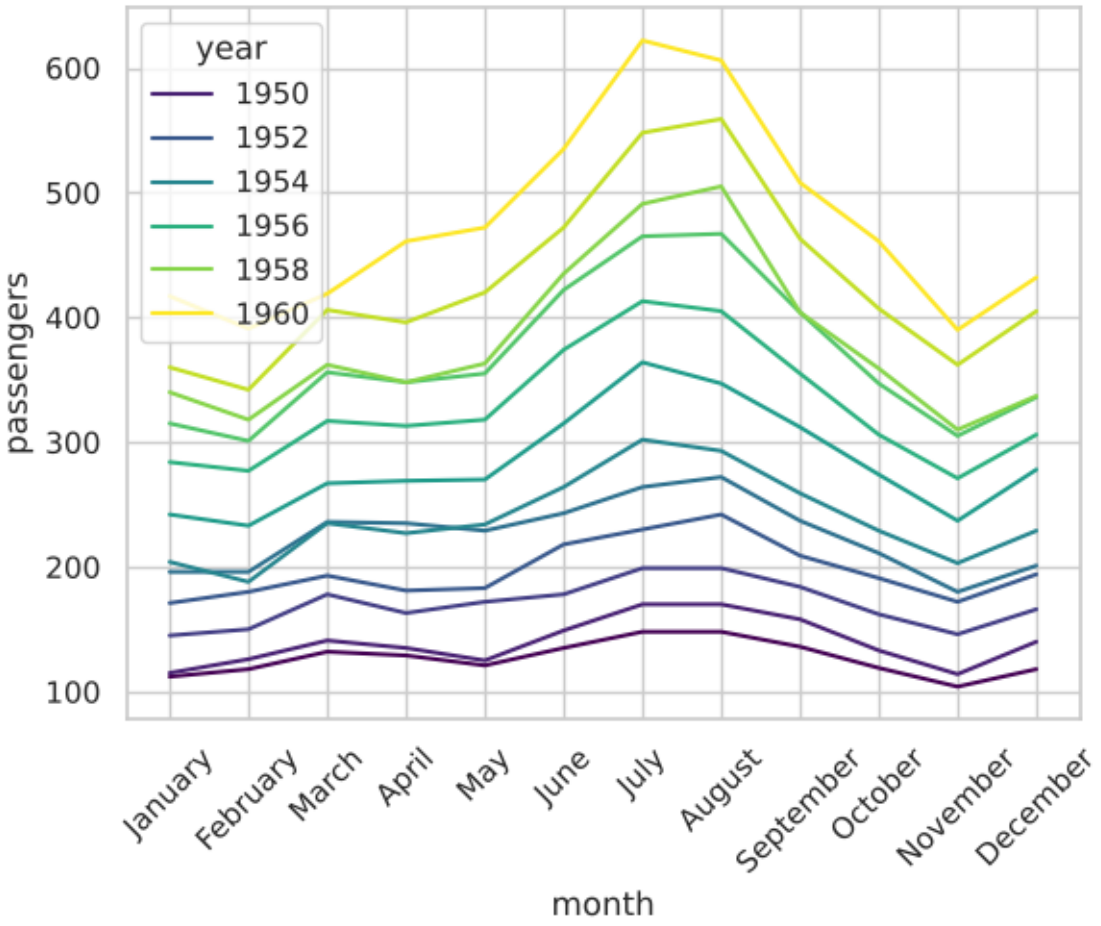
```
sns.scatterplot(data=df, x="bill_length_mm", y="bill_depth_mm",␣
↪hue="species", style="species")
plt.show()
```



Ce graphique fait apparaître trois groupes bien distincts : la forme du bec permet presque de reconnaître l'espèce ! `sns.regplot(data=..., x=..., y=...)` ajoute au nuage de points une droite de régression.

13.3.5 Évolution : lineplot

```
sns.lineplot(data=vols, x="month", y="passengers", hue="year",␣
↪palette="viridis")
plt.xticks(rotation=45)
plt.show()
```

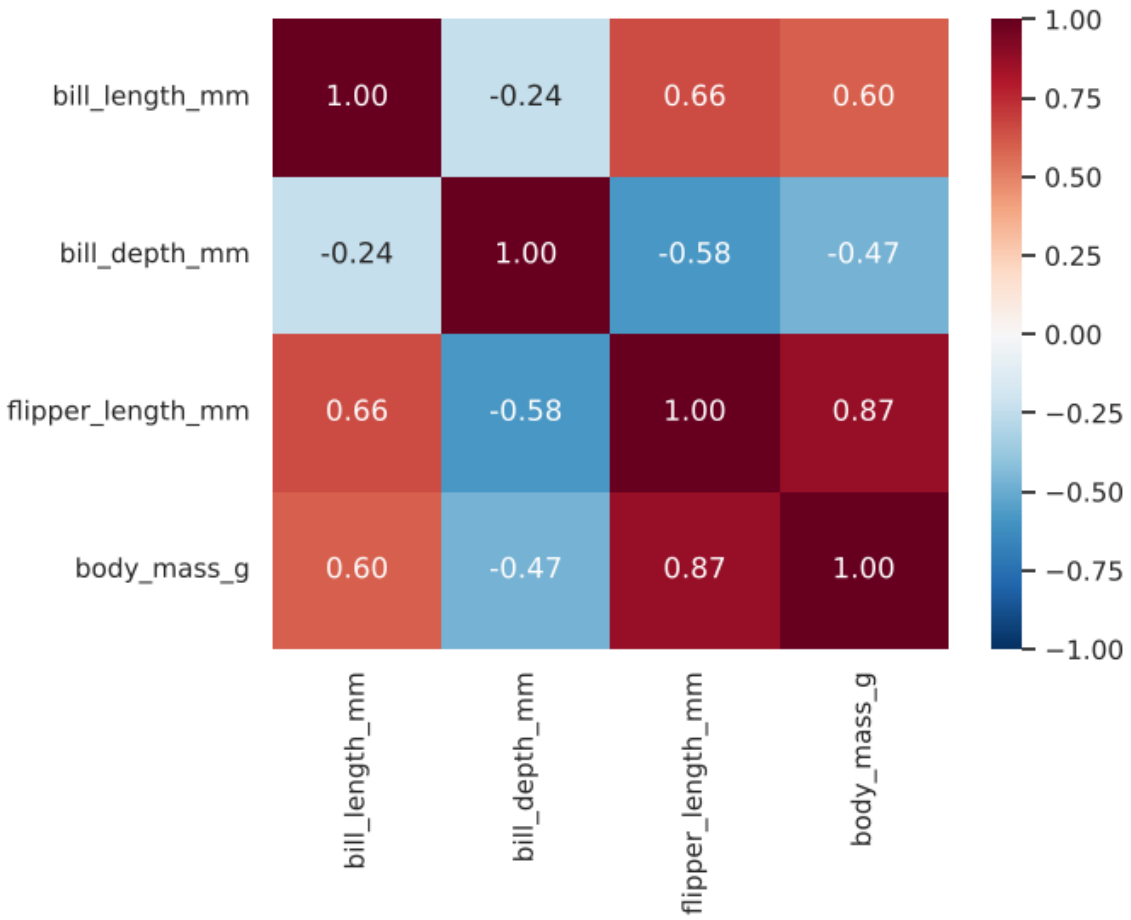


On distingue à la fois la tendance (chaque année, la courbe est plus haute que la précédente) et la saisonnalité (un pic de trafic chaque été).

13.3.6 Corrélations : heatmap

La matrice des corrélations mesure, pour chaque paire de variables numériques, la force de leur lien linéaire (entre -1 et 1). Une carte de chaleur la rend lisible :

```
correlations = df.select_dtypes("number").corr()
sns.heatmap(correlations, annot=True, fmt=".2f", cmap="RdBu_r",␣
↪vmin=-1, vmax=1)
plt.show()
```

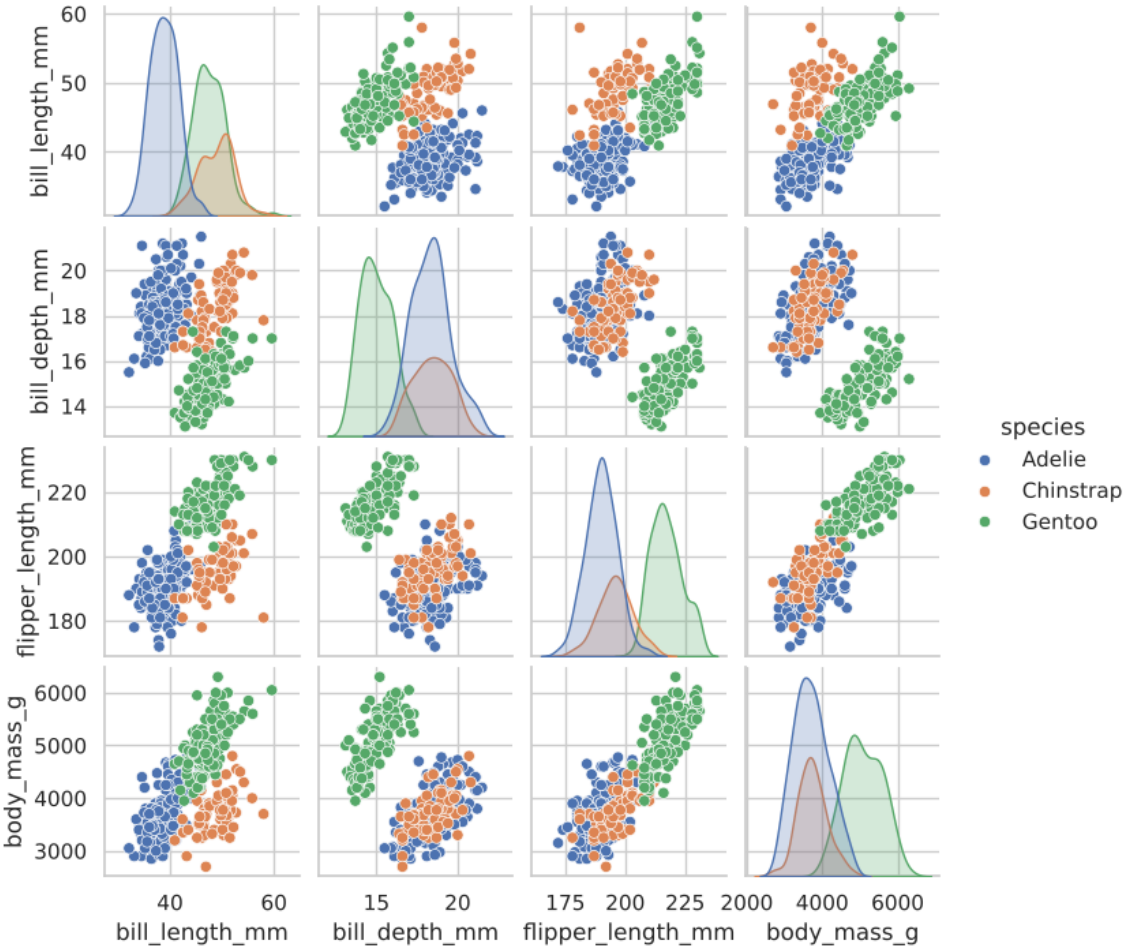


La masse est fortement corrélée à la longueur de la nageoire (0,87) : les manchots aux longues nageoires sont en général plus lourds.

13.3.7 Vue d'ensemble : `pairplot`

Enfin, `pairplot` trace d'un coup les nuages de points de toutes les paires de variables numériques, et la distribution de chacune sur la diagonale. C'est un excellent point de départ pour explorer un jeu de données :

```
sns.pairplot(data=df, hue="species", height=1.8)
plt.show()
```



Si vous venez de R ou de SAS

- La logique de `seaborn` est proche de celle de `ggplot2` : on donne un tableau de données (`data=df`) et on associe des colonnes à des rôles visuels (`x=`, `y=`, `hue=` pour la couleur, `style=` pour la forme), comme avec `aes(x, y, colour = ...)`.
- Pour les « facettes » (`facet_wrap` en R), utilisez les fonctions de niveau figure de `seaborn`, comme `sns.relplot`, `sns.displot` ou `sns.catplot`, avec les arguments `col=` et `row=` : par exemple `sns.displot(data=df, x="body_mass_g", col="species")`.
- Si vous tenez à la grammaire de `ggplot2`, le paquet Python `plotnine` la reproduit presque à l'identique (`ggplot(df, aes("x", "y")) + geom_point()`).
- En SAS, `PROC SGPLOT` (`HISTOGRAM`, `VBOX`, `SCATTER`, `SERIES`, `VBAR`) correspond aux fonctions `histplot`, `boxplot`, `scatterplot`, `lineplot` et `barplot`.

Personnaliser un graphique seaborn

Les fonctions `seaborn` comme `histplot`, `boxplot` ou `scatterplot` renvoient un système d'axes `matplotlib` : on peut donc le personnaliser avec les méthodes vues plus

haut.

```
ax = sns.boxplot(data=df, x="species", y="body_mass_g")
ax.set_title("Masse selon l'espèce")
ax.set_xlabel("Espèce")
ax.set_ylabel("Masse (g)")
```

Erreurs fréquentes de ce chapitre

Symptôme / message	Cause probable
`ValueError: Could not interpret value 'Species' for 'x'`	le nom de colonne passé à `seaborn` ne correspond pas exactement (majuscules...)
Le fichier enregistré est vide (tout blanc)	`savefig` appelé **après** `plt.show()`
Plusieurs graphiques se superposent dans la même figure	dans un script, il manque `plt.show()` (ou `fig, ax = plt.subplots()`) entre deux graphiques
`ValueError: x and y must have same first dimension`	`plt.plot(x, y)` avec des listes / colonnes de tailles différentes
Les étiquettes de l'axe des x se chevauchent	ajouter `plt.xticks(rotation=45)` ou utiliser un graphique horizontal

13.4 Exercices

Rappel

Ces exercices sont à faire dans un notebook, avec `pandas`, `matplotlib.pyplot` (`plt`) et `seaborn` (`sns`) importés.

Exercice 13.1 : Courbe soignée ☆

Avec `matplotlib`, tracez l'évolution mensuelle du nombre de passagers (`donnees/flights.csv`) sur toute la période, en utilisant une colonne de dates (voir le chapitre 12). Ajoutez un titre et des noms d'axes, puis enregistrez le graphique dans un fichier `passagers.png`.

Exercice 13.2 : Explorer les manchots ☆

Avec `seaborn` :

1. tracez l'histogramme de la longueur des nageoires, coloré par espèce ;
2. tracez des boîtes à moustaches de la longueur du bec selon l'île ;

3. tracez le nuage de points de la masse en fonction de la longueur de la nageoire, coloré par sexe.

Pour chaque graphique, écrivez en une phrase ce que vous en concluez.

Exercice 13.3 : Survie sur le Titanic ☆ ☆

Avec le fichier `donnees/titanic.csv`, produisez une figure contenant deux graphiques côte à côte :

1. à gauche, le taux de survie selon la classe, avec une barre par sexe (`sns.barplot` avec `hue`) ;
2. à droite, la distribution des âges des survivants et des non-survivants (`sns.histplot` avec `hue="survived"`).

Donnez un titre à chaque graphique et des noms d'axes en français.

Exercice 13.4 : Carte de chaleur de la saisonnalité ☆ ☆ ☆

Avec `donnees/flights.csv`, construisez un tableau croisé avec les années en colonnes, les mois en lignes et le nombre de passagers dans les cases, puis représentez-le avec `sns.heatmap`. Les mois doivent apparaître dans l'ordre du calendrier (et non par ordre alphabétique).

13.5 Bilan du chapitre

À retenir

- Le type de graphique dépend de la question : distribution (histogramme, boîtes), comparaison de catégories (barres), relation (nuage de points), évolution (courbe), corrélations (carte de chaleur).
- `matplotlib` : `fig, ax = plt.subplots()` puis `ax.plot`, `ax.scatter`, `ax.hist`, `ax.bar` ; `ax.set_title`, `ax.set_xlabel`, `ax.set_ylabel`, `ax.legend()` ; `fig.savefig("f.png")` puis `plt.show()`.
- `pandas` : `serie.plot(kind=...)` pour un coup d'œil rapide.
- `seaborn` : `sns.histplot`, `boxplot`, `barplot`, `countplot`, `scatterplot`, `lineplot`, `heatmap`, `pairplot`, avec `data=df`, `x="col"`, `y="col"`, `hue="col"` ; `ax=...` pour placer le graphique.

Faites le point : êtes-vous capable de…

- choisir un type de graphique adapté à une question ?
- tracer un graphique `matplotlib` avec titre, noms d'axes et légende, et l'enregistrer ?

- comparer la distribution d'une variable entre plusieurs groupes avec `seaborn` ?
- représenter la relation entre deux variables, colorée selon une troisième ?
- placer plusieurs graphiques côte à côte ?

CHAPTER 14

MODÉLISATION STATISTIQUE AVEC `STATSMODELS`

Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- ajuster un modèle de régression linéaire avec une **formule** (comme `lm` en R) et lire ses résultats ;
- intégrer des variables qualitatives et des interactions ;
- faire des prédictions et vérifier les résidus ;
- ajuster une régression logistique et interpréter des *odds ratios* ;
- réaliser les tests classiques (Student, khi-deux, ANOVA) avec `statsmodels` et `scipy.stats`.

Prérequis : chapitres 11 à 13, et des connaissances en statistique (régression, tests). **Durée indicative :** 2h.

Si vous avez déjà pratiqué la statistique avec R ou SAS, ce chapitre vous montre comment retrouver vos outils habituels en Python. La bibliothèque `statsmodels` est la plus proche de l'esprit de R et de SAS : elle produit des tableaux de résultats complets (coefficients, erreurs standard, tests, intervalles de confiance…) et permet de décrire les modèles avec des **formules** très proches de celles de R. Pour les tests statistiques élémentaires, on utilise aussi le module `stats` de la bibliothèque `scipy`.

Installation

`statsmodels` et `scipy` sont inclus dans Anaconda et dans Google Colab. Sinon : `pip install statsmodels scipy`.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.formula.api as smf
import statsmodels.api as sm
from scipy import stats

manchots = pd.read_csv("donnees/penguins.csv").dropna()
titanic = pd.read_csv("donnees/titanic.csv")
```

Par convention, on importe l'interface « formules » de `statsmodels` sous le nom `smf`, et l'interface générale sous le nom `sm`. Nous avons retiré d'emblée les manchots ayant des valeurs manquantes (`dropna()`), pour que tous les modèles portent sur les mêmes 333 individus.

14.1 Correspondances avec R et SAS

Objectif	R	SAS	Python
Régression linéaire	`lm(y ~ x1 + x2, data = d)`	`PROC REG` / `PROC GLM`	`smf.ols("y ~ x1 + x2", data=d).fit()`
Résultats détaillés	`summary(modele`	(sortie de la procédure)	`modele.summary()`
Variable qualitative	`factor(x)`	instruction `CLASS`	`C(x)` dans la formule
Régression logistique	`glm(y ~ x, family = binomial)`	`PROC LOGISTIC`	`smf.logit("y ~ x", data=d).fit()`
GLM (Poisson...)	`glm(..., family = poisson)`	`PROC GENMOD`	`smf.glm("y ~ x", data=d, family=sm.families.Poisson()).fit()`
Table d'ANOVA	`anova(modele)`	`PROC GLM`	`sm.stats.anova_lm(modele, typ=2)`
Test de Student	`t.test(x, y)`	`PROC TTEST`	`stats.ttest_ind(x, y)`
Test du khi-deux	`chisq.test(table)`	`PROC FREQ / CHISQ`	`stats.chi2_contingency(table)`
Corrélation	`cor.test(x, y)`	`PROC CORR`	`stats.pearsonr(x, y)`
Prédictions	`predict(modele newdata)`	`OUTPUT OUT= P=`	`modele.predict(nouvelles_donnees)`

14.2 La régression linéaire

14.2.1 Un premier modèle

Modélisons la masse d'un manchot en fonction de la longueur de sa nageoire. La syntaxe est presque celle de R : une formule `"y ~ x"` (sous forme de chaîne de caractères), le `DataFrame` passé en argument `data`, puis la méthode `fit()` qui ajuste le modèle :

```
modele = smf.ols("body_mass_g ~ flipper_length_mm", data=manchots).
↪fit()
print(modele.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:            body_mass_g   R-squared:                       ␣
↪       0.762
Model:                            OLS   Adj. R-squared:                  ␣
↪       0.761
Method:                 Least Squares   F-statistic:                     ␣
↪       1060.
Date:                Sat, 10 Oct 2026   Prob (F-statistic):              ␣
↪    3.13e-105
Time:                        14:12:41   Log-Likelihood:                  ␣
↪      -2461.1
No. Observations:                 333   AIC:                             ␣
↪       4926.
Df Residuals:                     331   BIC:                             ␣
↪       4934.
Df Model:                           1
Covariance Type:            nonrobust
==============================================================================
                     coef    std err          t      P>|t|      ␣
↪   [0.025      0.975]
------------------------------------------------------------------------------
↪---------------------
Intercept      -5872.0927    310.285    -18.925      0.000      ␣
↪-6482.472   -5261.713
flipper_length_mm    50.1533      1.540     32.562      0.000      ␣
↪  47.123      53.183
==============================================================================
Omnibus:                        5.922   Durbin-Watson:                   ␣
↪       2.102
Prob(Omnibus):                  0.052   Jarque-Bera (JB):                ␣
↪       5.876
Skew:                           0.325   Prob(JB):                        ␣
↪      0.0530
Kurtosis:                       3.025   Cond. No.                        ␣
↪     2.90e+03
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the␣
↪errors is correctly specified.
[2] The condition number is large, 2.9e+03. This might indicate␣
↪that there are
strong multicollinearity or other numerical problems.
```

`ols` signifie *Ordinary Least Squares* (moindres carrés ordinaires). Dans un notebook, écrire simplement `modele.summary()` (sans `print`) affiche les mêmes résultats sous forme de tableaux mis en forme. La sortie de `summary()` ressemble beaucoup à celle de R ou de SAS :

- en haut, les caractéristiques globales : nombre d'observations, R^2 (`R-squared`) et R^2 ajusté, statistique de Fisher et sa p-valeur, critères AIC et BIC ;
- au milieu, le tableau des coefficients : estimation (`coef`), erreur standard, statistique t, p-valeur (`P>|t|`) et intervalle de confiance à 95 % ;
- en bas, quelques diagnostics sur les résidus (normalité, autocorrélation).

Ici, chaque millimètre de nageoire supplémentaire est associé, en moyenne, à environ 50 g de masse en plus, et le modèle explique 76 % de la variance de la masse.

14.2.2 Accéder aux résultats

Comme en R (`coef(modele)`, `confint(modele)`...), chaque élément du résultat est accessible séparément, ce qui permet de l'utiliser dans la suite d'un programme :

```
print(modele.params)           # coefficients (une Series)
print(modele.rsquared)         # R²
print(modele.pvalues)          # p-valeurs
print(modele.conf_int())       # intervalles de confiance (un␣
 ↪DataFrame)
```

```
Intercept           -5872.092683
flipper_length_mm      50.153266
dtype: float64
0.7620921573403914
Intercept             1.183941e-54
flipper_length_mm    3.132836e-105
dtype: float64
                              0             1
Intercept          -6482.472236 -5261.713130
flipper_length_mm     47.123390    53.183142
```

Les attributs `modele.fittedvalues` et `modele.resid` contiennent les valeurs ajustées et les résidus, et `modele.nobs` le nombre d'observations utilisées.

14.2.3 Variables qualitatives et interactions

Les formules acceptent les mêmes constructions qu'en R :

Formule	Signification
`y ~ x1 + x2`	deux variables explicatives
`y ~ C(x)`	`x` traitée comme variable qualitative (codage par indicatrices, la première modalité servant de référence)
`y ~ x1 * x2`	effets principaux et interaction (`x1 + x2 + x1:x2`)
`y ~ x1:x2`	interaction seule
`y ~ np.log(x)`	transformation d'une variable
`y ~ I(x ** 2)`	calcul arithmétique dans la formule
`y ~ x - 1`	modèle sans constante

Les colonnes de texte sont automatiquement traitées comme qualitatives, mais il est plus lisible (et indispensable pour une variable codée par des nombres, comme une classe 1, 2, 3) d'écrire `C(...)` :

```
modele2 = smf.ols("body_mass_g ~ flipper_length_mm + C(species) + C(sex)", data=manchots).fit()
print(modele2.summary().tables[1])    # seulement le tableau des coefficients
```

```
==========================================================================================
                                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------------------
Intercept                  -365.8174    532.050     -0.688      0.492   -1412.479     680.844
C(species)[T.Chinstrap]     -87.6345     46.347     -1.891      0.060    -178.810       3.541
C(species)[T.Gentoo]        836.2600     85.185      9.817      0.000     668.681    1003.839
C(sex)[T.MALE]              530.3811     37.810     14.027      0.000     456.000     604.762
flipper_length_mm            20.0249      2.846      7.037      0.000      14.427      25.623
==========================================================================================
```

Le coefficient `C(sex)[T.MALE]` s'interprète ainsi : à longueur de nageoire et espèce égales, les mâles pèsent en moyenne 530 g de plus que les femelles (la modalité de référence, `FEMALE`, est la première par ordre alphabétique). Pour choisir une autre modalité de référence : `C(species, Treatment(reference="Gentoo"))`.

14.2.4 Prédire

La méthode `predict` calcule les prédictions pour de nouvelles observations, fournies sous la forme d'un `DataFrame` ayant les mêmes noms de colonnes que les données d'origine :

```
nouveaux = pd.DataFrame({
    "flipper_length_mm": [190, 220],
    "species": ["Adelie", "Gentoo"],
    "sex": ["FEMALE", "MALE"]
})
modele2.predict(nouveaux)
```

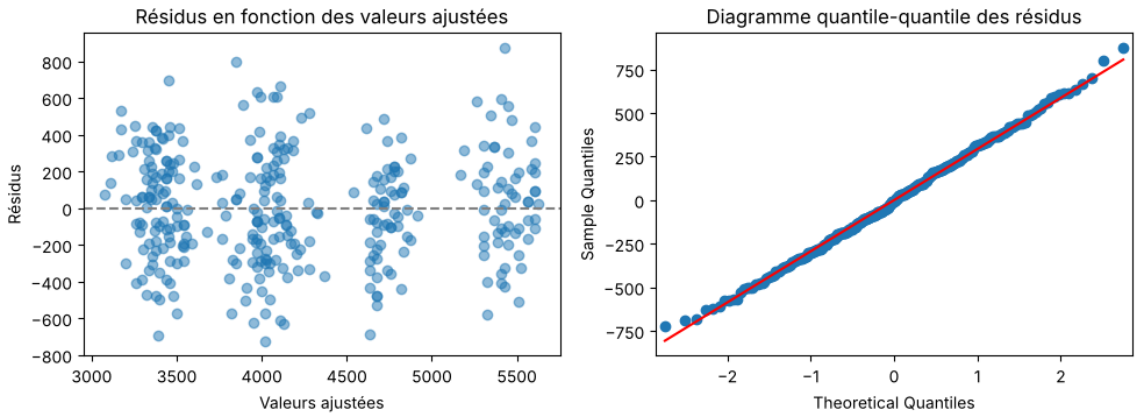
```
0    3438.916482
1    5406.305048
dtype: float64
```

Pour obtenir aussi les intervalles de confiance et de prédiction : `modele2.get_prediction(nouveaux).summary_frame(alpha=0.05)`.

14.2.5 Vérifier les hypothèses

Comme en R avec `plot(modele)`, on vérifie l'absence de structure dans les résidus et leur normalité approximative :

```
fig, axes = plt.subplots(1, 2, figsize=(10, 3.8))
axes[0].scatter(modele2.fittedvalues, modele2.resid, alpha=0.5)
axes[0].axhline(0, color="grey", linestyle="--")
axes[0].set_xlabel("Valeurs ajustées")
axes[0].set_ylabel("Résidus")
axes[0].set_title("Résidus en fonction des valeurs ajustées")
sm.qqplot(modele2.resid, line="s", ax=axes[1])
axes[1].set_title("Diagramme quantile-quantile des résidus")
fig.tight_layout()
plt.show()
```



14.2.6 Comparer des modèles et tables d'ANOVA

La fonction `sm.stats.anova_lm` produit des tables d'analyse de la variance (avec `typ=2` ou `typ=3` pour les sommes des carrés de type II ou III, comme dans `PROC GLM` ou `car::Anova` en R) :

```
sm.stats.anova_lm(modele2, typ=2)
```

```
                        sum_sq     df           F        PR(>F)
C(species)        1.314181e+07    2.0   75.217723  1.295035e-27
C(sex)            1.718958e+07    1.0  196.770645  2.414442e-35
flipper_length_mm  4.325617e+06    1.0   49.515732  1.154539e-11
Residual           2.865357e+07  328.0         NaN           NaN
```

Avec deux modèles emboîtés, elle réalise le test de Fisher de comparaison (comme `anova(m1, m2)` en R) :

```
sm.stats.anova_lm(modele, modele2)
```

```
   df_resid           ssr  df_diff       ss_diff           F  \
0     331.0  5.121196e+07      0.0           NaN         NaN
1     328.0  2.865357e+07      3.0  2.255839e+07   86.075999

         Pr(>F)
0           NaN
1  4.215933e-41
```

Une ANOVA à un facteur n'est rien d'autre qu'une régression sur une variable qualitative :

```
anova_espece = smf.ols("body_mass_g ~ C(species)", data=manchots).
↪fit()
sm.stats.anova_lm(anova_espece, typ=2)
```

```
                  sum_sq     df           F        PR(>F)
C(species)  1.451902e+08    2.0  341.894895  3.744505e-81
Residual    7.006945e+07  330.0         NaN           NaN
```

Si vous venez de R

- Ne pas oublier `.fit()` : `smf.ols(...)` seul ne fait que **définir** le modèle.
- Comme `lm` par défaut, `statsmodels` supprime les lignes ayant des valeurs manquantes dans les variables du modèle : vérifiez toujours `modele.nobs`.
- Il existe aussi une interface sans formule, `sm.OLS(y, X)`, plus proche de la notation matricielle ; elle n'ajoute **pas** de constante automatiquement (il faut utiliser `sm.add_constant(X)`). Les formules sont plus simples pour commencer.
- Pour les modèles mixtes (`lme4`), voir `smf.mixedlm` ; pour l'apprentissage automatique (forêts aléatoires, validation croisée...), la bibliothèque de référence est `scikit-learn`.

14.3 La régression logistique

Pour une variable à expliquer binaire (0 / 1), on utilise la régression logistique, avec `smf.logit` (ou, de façon équivalente, `smf.glm(..., family=sm.families.Binomial())`). Modélisons la survie des passagers du Titanic :

```
logit = smf.logit("survived ~ C(sex) + C(pclass) + age", data=titanic).fit()
print(logit.summary())
```

```
Optimization terminated successfully.
         Current function value: 0.453279
         Iterations 6
                           Logit Regression Results
==============================================================================
Dep. Variable:                survived   No. Observations:                  714
Model:                           Logit   Df Residuals:                      709
Method:                            MLE   Df Model:                            4
Date:                 Sat, 10 Oct 2026   Pseudo R-squ.:                 0.3289
Time:                         14:12:42   Log-Likelihood:                -323.64
converged:                       True   LL-Null:                       -482.26
Covariance Type:            nonrobust   LLR p-value:                 2.074e-67
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
Intercept      3.7770      0.401      9.416      0.000       2.991       4.563
C(sex)[T.male]    -2.5228      0.207    -12.164      0.000      -2.929      -2.116
C(pclass)[T.2]    -1.3098      0.278     -4.710      0.000      -1.855      -0.765
C(pclass)[T.3]    -2.5806      0.281     -9.169      0.000      -3.132      -2.029
age               -0.0370      0.008     -4.831      0.000      -0.052      -0.022
==============================================================================
```

L'argument `disp=0` de `fit()` supprime le message sur la convergence de l'algorithme. Remarquez le nombre d'observations : 714, car les 177 passagers sans âge ont été écartés.

Les coefficients sont exprimés sur l'échelle du logarithme de la cote (*log-odds*). Pour obtenir les ***odds ratios***, comme dans la sortie de `PROC LOGISTIC`, on prend leur exponentielle, ainsi que celle des bornes des intervalles de confiance :

```
odds_ratios = pd.DataFrame({
    "OR": np.exp(logit.params),
```

(suite sur la page suivante)

(suite de la page précédente)

```
    "IC 2.5 %": np.exp(logit.conf_int()[0]),
    "IC 97.5 %": np.exp(logit.conf_int()[1]),
}).round(3)
odds_ratios
```

```
                    OR  IC 2.5 %  IC 97.5 %
Intercept       43.685    19.902    95.890
C(sex)[T.male]   0.080     0.053     0.120
C(pclass)[T.2]   0.270     0.156     0.465
C(pclass)[T.3]   0.076     0.044     0.131
age              0.964     0.949     0.978
```

À classe et âge égaux, la cote de survie d'un homme est environ 12 fois plus faible que celle d'une femme (*odds ratio* de 0,08) ; chaque année d'âge supplémentaire multiplie la cote par 0,96.

La méthode `predict` donne ici des **probabilités** :

```
passagers = pd.DataFrame({"sex": ["female", "male"], "pclass": [1,
 ↪ 3], "age": [30, 30]})
logit.predict(passagers).round(3)
```

```
0    0.935
1    0.080
dtype: float64
```

14.4 Les tests statistiques classiques

Le module `scipy.stats` fournit les tests usuels. Ils prennent en argument des `Series` (ou des tableaux) et retournent un résultat contenant la statistique et la *p*-valeur.

14.4.1 Comparer deux moyennes : test de Student

Les mâles Adélie sont-ils plus lourds que les femelles ?

```
adelie = manchots[manchots["species"] == "Adelie"]
males = adelie.loc[adelie["sex"] == "MALE", "body_mass_g"]
femelles = adelie.loc[adelie["sex"] == "FEMALE", "body_mass_g"]

resultat = stats.ttest_ind(males, femelles, equal_var=False)    #␣
 ↪test de Welch
print(resultat.statistic, resultat.pvalue)
```

```
13.126285923485874 6.402319748031804e-26
```

`equal_var=False` correspond au test de Welch (le défaut de `t.test` en R) ; avec `equal_var=True`, on obtient le test de Student classique. Pour des échantillons appariés, on utilise `stats.ttest_rel`, et pour un test non paramétrique, `stats.mannwhitneyu`.

14.4.2 Indépendance de deux variables qualitatives : test du khi-deux

La survie est-elle indépendante du sexe ?

```
tableau = pd.crosstab(titanic["sex"], titanic["survived"])
khi2, p, ddl, attendus = stats.chi2_contingency(tableau)
print(tableau)
print(f"khi-deux = {khi2:.1f}, ddl = {ddl}, p-valeur = {p:.2e}")
```

```
survived    0    1
sex
female     81  233
male      468  109
khi-deux = 260.7, ddl = 1, p-valeur = 1.20e-58
```

`chi2_contingency` retourne quatre valeurs (statistique, *p*-valeur, degrés de liberté et effectifs attendus sous l'hypothèse d'indépendance), que l'on récupère par **déballage**, comme un *tuple* (chapitre 7). Remarquez qu'il applique par défaut la correction de continuité de Yates pour les tableaux 2 × 2, comme `chisq.test` en R.

14.4.3 Corrélation

```
r, p = stats.pearsonr(manchots["flipper_length_mm"], manchots[
 ↪"body_mass_g"])
print(f"r = {r:.3f}, p-valeur = {p:.2e}")
```

```
r = 0.873, p-valeur = 3.13e-105
```

`stats.spearmanr` calcule la corrélation des rangs de Spearman.

Testez-vous

1. Quelle formule `statsmodels` correspond au modèle R `lm(y ~ factor(groupe) + age, data = d)` ?
2. Que se passe-t-il si l'on écrit `modele = smf.ols("y ~ x", data=d)` puis `modele.params` ?
3. Comment obtenir l'"*odds ratio* associé à une variable dans un modèle `logit` ?

14.5 Exercices

Rappel

Ces exercices sont à faire dans un notebook, avec `pandas`, `numpy`, `statsmodels.formula.api as smf`, `statsmodels.api as sm` et `scipy.stats` importés.

Exercice 14.1 : Le bec des manchots ☆

Avec les manchots (sans valeurs manquantes) :

1. ajustez la régression de l'épaisseur du bec (`bill_depth_mm`) sur sa longueur (`bill_length_mm`). Quel est le signe de la pente ?
2. ajoutez l'espèce comme variable explicative. Que devient le signe de la pente ? Comment l'expliquer (pensez au nuage de points coloré par espèce du chapitre 13) ?

Exercice 14.2 : Interaction ☆ ☆

Le lien entre la longueur de la nageoire et la masse est-il le même pour les trois espèces ? Ajustez un modèle avec interaction entre `flipper_length_mm` et `species`, puis comparez-le au modèle sans interaction avec un test de Fisher.

Exercice 14.3 : Survie et tarif ☆ ☆

1. Ajustez une régression logistique de la survie sur le sexe, la classe et le prix du billet (`fare`). Le prix du billet a-t-il un effet significatif une fois la classe prise en compte ?
2. Remplacez `fare` par `np.log1p(fare)` (le logarithme de 1 + prix, car certains billets valent 0). Que constatez-vous ?
3. Calculez la probabilité de survie prédite par le premier modèle pour un homme de 3e classe ayant payé 8 et pour une femme de 1re classe ayant payé 80.

Exercice 14.4 : Tests ☆ ☆

1. La longueur moyenne de la nageoire diffère-t-elle entre les manchots Adélie de l'île Dream et ceux de l'île Biscoe ? (Test de Welch.)
2. La survie sur le Titanic est-elle indépendante de la classe ? (Test du khi-deux.)
3. Retrouvez le résultat de la question 2 avec une régression logistique et un test du rapport de vraisemblance (attribut `llr_pvalue` du modèle).

14.6 Bilan du chapitre

À retenir

- `import statsmodels.formula.api as smf;modele = smf.ols("y ~ x1 + C(x2)", data=df).fit();modele.summary()`.
- Résultats : `params`, `pvalues`, `conf_int()`, `rsquared`, `resid`, `fittedvalues`, `nobs` ; prédictions : `modele.predict(nouveau_df)`.
- Formules : `C(x)` (qualitative), `x1 * x2` (interaction), `np.log(x)`, `I(x ** 2)`.

- ANOVA : `sm.stats.anova_lm(modele, typ=2)` ; comparaison de modèles emboîtés : `sm.stats.anova_lm(m1, m2)`.
- Logistique : `smf.logit("y ~ ...", data=df).fit()` ; *odds ratios* : `np.exp(modele.params)`.
- Tests : `stats.ttest_ind`, `stats.chi2_contingency`, `stats.pearsonr` (module `scipy.stats`).

Faites le point : êtes-vous capable de...

- traduire un modèle `lm` ou `glm` de R (ou une `PROC REG` / `PROC LOGISTIC`) en Python ?
- lire le tableau de résultats d'un modèle et en extraire les coefficients et les p-valeurs ?
- inclure une variable qualitative et une interaction dans un modèle ?
- vérifier les résidus d'une régression linéaire ?
- réaliser un test de Student ou du khi-deux ?

CHAPTER 15

MINI-PROJETS

Vous avez maintenant tous les outils pour mener une petite analyse de données de bout en bout. Les mini-projets de ce chapitre vous proposent de les mettre en pratique sur des problèmes plus longs, qui mobilisent plusieurs chapitres à la fois.

Comment aborder un projet ?

- **Lisez tout l'énoncé** avant de commencer.
- **Avancez étape par étape**, en vérifiant chaque résultat intermédiaire (affichez `head()`, `shape`, les valeurs obtenues…) avant de passer à la suite.
- Travaillez dans un **notebook** : alternez cellules de code et cellules de texte (*Markdown*) dans lesquelles vous écrivez vos observations et vos conclusions. Un bon notebook d'analyse se lit comme un petit rapport.
- Ne regardez la solution qu'à la toute fin… et sachez qu'il existe beaucoup de solutions correctes !

Projet	Chapitres mobilisés	Difficulté
1. Bulletins de notes en Python « pur »	4, 5, 7, 9	☆ ☆
2. Les manchots de l'archipel Palmer	11 à 14	☆ ☆
3. Qui a survécu au naufrage du Titanic ?	11 à 14	☆ ☆ ☆

15.1 Projet 1 : Bulletins de notes en Python « pur » ☆ ☆

Chapitres mobilisés : 4, 5, 7 et 9.

Ce premier projet n'utilise pas `pandas` : il vous fait pratiquer les bases (fonctions, listes, dictionnaires, fichiers) sur lesquelles tout le reste repose. Vous pourrez ensuite comparer avec la version `pandas` (étape 8).

Le fichier `donnees/notes.csv` contient les notes de 8 étudiants dans trois matières. On souhaite produire des statistiques par matière et un bulletin pour chaque étudiant.

```
nom;prenom;maths;info;anglais
Martin;Aude;14;17;12
Benali;Yanis;9;12;15
Le Goff;Maëlle;16;13;18
Dupont;Hugo;7;10;9
Nguyen;Linh;12;19;14
Kerbrat;Erwan;11;8;13
Diallo;Fatou;15;14;16
Moreau;Inès;10;11;8
```

15.1.1 Étapes

1. Écrivez une fonction `charge_notes(nom_fichier)` qui lit le fichier et retourne une **liste de dictionnaires**, un par étudiant, dans lesquels les notes ont été converties en nombres. Par exemple, le premier élément doit être `{"nom": "Martin", "prenom": "Aude", "maths": 14.0, "info": 17.0, "anglais": 12.0}`.
2. Écrivez une fonction `moyenne_etudiant(etudiant, matieres)` qui retourne la moyenne d'un étudiant (un dictionnaire) sur une liste de matières.
3. Écrivez une fonction `stats_matiere(etudiants, matiere)` qui retourne un *tuple* `(minimum, moyenne, maximum)` des notes d'une matière.
4. Écrivez une fonction `mention(moyenne)` qui retourne la mention correspondant à une moyenne (voir l'exercice 2.3).
5. Écrivez une fonction `rang(etudiants, matieres)` qui retourne la liste des noms complets des étudiants (prénom et nom), classés par moyenne décroissante.
6. Écrivez une fonction `ecrit_bulletins(etudiants, matieres, nom_fichier)` qui produit un fichier texte contenant un bulletin pour chaque étudiant, de la forme :

```
=== Bulletin de Aude Martin ===
maths   : 14.0
info    : 17.0
anglais : 12.0
Moyenne : 14.33 (Bien)
Rang    : 4 / 8
```

7. **Bonus** : les matières n'ont pas toutes la même importance. Ajoutez des coefficients (stockés dans un dictionnaire `{"maths": 3, "info": 3, "anglais": 1}`) au calcul de la moyenne.

8. **Pour comparer** : refaites les étapes 1 à 3 avec `pandas` (chapitres 11 et 12). Combien de lignes de code faut-il ?

15.2 Projet 2 : Les manchots de l'archipel Palmer ☆ ☆

Chapitres mobilisés : 11 à 14.

On souhaite réaliser une étude descriptive du jeu de données `donnees/penguins.csv` (présenté au chapitre 11), puis répondre à une question : **peut-on reconnaître l'espèce d'un manchot à partir des seules mesures de son bec ?**

15.2.1 Étapes

1. **Charger et décrire** : chargez les données, affichez leur taille, les types des colonnes et le nombre de valeurs manquantes par colonne.
2. **Nettoyer** : supprimez les lignes pour lesquelles les mesures du bec sont manquantes. Combien de manchots reste-t-il ?
3. **Résumer** : construisez un tableau donnant, pour chaque espèce, le nombre d'individus et la moyenne de chacune des quatre mesures, arrondie à une décimale.
4. **Visualiser** : tracez le nuage de points longueur du bec / épaisseur du bec, coloré par espèce. Décrivez les trois groupes que vous observez.
5. **Une règle de décision** : à partir du graphique, proposez une règle simple, sous la forme d'une fonction `devine_espece(longueur, epaisseur)` qui retourne `"Adelie"`, `"Chinstrap"` ou `"Gentoo"` à l'aide de deux seuils (par exemple : « si le bec est court, c'est un Adélie ; sinon, s'il est fin, c'est un Gentoo ; sinon c'est un Chinstrap »).
6. **Évaluer la règle** : ajoutez au tableau une colonne `prediction` contenant l'espèce devinée par votre fonction pour chaque manchot, puis calculez la proportion de manchots bien classés. Affichez le tableau croisé espèce réelle / espèce prédite : quelles erreurs votre règle commet-elle ?
7. **Modéliser la masse** : ajustez une régression linéaire de la masse sur la longueur de la nageoire, l'espèce et le sexe (chapitre 14). Quel est, à nageoire et espèce égales, l'écart de masse moyen entre mâles et femelles ? Vérifiez les résidus.
8. **Bonus** : essayez d'autres seuils pour améliorer votre taux de bonne classification. Est-ce une bonne idée de chercher les seuils « parfaits » pour ces données précises ? (Pensez à ce qui se passerait sur de nouveaux manchots.)

15.3 Projet 3 : Qui a survécu au naufrage du Titanic ? ☆ ☆ ☆

Chapitres mobilisés : 11 à 14.

Le 15 avril 1912, le Titanic sombre lors de son voyage inaugural. Le fichier `donnees/titanic.csv` décrit 891 de ses passagers (voir l'exercice 11.5). L'objectif de ce projet est de produire une petite **étude**, sous la forme d'un notebook commenté, répondant à la question : **quels facteurs ont influencé les chances de survie ?**

15.3.1 Étapes

1. **Découvrir les données** : taille, types, valeurs manquantes, statistiques descriptives. Quelle est la proportion de survivants ?
2. **Préparer les données** :
   - supprimez la colonne `deck`, trop incomplète ;
   - complétez les âges manquants par l'âge médian de la classe du passager (exercice 12.2) ;
   - créez une colonne `taille_famille` égale à `sibsp + parch + 1` (le passager, plus ses frères, sœurs et conjoint·e, plus ses parents et enfants à bord) ;
   - créez une colonne `tranche_age` (exercice 12.3).
3. **Analyser** : calculez le taux de survie selon le sexe, la classe, la tranche d'âge, le port d'embarquement et la taille de la famille. Présentez chaque résultat dans un petit tableau trié.
4. **Visualiser** : produisez une figure de 4 graphiques (une grille 2×2) qui résume vos principaux résultats.
5. **Modéliser** : ajustez une régression logistique de la survie sur le sexe, la classe, l'âge (complété) et la taille de la famille (chapitre 14). Présentez les *odds ratios* et leurs intervalles de confiance. L'effet de la classe persiste-t-il à sexe et âge égaux ?
6. **Conclure** : rédigez, dans une cellule de texte, une conclusion de 5 à 10 lignes. Attention à ne pas confondre **corrélation** et **causalité** : par exemple, les passagers embarqués à Cherbourg ont-ils survécu davantage *parce qu'ils* ont embarqué à Cherbourg ? (Indice : regardez la répartition des classes selon le port.)
7. **Exporter** : enregistrez le tableau préparé à l'étape 2 dans un fichier `titanic_prepare.csv`.

CHAPTER 16

FICHE MÉMO

Cette fiche rassemble la syntaxe essentielle vue dans ce document. Elle ne remplace pas les chapitres, mais vous permet de retrouver rapidement une notation.

16.1 Bases (chapitres 1 à 4)

```
# Variables, types, conversions
x = 12                  # int
y = 3.5                 # float
nom = "Aude"            # str
ok = True               # bool (True / False), et la valeur␣
 ↪spéciale None
type(x)                 # <class 'int'>
int("12"), float("3.5"), str(12)

# Opérateurs
7 + 2, 7 - 2, 7 * 2, 7 / 2    # 9, 5, 14, 3.5
7 // 2, 7 % 2, 7 ** 2         # 3 (quotient), 1 (reste), 49␣
 ↪(puissance)
x += 1                        # x = x + 1

# Affichage
print("x vaut", x)
print(f"{nom} a {x} ans, prix : {y:.2f} euros")

# Comparaisons et logique
x == 12, x != 12, x < 12, x >= 12, x in [1, 12]
(x > 0) and (x < 20), (x < 0) or (x > 20), not ok

# Conditions
if x > 0:
    print("positif")
elif x < 0:
    print("négatif")
```

(suite sur la page suivante)

(suite de la page précédente)

```
else:
    print("nul")

# Boucles
while x > 0:
    x = x - 3
for i in range(5):              # 0, 1, 2, 3, 4
    print(i)
for i in range(2, 10, 2):       # 2, 4, 6, 8
    print(i)

# Fonctions
def aire(largeur, longueur=1):
    """Retourne l'aire d'un rectangle."""
    return largeur * longueur

a = aire(3, 4)
a = aire(longueur=4, largeur=3)

# Modules
import math
import random
import numpy as np
math.sqrt(2), math.pi, random.randint(1, 6)
```

16.2 Listes (chapitre 5)

```
l = [1, 5, 7]
len(l), l[0], l[-1], l[1:3], l[:2], l[2:]
l[0] = 2                            # modification
l.append(4)                         # ajout à la fin
l.insert(0, 9)                      # insertion à l'indice 0
del l[0]                            # suppression par indice
l.remove(7)                         # suppression de la première␣
↪occurrence de 7
5 in l, l.index(5)                  # recherche
l + [8, 9], 3 * [0]                 # concaténation, répétition
sorted(l), sorted(l, reverse=True)
sum(l), min(l), max(l)
copie = list(l)                     # vraie copie (et non copie = l)

for elem in l: ...                  # parcours des éléments
for i in range(len(l)): ...         # parcours des indices
for i, elem in enumerate(l): ...    # indices et éléments
for a, b in zip(l1, l2): ...        # deux listes en parallèle

carres = [i ** 2 for i in range(10) if i % 2 == 0]   # liste en␣
↪compréhension
```

16.3 Chaînes de caractères (chapitre 6)

```
s = "Bonjour"
len(s), s[0], s[-1], s[1:4], "jour" in s
s.lower(), s.upper(), s.strip(), s.replace("o", "0")
"a;b;c".split(";")          # ['a', 'b', 'c']
"-".join(["a", "b", "c"])   # 'a-b-c'
s.count("o"), s.find("j"), s.startswith("Bon"), s.endswith("!")
f"{3.14159:.2f}", f"{42:5d}", f"{'ab':<6}|", f"{'ab':>6}|"
```

16.4 Dictionnaires, *tuples*, *sets* (chapitre 7)

```
d = {"pomme": 3, "kiwi": 12}
d["pomme"], d.get("banane", 0), "kiwi" in d
d["banane"] = 6                  # ajout ou modification
del d["kiwi"]
for cle in d.keys(): ...
for valeur in d.values(): ...
for cle, valeur in d.items(): ...
d[mot] = d.get(mot, 0) + 1       # compter des occurrences

t = (3, 7)                       # tuple (non modifiable)
x, y = t                         # déballage
a, b = b, a                      # échange

e = {1, 2, 3}                    # set (sans ordre ni doublon) ;␣
↪set() pour un set vide
e.add(4), e.remove(1)
set(liste)                       # supprimer les doublons
e1 | e2, e1 & e2, e1 - e2        # union, intersection, différence
```

16.5 Tests (chapitre 8)

```
def test_ma_fonction():
    assert ma_fonction(2) == 4
    assert ma_fonction(0) == 0           # cas limite
    assert abs(ma_fonction(0.1) - 0.2) < 1e-9   # résultat à␣
↪virgule
    print("OK")
```

16.6 Fichiers (chapitre 9)

```
import os, csv, json

os.listdir("donnees"), os.path.join("donnees", "notes.csv")

with open("texte.txt", "r", encoding="utf-8") as fp:       # "r"␣
 ↪lire, "w" écrire, "a" ajouter
    for ligne in fp.readlines():
        ligne = ligne.strip()

with open("notes.csv", "r", encoding="utf-8") as fp:
    for ligne in csv.DictReader(fp, delimiter=";"):      # ou csv.
 ↪reader : une liste par ligne
        note = float(ligne["maths"])

with open("sortie.txt", "w", encoding="utf-8", newline="\n") as fp:
    fp.write("texte\n")
    csv.writer(fp, delimiter=";").writerow([1, 2, 3])

with open("donnees.json", "r", encoding="utf-8") as fp:
    donnees = json.load(fp)                    # json.loads(chaine)␣
 ↪pour une chaîne
with open("sortie.json", "w", encoding="utf-8") as fp:
    json.dump(donnees, fp, indent=2)
```

16.7 numpy (chapitre 10)

```
import numpy as np
A = np.array([[0, 1], [2, 3]])
A.shape, A.ndim, A.dtype, A.T
A[1, 0], A[:, 0], A[0, :], A[0:2, 1:]
A * A, A ** 2, A @ A                           # élément par élément␣
 ↪vs produit matriciel
np.zeros((2, 3)), np.ones((2, 3)), np.eye(2), np.arange(10), np.
 ↪linspace(0, 1, 11)
np.arange(10).reshape((5, 2))
A.sum(), A.mean(), A.std(), A.min(), A.max()
A.mean(axis=0)                                 # un résultat par␣
 ↪colonne
A.mean(axis=1)                                 # un résultat par ligne
A > 1, np.sum(A > 1), A[A > 1]                 # masques
```

16.8 pandas (chapitres 11 et 12)

```
import pandas as pd
df = pd.read_csv("donnees.csv", sep=";", decimal=",")    # encoding=
 ↪"iso-8859-1" si besoin

# Explorer
df.head(), df.shape, df.columns, df.dtypes
df.info(), df.describe()
df["col"].value_counts(), df["col"].unique(), df["col"].nunique()
df["col"].mean(), df["col"].median(), df["col"].std(), df["col"].
 ↪min()

# Sélectionner
df["col"]                                    # une colonne (Series)
df[["col1", "col2"]]                         # plusieurs colonnes␣
 ↪(DataFrame)
df[df["age"] > 18]                           # filtrer des lignes
df[(df["age"] > 18) & (df["sexe"] == "F")]   # & (et), | (ou), ~␣
 ↪(non), parenthèses !
df[df["ville"].isin(["Rennes", "Brest"])]
df.loc[df["age"] > 18, ["nom", "age"]]       # par noms (fin incluse)
df.iloc[0:5, 0:2]                            # par positions (fin␣
 ↪exclue)
df.sort_values("col", ascending=False)

# Transformer
df["c"] = df["a"] / df["b"]
df["cat"] = df["code"].map({"F": "femme", "H": "homme"})
df["classe"] = pd.cut(df["age"], bins=[0, 18, 60, 120], labels=[
 ↪"jeune", "adulte", "senior"])
df["d"] = df["col"].apply(ma_fonction)
df = df.rename(columns={"ancien": "nouveau"})
df = df.drop(columns=["inutile"])
df["col"].str.lower(), df["col"].str.contains("motif")
df["date"] = pd.to_datetime(df["date"], format="%d/%m/%Y");  df[
 ↪"date"].dt.year

# Valeurs manquantes
df.isna().sum()
df = df.dropna(subset=["col"])
df["col"] = df["col"].fillna(df["col"].median())

# Résumer par groupe
df.groupby("cat")["num"].mean()
df.groupby(["cat1", "cat2"])["num"].agg(["count", "mean"])
df.groupby("cat").agg(effectif=("num", "count"), moyenne=("num",
 ↪"mean"))
pd.crosstab(df["cat1"], df["cat2"], normalize="index")

# Combiner et enregistrer
pd.merge(df1, df2, on="cle", how="left")
pd.concat([df1, df2])
df.to_csv("resultat.csv", index=False)
```

16.9 Visualisation (chapitre 13)

```
import matplotlib.pyplot as plt
import seaborn as sns

fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(x, y, marker="o", label="série")     # courbe
ax.scatter(x, y)                             # nuage de points
ax.hist(valeurs, bins=20)                    # histogramme
ax.bar(categories, hauteurs)                 # barres
ax.set_title("Titre"); ax.set_xlabel("x (unité)"); ax.set_ylabel(
 ↪"y (unité)"); ax.legend()
fig.savefig("graphique.png", dpi=150)        # AVANT plt.show()
plt.show()

fig, axes = plt.subplots(1, 2, figsize=(10, 4))    # plusieurs␣
 ↪graphiques : axes[0], axes[1]

df["col"].plot(kind="hist")                  # raccourci pandas

sns.histplot(data=df, x="num", hue="cat")
sns.boxplot(data=df, x="cat", y="num")
sns.barplot(data=df, x="cat", y="num")       # moyenne par catégorie
sns.countplot(data=df, x="cat")
sns.scatterplot(data=df, x="num1", y="num2", hue="cat")
sns.lineplot(data=df, x="date", y="num")
sns.heatmap(df.select_dtypes("number").corr(), annot=True, cmap=
 ↪"RdBu_r", vmin=-1, vmax=1)
sns.pairplot(data=df, hue="cat")
```

16.10 Modélisation (chapitre 14)

```
import statsmodels.formula.api as smf
import statsmodels.api as sm
from scipy import stats

m = smf.ols("y ~ x1 + C(cat) + x1:x2", data=df).fit()       # lm(y ~␣
 ↪x1 + factor(cat) + x1:x2)
m.summary(), m.params, m.pvalues, m.conf_int(), m.rsquared, m.resid
m.predict(nouveau_df)
sm.stats.anova_lm(m, typ=2)                                 # table d
 ↪'ANOVA
sm.stats.anova_lm(m_simple, m_complet)                      #␣
 ↪comparaison de modèles emboîtés

lg = smf.logit("y01 ~ x + C(cat)", data=df).fit()           # glm(...
 ↪, family = binomial)
np.exp(lg.params)                                           # odds␣
 ↪ratios

stats.ttest_ind(a, b, equal_var=False)                      # t.test␣
 ↪(Welch)
stats.chi2_contingency(pd.crosstab(df["a"], df["b"]))      # chisq.
```

(suite sur la page suivante)

(suite de la page précédente)

```
 ↪test
stats.pearsonr(x, y)                                    # cor.
 ↪test
```

16.11 Correspondances R / SAS / Python

R	SAS	Python
`x <- 5`	`x = 5;`	`x = 5`
`x[1]` (premier élément)		`x[0]`
`x[-1]` (tout sauf le premier)		`x[1:]` (et `x[-1]` = le dernier !)
`1:10`	`DO i = 1 TO 10;`	`range(1, 11)`
`function(x) { ... }`	`%macro`	`def f(x): ... return ...`
`library(pkg)`		`import pkg`
`read.csv`	`PROC IMPORT`	`pd.read_csv`
`summary(df)`	`PROC MEANS`	`df.describe()`
`table(x)`	`PROC FREQ`	`df["x"].value_counts()`
`filter, select, arrange`	`WHERE,KEEP,PROC SORT`	`df[masque], df[["x", "y"]], df.sort_values("x")`
`mutate`	étape DATA	`df["z"] = ...`
`group_by + summarise`	`PROC MEANS; CLASS`	`df.groupby("g")["x"].mean()`
`left_join / merge`	`MERGE; BY`	`pd.merge(a, b, on="id", how="left")`
`ggplot2`	`PROC SGPLOT`	`seaborn` (ou `plotnine`)
`lm, glm`	`PROC REG, PROC LOGISTIC`	`smf.ols, smf.logit, smf.glm`
`NA, is.na`	`., MISSING`	`NaN, isna()`

16.12 Les erreurs les plus courantes

Erreur	Cause probable
`SyntaxError`	`:` ou parenthèse ou guillemet oublié ; `=` au lieu de `==`
`IndentationError`	décalage manquant ou incohérent
`NameError`	faute de frappe, variable non définie, module non importé
`TypeError`	opération entre types incompatibles (`"12" + 2`), mauvais nombre d'arguments
`ValueError`	conversion impossible (`int("douze")`)
`IndexError`	indice trop grand (`l[len(l)]`)
`KeyError`	clé absente d'un dictionnaire, ou nom de colonne inexistant dans un `DataFrame`
`AttributeError`	méthode ou attribut inexistant (`None.append`...)
`ZeroDivisionError`	division par zéro
`FileNotFoundError`	mauvais nom de fichier ou mauvais dossier
`ValueError: The truth value of a Series is ambiguous`	`and` / `or` utilisés au lieu de `&` / `|` pour combiner des filtres `pandas`

CHAPTER 17

ET APRÈS ?

Félicitations, vous êtes arrivé·e au bout de ce document ! Vous savez maintenant écrire des programmes Python, et charger, préparer, résumer, visualiser et modéliser un jeu de données. Pour continuer à progresser en analyse de données, la clé est de **pratiquer régulièrement sur de vraies données**. Voici quelques pistes.

- **Analysez des données qui vous intéressent.** Le portail data.gouv.fr rassemble des milliers de jeux de données publics français (élections, transports, santé, environnement…), généralement au format CSV : choisissez-en un qui vous intéresse et posez-lui des questions.
- **Approfondissez la modélisation** avec la documentation de `statsmodels` : modèles linéaires généralisés, modèles mixtes (`mixedlm`), séries temporelles (ARIMA), modèles de survie…
- **Découvrez l'apprentissage automatique** (*machine learning*) avec la bibliothèque `scikit-learn`, dont la documentation contient de nombreux tutoriels : c'est la suite naturelle du projet sur les manchots.
- **Approfondissez `pandas`** grâce à sa documentation officielle, en particulier les guides sur le regroupement (`groupby`), la restructuration des tableaux (`pivot`, `melt`) et les séries temporelles.
- **Entraînez-vous à programmer** : le site France-IOI propose des centaines d'exercices de programmation corrigés automatiquement, classés par niveau, et le tutoriel officiel de Python, en français, approfondit de nombreux points abordés ici.

Un dernier conseil

Quand vous êtes bloqué·e, pensez à la démarche du chapitre 8 : lire le message d'erreur, observer ce que fait réellement le programme (en affichant `head()`, `shape`, `dtypes`… pour un `DataFrame`), comparer avec ce que vous attendiez. Et n'oubliez pas que les meilleurs analystes passent, eux aussi, une bonne partie de leur temps à chercher des *bugs* et à nettoyer des données !

Ce document se veut évolutif. N'hésitez donc pas à faire vos remarques à son auteur, dont vous trouverez le contact sur sa page web.